



National University of Computer and Emerging Sciences, Lahore



LLM-Ranker: Retrieval-Style Ranking of LLMs

Muhammad Umer 22L-6793 BS(CS)

Fatima Imran 22L-6946 BS(CS)

Hira Ikhlāq 21L-5451 BS(CS)

Supervisor: Dr. Maryam Bashir

Final Year Project

April 26, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: LLM-Ranker: Retrieval-Style Ranking of LLMs

is the sole contribution of the author(s), and no part hereof has been reproduced, as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I take responsibility for any liable consequence and rule breaking of this declaration.

Date: 21st April 2026

Name: Muhammad Umer

Signature: 

Name: Fatima Imran

Signature: 

Name: Hira Ikhlq

Signature: 

Author's Declaration

The work presented in this report is our own and we declare that it has not been used submitted or shown in any other institution or organization before.

Abstract

Since the introduction of Large Language Models (LLMs), the quality of their performance has become a rather burning issue among researchers and developers, both on an international scale. More recent models of LLMs, including GPT, Claude and LLaMA, have shown significant ability to produce human-like text; however, their outputs often have inconsistency in coherence and relevance. Therefore, the field of LLM assessment has now become one of the central points of research in artificial intelligence. Out of the assortment of evaluation methods, human-based and metric-based assessment and scoring are the most widely used at present. Although such methodologies can provide useful information, they are usually not scalable, consistent, and transparent when implemented in the context of multiple LLMs. The owning of this shortcoming is the Retrieval-Style Ranking of LLMs project, which proposes the creation of a standardized automated evaluation framework that can compare the results of the LLMs. The approach implies collecting a list of answers produced by various LLMs in response to the same line of questions and using assessment tools like coherence and relevance. This evaluation scheme is a comprehensive scheme because it provides a clear and measurable comparison of models. Data set building is then coupled with algorithmic ranking methods and human preferences to generate unbiased and explainable rankings. By these concerted efforts, we hope to not only increase the reliability of the LLM benchmarking, but also provide a solid transparent framework that will help researchers, developers and end-users to learn about the strength and weaknesses of various language models in real-world scenarios.

Executive Summary

The use of Large Language Models (LLM) like GPT, Claude, and LLaMA has radically revolutionized natural language processing, and the evaluation of performance is also a very difficult task. Traditional appraisal systems, either human- or metric-based, receive little scalability, consistency, as well as transparency. Consequently, it is unclear which model is best at the most diverse tasks, thus the need to have a standardized, automated, and interpretable assessment framework. To fill in this gap, the LLM-Ranker project proposes a comprehensive scheme clearly aimed at ranking and evaluating the responses of an LLM. The framework summarizes the outputs of several models to answer the same prompts and uses a set of multi-metric assessment methods which measure coherence and relatedness. These metrics of evaluation are combined into a retrieval-style ranking system, which allows the development of impartial, quantifiable, and repeatable assessments of the performance of LLM. The suggested architecture is a combination of algorithm evaluation and user preferences where the user is allowed to program and understand the architecture. Besides, the system also uses bias mitigation and calibration techniques based on recent academic research including LLM-as-a-Judge and xFinder. Additional efficiencies are achieved by having adaptive assessment processes supported by Active Evaluation Acquisition and by elaborate frameworks, including MELD. The front-end interface allows users to post prompts, visualise model outputs, and real-time performance rankings. A database layer tracks model outputs, measures of merit, and histories of rankings and holds a persistent and clear-cut record of evaluations. By merging retrieval-based metrics' rigor with multi-criteria evaluation's interpretability, LLM-Ranker seeks to provide a scalable, non-biased, and transparent benchmarking system to compare large language models. This framework will assist researchers, developers, and organizations in learning model weaknesses and strengths in practical and enterprise applications. Briefly put, the project seeks to establish a standardized and unbiased assessment methodology that advances LLM benchmarking's reproducibility, reliability, and interpretability, a step toward the responsible deployment of AI and reputable model comparison.

Table of Contents

List of Figures	xiii
List of Tables	xv
1 Introduction	1
1.1 Purpose of this Document	1
1.2 Intended Audience	2
1.3 Definitions, Acronyms, and Abbreviations	2
1.4 Conclusion	2
2 Project Vision	3
2.1 Problem Domain Overview	3
2.2 Problem Statement	3
2.3 Problem Elaboration	4
2.4 Goals and Objectives	4
2.5 Project Scope	5
2.5.1 Included:	5
2.5.2 Excluded:	5
2.6 Sustainable Development Goal (SDG)	5
2.6.1 Industry, Innovation and Infrastructure	5
2.6.2 Quality Education	6
2.7 Conclusion	6
3 Literature Review	7
3.1 Definitions, Acronyms, and Abbreviations	7
3.2 Detailed Literature Review	7
3.2.1 TrustJudge: Inconsistencies of LLM-as-a-Judge and How to Alleviate Them . . .	7
3.2.2 Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge . .	9
3.2.3 Zero-Shot Benchmarking Framework for Automatic Evaluation of LLMs	10

3.2.4	A Survey on Evaluation of Large Language Models	11
3.2.5	UDA: Unsupervised Debiasing Alignment for Pair-wise LLM-as-a-Judge	12
3.2.6	Decentralized Arena: Towards Democratic and Scalable Automatic Evaluation of LLMs	14
3.2.7	xFinder: Large Language Models as Automated Evaluators for Reliable Evaluation	15
3.2.8	Benchmarking LLMs' Judgments with No Gold Standard (GEM)	16
3.2.9	Large Language Models Are Not Fair Evaluators	18
3.2.10	Active Evaluation Acquisition for Efficient LLM Benchmarking	19
3.2.11	A Fine-grained Evaluation Framework for Large Language Models	20
3.2.12	Enterprise Benchmarks for Large Language Model Evaluation	22
3.2.13	Re-evaluating Automatic Benchers (ACL 2023)	23
3.2.14	Anatomy of Automatic LLM Benchers (2025)	24
3.2.15	G-EVAL: NLG Evaluation with GPT-4 (2023)	25
3.2.16	Holistic Evaluation of Language Models (HELM)	27
3.2.17	Scalable Reliable Evaluation Using LLM Pairwise Comparisons and Elo	28
3.2.18	Chatbot Arena: Evaluating LLMs by Human Preference	29
3.3	Literature Review Summary Table	30
3.4	Conclusion	35
4	Software Requirement Specifications	36
4.1	List of Features	36
4.2	Quality Attributes	36
4.3	Functional Requirements	37
4.4	Non-Functional Requirements	38
4.5	Assumptions	38
4.6	Use Cases	38
4.7	Hardware and Software Requirements	41
4.7.1	Hardware Requirements	41
4.7.2	Software Requirements	41
4.7.3	The Graphical User Interface	42
4.8	Database Design	43
4.8.1	ER Diagram	43
4.8.2	Data Dictionary	45
4.9	Risk Analysis	46
4.10	Conclusion	47

5	Proposed Approach and Methodology	48
5.1	System Overview	48
5.1.1	Frontend	48
5.1.2	Backend Layer (Computation and Evaluation Engine)	49
5.2	Workflow and Data Flow	50
5.2.1	Dataset Collection and Preprocessing	50
5.2.2	Response Generation	50
5.2.3	Metric Calculation	51
5.2.4	Composite Ranking	51
5.2.5	Visualization and Feedback Integration	52
5.3	Retrieval and Ranking Methodology	52
5.3.1	Phase 1 – Dynamic Query Matching	52
5.3.2	Phase 2 – Composite Rank Aggregation	53
5.4	Novel Contribution and Research Implication	55
5.5	Tools and Techniques Utilized	56
5.6	Data Collection	58
5.6.1	Dataset Description	58
5.6.2	Large Language Models Used	60
5.7	Proposed Solution Advantages	61
5.8	Expected Research Impact	61
5.9	Conclusion	62
6	High-Level and Low-Level Design	63
6.1	System Overview	63
6.1.1	Design Approach:	63
6.2	Design Considerations	64
6.2.1	Assumptions and Dependencies	64
6.2.2	General Constraints	64
6.2.3	Goals and Guidelines	64
6.2.4	Development Methods	65
6.3	System Architecture	66
6.3.1	Subsystem Architecture	67
6.4	Architectural Strategies	67
6.4.1	Strategy 1 – Modular and Extensible Design	67
6.4.2	Strategy 2 – Evaluation Abstraction Layer	67

6.4.3	Strategy 3 – Scalable Data Flow	68
6.5	Domain Model	68
6.6	Policies and Tactics	68
6.6.1	Policy 1 – Coding and Version Control	68
6.6.2	Policy 2 – Testing and Validation	68
6.6.3	Policy 3 – Security and Data Handling	68
6.6.4	Policy 4 – Maintainability and Extensibility	69
6.7	Conclusion	69
7	Implementation	70
7.1	Implementation Overview	70
7.1.1	Frontend Architecture	70
7.1.2	Backend Architecture	71
7.1.3	Ranking Engine	72
7.1.4	Cross-Encoder Architecture and Operation	72
7.2	Database Architecture with MongoDB	73
7.3	LLM Provider API Integration	74
7.3.1	Currently Supported Models	74
7.3.2	Free-Tier Constraints and Future Upgrade Path	74
7.3.3	API Communication Workflow	74
7.4	Test Case Design	75
7.4.1	Unit Testing	75
7.4.2	Integration Testing	75
7.4.3	UI / End-to-End Testing	75
7.5	Test Case Table	76
7.6	Detailed Test Case Descriptions	78
7.6.1	Ranking Engine Test Cases (TC-26 to TC-29)	78
7.6.2	API and Backend Test Cases (TC-22 to TC-25, TC-30 to TC-31)	79
7.6.3	Cross-Encoder Test Cases (TC-33 to TC-38)	79
7.6.4	Authentication Test Cases (TC-05 to TC-09)	80
7.7	Test Metrics	80
7.7.1	Ranking Accuracy Analysis	81
7.7.2	Response Time and API Success Rate	81
7.7.3	Ranking Consistency	81
7.8	System Validation	81

7.9	Summary	82
8	User Manual	83
8.1	Introduction	83
8.2	System Requirements	83
8.3	Accessing the System	83
8.4	User Authentication	85
8.5	Homepage Features	86
8.5.1	Multi-Model Comparison	87
8.5.2	Smart Ranking System	87
8.5.3	Query History	88
8.6	Supported AI Models	89
8.7	Main Query Interface	89
8.8	Submitting Queries	90
8.9	Response Ranking Module	90
8.9.1	Manual Ranking Feedback	91
8.10	Viewing Full Responses	91
8.11	Dataset Training Module	92
8.12	Results Module	93
8.13	Query History	93
8.14	Dashboard	93
8.15	Profile Management	94
8.16	Navigation Menu	95
8.17	Error Handling and Troubleshooting	96
8.18	Typical Workflow	96
8.19	Intended Users	96
8.20	Chapter Summary	97
9	Experimental Results and Discussion	98
9.1	Initial Evaluation Using LLM-Ranker	98
9.2	Model-Specific Results	99
9.2.1	GPT	99
9.2.2	Claude	99
9.2.3	Gemini	99
9.2.4	LLaMA	99

9.2.5	Perplexity	99
9.3	Handling Outliers and Unknown Responses	99
9.4	Common Issues Observed	99
9.5	Additional Experiments Conducted on LLM Arena	99
9.5.1	Problems Highlighted by Arena Testing	100
9.6	Experimental Setup	100
9.6.1	Evaluation Metrics Defined	101
9.7	Dataset and Evaluation Procedure	101
9.8	Initial Experiment: Three-Point Relevance Scale (Version 1)	102
9.8.1	Results Under the Three-Point Scale	102
9.9	Revised Experiment: Five-Point Relevance Scale (Version 2)	103
9.9.1	Resolution of Zero-Score Queries	104
9.9.2	Embedding Model Performance Under Version 2	104
9.10	NDCG Performance Analysis	105
9.10.1	Version 2 Model Rankings	105
9.10.2	Large-Scale Validation: 148-Query Cross-Encoder Evaluation	107
9.10.3	Query 5 Case Study: Effect of Annotation Scale	109
9.11	Comparative Analysis: TF-IDF, Embedding, and Cross-Encoder Scoring	110
9.11.1	TF-IDF as a Lexical Baseline	110
9.11.2	Neural Embedding Models	110
9.11.3	Cross-Encoder as the Reranking Layer	111
9.12	Discussion of Findings	112
9.12.1	Annotation Granularity is the Primary Driver of Evaluation Quality	112
9.12.2	Neural Semantic Models Consistently Outperform Lexical Baselines	113
9.12.3	Cross-Encoder Performance at Scale	113
9.12.4	Language Barrier as a Residual Challenge	114
9.13	Limitations and Challenges	114
9.14	Discussion	115
9.15	Future Improvements	115
9.16	Chapter Summary	116
10	Conclusion and Completed Work	118
10.1	Conclusion	118
10.1.1	What Was Built	118
10.1.2	Principal Findings	119

10.1.3 Contributions of This Work	120
10.2 Work Completed	120
10.2.1 Research and Literature Review	120
10.2.2 System Architecture and Design	120
10.2.3 Dataset Construction and Annotation	121
10.2.4 Evaluation Pipeline Implementation	121
10.2.5 Experimental Evaluation	122
10.2.6 Web Application — Frontend	122
10.2.7 Web Application — Backend and API	123
10.2.8 Database and Persistent Storage	123
10.2.9 Testing and Validation	123
10.3 Future Work	124
10.3.1 Urdu-Specific and Multilingual Model Support	124
10.3.2 Inter-Annotator Agreement Studies	124
10.3.3 Expanded Query Coverage	124
10.3.4 LLM-as-Judge Supplementary Evaluation	124
10.3.5 Ensemble and Adaptive Ranking	125
10.4 Closing Remarks	125

List of Figures

2.1	Industry, Innovation and Infrastructure	6
2.2	Quality Education	6
4.1	GUI of the LLM-Ranker system.	43
4.2	Entity-Relationship Diagram of the LLM-Ranker System	44
6.1	Architecture Diagram (rotated)	66
8.1	LLM Ranker Homepage	84
8.2	User Login Interface	85
8.3	User Sign-Up Interface	86
8.4	Features of LLM Ranker	86
8.5	Select AI Models	87
8.6	Response Rankings (can be manually reordered)	88
8.7	Query History	88
8.8	Main Query Interface	90
8.9	Detailed Response Viewer	92
8.10	Dashboard Analytics	94
8.11	Profile Management Interface	95
8.12	Navigation Sidebar	95
9.1	Grouped bar comparing V1 vs V2 NDCG for all 9 models side by side. Every orange bar (V2) is visibly taller than its blue pair, making the annotation improvement argument instantly clear.	105
9.2	Horizontal bar ranking all 9 models by V2 NDCG with the dataset mean as a reference line. Cross-encoder highlighted in orange as the winner, TF-IDF in dark blue as the baseline.	107
9.3	Two-panel figure: a histogram of the 148-query NDCG distribution on the left, and a pie chart of the 5 NDCG bands on the right. Shows that 85.8% of queries scored above 0.75.	108

9.4	Line chart of all 148 individual query NDCG scores with the mean band, green/orange shading above/below mean, and the top-3 and bottom-3 queries annotated.	108
9.5	Your strongest single chart. Query 5 side-by-side bars with green arrows showing the +0.67 jump from V1 to V2 for every model. This visually proves the annotation problem.	109
9.6	Scatter plot of Spearman P vs NDCG per model	112
9.7	Horizontal bar showing the delta NDCG improvement per model (V2, V1). Cross encoder gained the most (+0.0675), SBERT the least (+0.0493).	113

List of Tables

3.1	Summary of Relevant Research Studies for LLM Evaluation and Ranking	31
4.1	Use Case: Upload Dataset	38
4.2	Use Case: Rank LLM Responses	39
4.3	Use Case: View Rankings	40
4.4	Data Dictionary: Users Table	45
4.5	Data Dictionary: Prompts Table	45
4.6	Data Dictionary: Responses Table	45
4.7	Data Dictionary: EvaluationMetrics Table	45
4.8	Data Dictionary: Rankings Table	46
4.9	Risk Analysis for LLM-Ranker System	46
5.1	Tools and Technologies used in the LLM-Ranker System.	57
5.2	LLM Responses Data Insight	58
5.3	Judgement Score Criteria for LLM Responses	59
5.4	List of LLMs Used in the LLM-Ranker Framework	60
7.1	Comprehensive System Test Cases	76
8.1	System Requirements	83
8.2	Sample Ranked Results	91
8.3	Common Errors and Solutions	96
9.1	Model Comparison Results	98
9.2	Common Issues Observed	100
9.3	Arena-Style Results Summary	100
9.4	Experimental Dataset and Configuration Summary	102
9.5	Comparison of Three-Point and Five-Point Relevance Annotation Scales	103
9.6	Per-Model NDCG and Spearman ρ Comparison: Version 1 vs Version 2	105
9.7	Model Performance Rankings — Version 2, Average NDCG (50 Queries)	106
9.8	Cross-Encoder NDCG Score Distribution — 148-Query Large-Scale Evaluation . .	107

9.9 Query 5 NDCG Comparison — Version 1 vs Version 2	109
9.10 Summary of Experimental Limitations	114

Chapter 1 Introduction

LLMs such as GPT, Claude, and LLaMA have gained immense popularity due to their ability to generate human-like responses to natural language queries. However, their outputs can vary significantly in terms of accuracy, relevance, coherence, and usefulness. There is currently no standardized framework to compare and rank these responses fairly across multiple LLMs for the same set of prompts.

Our project, LLC Ranker, will seek to design a ranking system to evaluate and compare LLC responses to queries by users. The researchers built a dataset of the answers of several LLMs to various. we will use evaluation measures and preferences, which are defined by the user. The system will help researchers, developers, and end-users detect the strengths and weaknesses of the various LLLMs in practice.

1.1 Purpose of this Document

This document is intended to introduce the design, implementation, and evaluation of our Final Year Project (FYP) that will be called LLM Ranker: Retrieval-Style Ranking of LLMs. The main objective of this project is to create a complex framework that is able to assess and prioritize the answers produced by various Large Language Models (LLMs) including GPT, Claude, Gemini, and LLaMA.

This paper will set out to provide a clear definition of the objectives, methodology and scope of our research. The project aims at filling the gap of a standardized benchmarking system to compare the quality of the outputs obtained by various LLMs when using the same prompts. Our framework is an aspiration to offer a transparent, fair, and reproducible ranking system of the content generated by LLM by combining both automated evaluation measures, like coherence, and relevance, and human-in-the-loop assessments.

This report is also aimed at explaining how the suggested framework gathers responses and utilizes quantitative and qualitative methods of evaluation and generates comparative knowledge that should guide researchers and developers, as well as end-users, to make a wise choice and use LLMs.

In addition, the document outlines the research methodology adopted, including dataset creation, evaluation metrics, and ranking algorithms, as well as the tools and technologies used. It also discusses the current challenges in LLM evaluation, the rationale behind our design choices, and the expected outcomes of the system.

Ultimately, this report serves to document every phase of the project from background study and system design to implementation, testing, and evaluation and to demonstrate how LLM Ranker contributes to the growing field of AI model benchmarking and evaluation.

1.2 Intended Audience

This project is aimed at the researchers, developers, and other general users of large language models (LLMs) worldwide, as well as those interested in the performance of different models. The system can be applicable to all persons or organizations who wish to verify the accuracy, quality, and reliability of responses generated by different AI models. The audience of LLM-Ranker is large, as the model is used in various areas, covering both academic scientists who analyze the behavior of models and developers who use AI systems, and regular people who need to know which model can give the most relevant answers.

1.3 Definitions, Acronyms, and Abbreviations

LLM: Large Language Model

AI: Artificial intelligence

SDG: Sustainable Development Goal

ER: Entity-Relationship

GUI: Graphical User Interface

1.4 Conclusion

Chapter 1 presented the idea and the drive behind the project of the LLM-Ranker. As the use of large language models (LLMs) such as GPT, Claude, and LLaMA has increased, there is a distinct necessity for a standardized framework to compare their responses in an equal manner. This project will help close that gap by creating a ranking system that will evaluate the performance of various LLMs according to accuracy, coherence, relevance, and other significant variables.

The chapter has also described the purpose of the project, which is the development of a transparent, automated, and human-assisted ranking tool that can be used to guide the researchers, developers, and users in understanding the strengths and weaknesses of various models better. As the intended viewers of this project comprise individuals living on every continent of the globe, both technical specialists and average users, LLM-Ranker can become one of the useful, global standards of AI models that can be chosen and enhanced.

Chapter 2 Project Vision

LLMs such as ChatGPT, Gemini, Claude, and Perplexity are now central to the way people search, learn, and make decisions; it has become necessary to determine which model returns superior, more precise, and fairer outcomes. Since the notions of online visibility were previously determined by SEO, now, the brands, facts, and tools in answers generated by AI are decided by the rankings called LLM. Nevertheless, a clear and standardized way of quantifying and comparing the performance of various LLMs in response to prompts, domains, and contexts does not exist.

This project attempts to address this gap to develop a single evaluation and ranking system of various LLMs. It will train a model that answers typical questions, compare them to measurable criteria (accuracy, coherence, relevance), and generate comparative rankings that allow researchers, developers, and businesses to gain insight into how models operate.

2.1 Problem Domain Overview

The primary issue that introduced LLM-Ranker is that there is no fair and standardized method of assessing and comparing the output of various Large Language Models (LLMs). Although models such as GPT, Claude, Gemini, and LLaMA can all respond in a human-like way, they can be highly incorrect, inaccurate, coherent, or irrelevant, even when asked the same question or prompt. Nowadays, all these systems are not unified, and users, researchers, or developers cannot verify which model is more effective or offers the most credible answers. Due to this, no definite way exists through which organizations and individuals can decide on the best option.

Thus, the solution to this issue was the creation of LLM-Ranker, which is an open-source, automated, and human-reviewed system to assess and rank the answers of several LLMs based on transparent criteria like accuracy, coherence, and readability. This would guarantee the consistency, fairness, and meaning of model comparisons across users and applications.

2.2 Problem Statement

The issue that prompted the development of LLM-Ranker is that there are no means of measuring and comparing the outputs of different Large Language Models (LLMs). While they might be used extensively, there is no objective procedure for measuring and comparing the effectiveness of such models. They can be less or more truthful, less or more relevant, less or more fluent, less or more logical, even when answering the same question.

As a result of this inconsistency, users, developers, and researchers find it difficult to predict which model gives the most reliable or context-sensitive results. In order to cope with this gap, a standardized and unbiased ranking system, LLM-Ranker, is necessary. It seeks to test and compare various models using quantifiable metrics and manual human verification, which provides transparency, reliability, and fair comparison of the results of the various domains and real-world applications of the LLM.

2.3 Problem Elaboration

The only difficulty is that the results of an LLM are not predictable in any way; the same question can result in completely different interpretations, tones, and degrees of correctness when approaching various models. Since these systems are slowly taking control of the flow of information, recommendations, and even decision-making, without a standard evaluation structure, confusion and false trust are the results. Organizations and individuals often compel the selection of a model to depend on subjective impressions or marketing claims as opposed to measurable data of performance. It is also not very transparent, which complicates the process of detecting biases, misinformation, or inconsistencies in model-generated content.

Besides, since LLMs are now applied to business tools, education, health care, and population communication, quality assurance is of utmost importance. It is not possible to establish accountability or keep track of progress with time without some clear and objective way of comparing the behavior of models. The LLM-Ranker aims to address this gap in analysis by providing a systematic, reproducible, and objective evaluation process that emphasizes the performance of various models in practice.

2.4 Goals and Objectives

Our primary goal is to improve the overall accuracy and reliability of LLM evaluation and ranking. the following are the goals that we would like to accomplish during our plan:

- To create a systematic set of outputs of several LLMs to a large set of prompts and query types.
- To develop effective assessment criteria and measures of response such as coherence, readability and relevance.
- To develop and deploy an automated pipeline to rank outputs of LLM based on quantitative metrics, as well as human-in-the-loop metric.
- In order to create some comparative understanding related to the analysis and visualization, it is important to outline the positive and negative aspects of various LLMs.

- To develop a reliable framework that supports researchers, developers, and end-users in selecting and utilizing LLMs more effectively.

2.5 Project Scope

2.5.1 Included:

- Dataset creation with multiple LLM responses for varied domains (general knowledge, reasoning, creative writing, coding tasks).
- Implementation of ranking metrics (semantic similarity using embeddings, grammatical correctness).
- Development of a ranking system capable of automated evaluation and human-in-the-loop feedback.
- Dashboard/visualization for presenting comparative results across models.
- Multilingual ranking beyond English (for initial phase).
- Comparative evaluation across open-source and proprietary LLMs.

2.5.2 Excluded:

- Training or fine-tuning of new LLMs.
- Deployment of LLMs themselves (focus is on evaluation, not generation).

2.6 Sustainable Development Goal (SDG)

2.6.1 Industry, Innovation and Infrastructure

One of the SDG areas that our research is targeting is Industry, Innovation, and Infrastructure. **Fig: 2.1** The LLM Ranker is also useful for this purpose and by:

- Promoting artificial intelligence assessment and benchmarking.
- Developing the digital infrastructure of measuring AI models, being accountable and fair with technology.



Figure 2.1: Industry, Innovation and Infrastructure

2.6.2 Quality Education

The primary goal of this SDG is to ensure inclusive and equitable quality education and promote lifelong learning opportunities for all. **Fig: 2.2**

The LLM Ranker project contributes to this goal by:

- Providing students and researchers with a transparent framework for understanding and comparing large language models.
- Promoting technical skill development in data analysis, evaluation metrics, and AI system benchmarking.



Figure 2.2: Quality Education

2.7 Conclusion

This Chapter has provided an overview of the project's vision, identified the problem domain, stated the problem, elaborated on its significance, outlined goals and objectives, defined the project's scope and constraints, and highlighted the business opportunity it presents.

Chapter 3 Literature Review

This chapter presents a comprehensive review of existing literature and methodologies pertaining to the evaluation and ranking of LLMs. It discusses various methodologies, benchmarking frameworks, and scoring algorithms that different researchers have proposed to assess the performance of models. Additionally, a summary table has been appended at the end of this section, which provides an overview of the main contributions and limitations identified in each study.

3.1 Definitions, Acronyms, and Abbreviations

LLM: Large Language Model

AI: Artificial Intelligence

HELM: Holistic Evaluation of Language Models

BIG-Bench: Beyond the Imitation Game Benchmark

MT-Bench: Multi-Turn Benchmark

3.2 Detailed Literature Review

This section provides detailed information about the research conducted by various researchers on the evaluation and ranking of LLMs, which serves as a foundation and reference for our proposed system.

3.2.1 TrustJudge: Inconsistencies of LLM-as-a-Judge and How to Alleviate Them

3.2.1.1 Summary of the Research Item

Wang et al. [1] propose a model called TrustJudge that was developed to solve the intrinsic inconsistencies in the evaluation of the performance of LLM-as-a-Judge systems. The authors point out two major issues: score-comparison inconsistency, where lower-scoring responses sometimes outperform higher-scoring ones when paired, and pairwise transitivity inconsistency, where preferences form circular or mutually contradictory preference networks. These are believed to be due to loss of information inherent in discrete scoring and ambiguity caused by tie ratings.

To mitigate these problems, TrustJudge introduces the following two main innovations: Distribution-sensitive scoring: computes continuous expectations from discrete probability distributions in a way that preserves information entropy.

Likelihood-aware aggregation that removes transitivity violations by incorporating bi-directional preference probabilities or perplexity measures. Experiments show that TrustJudge significantly improves

the reliability of evaluation: it reduces score-comparison inconsistency by 8.43 and pairwise transitivity inconsistency compared to baseline LLM-judge methods by 10.82. Notably, this is achieved without extra training or human annotation, which makes TrustJudge an affordable and easy enhancement of existing automated evaluation frameworks.

3.2.1.2 Critical Analysis of the Research Item

TrustJudge is an important first systematic study of inconsistencies in the LLM-as-a-Judge paradigm. Its strengths are three-fold: a probabilistic approach that better retains informational content during evaluation than discrete ratings; hypothetical insight into the origins of inconsistencies in the existing frameworks; and practical advances in score accuracy and transitivity on a variety of LLM architectures.

However, several limitations need to be considered. First, TrustJudge remains dependent on the judgments made by LLM judges themselves, and while it reduces inconsistencies, the intrinsic biases of such models still persist. Second, the framework focuses on internal consistency; it does not deal with external validity, that is, correspondence with human judgment on various domains. Finally, the probabilistic models that are used, such as distribution-sensitive scoring and likelihood-sensitive aggregation, may pose an interpretive challenge to the uninitiate in comparison with more straightforward evaluation metrics.

Overall, TrustJudge enhances the credibility and reliability of LLM-as-a-Judge evaluations, but by no means represents anything more than a single step toward a more holistic and robust benchmarking ecosystem.

3.2.1.3 Relationship to the proposed research work

The goal of **LLM-Ranker** is to develop a clear, metrics-driven methodology for comparing and ranking different LLMs, using both quantitative metrics—such as accuracy, coherence, and readability—and human-in-the-loop verification. **TrustJudge** is directly relevant to the current work because it tackles the consistency shortcomings in both the pairwise and scoring-based evaluation methods that were considered during the initial study (pairwise ranking and LLM-as-a-judge). The main points of relevance are as follows:

- **Shared Problem Space:** TrustJudge recognizes that most existing evaluation methodologies have poor reliability and fairness; this limitation is also acknowledged by LLM-Ranker.
- **Complementary Methods:** TrustJudge proposes a probabilistic scoring and aggregation framework that can be integrated within LLM-Ranker to improve the coherence of pairwise ranking and automated scoring pipelines.

- **Broader Scope of LLM-Ranker:** While TrustJudge focuses on the optimal aggregation of judgments, the approach of LLM-Ranker is more general, covering multi-metric evaluation, dataset diversity, multilinguality, and visualization.
- **Potential Integration:** LLM-Ranker can incorporate TrustJudge’s distribution-sensitive scoring mechanism to minimize inconsistencies in automated rankings, thereby increasing transparency through structured metrics and interactive dashboards.

In a nutshell, TrustJudge offers substantive technical dividends toward the goal of reducing evaluation inconsistency; LLM-Ranker, on the other hand, is a more general benchmarking framework that incorporates such methods in its effort to advance values such as fairness, trustworthiness, and interpretability.

3.2.2 Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge

3.2.2.1 Summary of the Research Item

Liu et al. discuss the performance of LLMs as adjudicators, specifically focusing on the position bias - the tendency of LLM adjudicators to give more favourable answers depending on their location in a prompt. Their report presents an experiment of 12 LLM judges, 22 tasks and about 40 solution-generating models with benchmarks of about 100,000 evaluation cases. Three measures of evaluation, repetition stability, position consistency, and preference fairness, are used to measure position bias. Findings show that position bias is systematic rather than stochastic, and it will be affected at the judge, candidate, and task levels. The bias level is especially moderated by the quality difference between competing solutions, but the prompt length influences it comparatively the least. This study also shows that there is disparity in the bias between tasks and judges, thus demonstrating the need to change data sets and come up with sound evaluation policies.

3.2.2.2 Critical Analysis of the Research Item

Liu et al. (2024) contribute greatly by marking the inherent shortcomings of the evaluation frameworks built on the topic of large language models. The study overcomes the issue of a single emphasis on accuracy and deals with the meta-problem of evaluation of responses, which is a cornerstone issue of justifiable benchmarking. The strength of their work is obtained by a large scale of experiment (more than 100,000 cases), which is accompanied by bias-specific measures, which cumulatively give strength to the validity of results. However, it is also mostly diagnostic: the work determines the existence and nature of position bias without discussing much on the remedies other than proposing the possibility of changes in datasets. Also, although the metrics offer meaningful information on how well an evaluation is conducted, it is rather limited and focuses on bias detection rather than use a more broad measure of

quality that would include other aspects of evaluation like coherence or relevance. On the whole, the research highlights one of the underlying observations: the existing approaches to LLM-as-a-judge can be biased, which undermines their authority as unbiased judges.

3.2.2.3 Relationship to the Proposed Research Work

The findings presented in Liu et al. [2] have significant relevance to the proposed framework of LLM-Ranker. Considering that LLM-Ranker aims to objectively assess and rank various LLMs on an equal footing, the challenges linked to solely relying on LLM-as-a-judge assessment become pertinent. The presence of position bias underlines the necessity to consider human-in-the-loop assessment and other metrics, like coherence and semantic similarity, instead of mere automated pair-wise ranking.

Besides, the design of LLM-Ranker that merges automated assessment with human supervision lowers the risks that were pointed out in this research. Utilizing the findings of Liu et al., the proposed system is capable of forming ranking methods that will reduce both positional and system biases, thereby creating a benchmarking tool that is more transparent, just, and trustworthy.

3.2.3 Zero-Shot Benchmarking Framework for Automatic Evaluation of LLMs

3.2.3.1 Summary of the research item

Zero-shot Benchmarking (ZSB) is a new framework proposed by Pombal et al. (2025) [1] in the paper titled *Zero-shot Benchmarking: A Framework to Flexible and Scalable Automatic Evaluation of Language Models*. ZSB relies on language models to generate both synthetic and evaluation test data to be used in evaluating other language models. The authors state that the more powerful and sophisticated LLMs are, the more time-consuming, costly, and soon saturated traditional evaluation methods become, which are based on human-labeled datasets and task-specific measures.

The benchmarking process is simplified by ZSB through two components: a meta-prompt to produce test data, and a judgment prompt that models output on a Likert scale or pairwise. ZSB supports various tasks and modalities (text-only and vision-language tasks) and can be generalized to new domains or languages. It automatically compares the performance of LLMs, including GPT, Claude, and Gemini on different tasks, finding a high correlation with human judgment, indicating that automated measures can closely reflect human evaluations. ZSB is scalable, flexible, and self-improving, adapting as new models are developed.

3.2.3.2 Critical analysis of the research item

The ZSB framework makes a significant contribution to the evaluation of LLCMs, and its advantages are in automation, scalability, consistency with human assessment, and the multilingual/multimodal properties. A prejudiced or poor model as a judge may however give the results that are not representative of true quality. Automated decisions could not reflect subjective aspects of creativity or empathy. In addition, the produced prompts might not be a true reflection of the actual human input in all its diversity. Nonetheless, ZSB has high potential of low cost, automated and reproducible assessment pipelines in spite of these limitations.

3.2.3.3 Relationship to the proposed research work

LLM-Ranker has the same goal as ZSB: to offer a fair, consistent and transparent platform where outputs of a wide range of LLMs can be compared with each other. However, LLM-Ranker develops ZSB further in the following critical directions:

- **Comparative ranking:** LLM-Ranker is capable of ranking several models simultaneously with prompts with metrics of coherence, readability and relevance.
- **Human-in-the-loop refinement:** Automatic rankings can be verified and fixed by humans- a weakness that had hampered ZSB before.
- **More extensive task coverage:** the framework works with reasoning, creative writing, programming, general knowledge, and the response based on multiple languages.
- **Visualization:** The dash board is there to help the users check the strong points and weak points of LLM.

ZSB gives a base to work on for the LLM Ranker, the LLM Ranker then uses that base and strengthens its concepts by using ranking and transparency and semi human evaluated eval framework.

3.2.4 A Survey on Evaluation of Large Language Models

3.2.4.1 Summary of the research item

The survey proposes an in-depth examination of the evaluation procedures of the major language models (LLMs) such as GPT, Claude, and LLaMA. It captures the chronological trend of machine-intelligence measures and the paramount significance of assessment in improving the elucidation of logic, precision, and trustworthiness. The authors single out three main dimensions of evaluation:

- **What to evaluate:** language understanding, reasoning, factuality, ethics, and robustness.

- **Where to evaluate:** datasets and benchmarks such as HELM, BIG-Bench, and MT-Bench.
- **How to evaluate:** Human evaluation and automatic performance measures.

The research highlights the absence of a unified and fair system of evaluation of all LLMs and identifies the issues of dynamic evaluation, multilingual support and transparency. The authors, therefore, mention that a holistic, value-based, and adaptive evaluation framework is needed, which will be able to keep up with the pace of the LLMs advancement.

3.2.4.2 Critical analysis of the research item

This paper categorizes and systematically re-views LLM evaluation, enabling re-researchers to identify what has been evaluated and what has not yet been assessed [4]. Its strong points are that it systematizes a very wide field and clearly identifies major gaps. Omai provides a more benchmark description without practically offering a measurement system. Limited integration of human feedback; automated metrics are not indicative of preferences, context. or readability. Fixed benchmarks cannot keep pace with the rapid development of LLMs, which means that adaptive evaluation systems.

3.2.4.3 Relationship to the proposed research work

In particular, the key relationships derived, directly motivate the development of LLM-Ranker. Creation of Datasets: LLM-Ranker compiles structured responses given a common prompt for comparison in parity. Evaluation Metrics combines coherence, relevance, and readability using a hybrid scoring pipeline. Automation and Human Refinement: Bridges quantitative assessment with human-in-the-loop evaluation. Transparency and Accessibility Provides an open benchmarking tool for researchers, developers, and general users. LLM-Ranker thus operationalizes the identified theoretical gaps in the survey, creating a practical, adaptive, and standardized evaluation system for LLMs.

3.2.5 UDA: Unsupervised Debiasing Alignment for Pair-wise LLM-as-a-Judge

3.2.5.1 Summary of the research item

Zhang et al. (2024) propose a framework known as UDA: Unsupervised Debiasing Alignment, which aims to reduce bias from pairwise comparison analysis of Large Language Models [5]. Traditional systems consisting of LLMs as judges have been plagued by preference biases of the judges for certain outputs. That led to biased rankings and discrepancies in evaluations. The UDA method dynamically modified the K-factor and optimized win probabilities for each comparison using an Elo rating system that continuously adapted itself with the help of a small neural network. This technique minimizes the opinion difference among judges and stabilizes the rankings, further improving the correlation with

human judgments empirically. UDA reduces the standard deviation of inter-judge ratings by 63.4%, strengthening correspondence with human ratings by 24.7%, and providing a more reliable evaluation ecosystem.

3.2.5.2 Critical analysis of the research item

UDA framework is a substantive contribution since it does not need labeled data to eliminate bias in pairwise decisions. It is an effective instrument in reducing bias because of its strong theoretical basis, which builds on the principle of consensus, and its empirical achievements. Still, there are a number of limitations:

- **Narrow applicability:** UDA is only applicable to pair-wise comparisons, and thus it is not useful in the multi-metric assessment, e.g., coherence or creativity.
- **Black-box adaptation:** The Dynamic K-factor corrections are not quite interpretable, and this reduces the transparency of stakeholders.
- **Consensus assumption:** The framework assumes that the results of the judgment would be fair when disagreement is lowest, which might be untrue in the case of subject tasks like creative writing.

Overall, UDA must be seen as a bias-correction layer as part of larger evaluation pipelines, and not as a multi-dimensional methods of evaluating the LLM.

3.2.5.3 Relationship to the proposed research work

LLM-Ranker attempts to create a retrieval-based ranking system that relies on the metrics of coherence, factuality, readability, and relevance to rank the results of the LLM. Whereas UDA is focused on pairwise comparisons de-biasing, LLM-Ranker presents a comprehensive framework of benchmarking that incorporates automated measures and human-in-the-loop assessment. The following are the main relationships:

- **Complementary approaches:** The bias-reduction methods at UDA can be utilized to improve fairness in the LLM-Ranker in two different ways, either by directing pairwise comparisons or improving aggregated human judgments.
- **Greater scope in LLM-Ranker:** In contrast to UDA where the rating is only stabilized, the LLM-Ranker compares the models in several dimensions, providing a more realistic and comprehensive evaluation.
- **Potential synergy:** The consensus-minimizing aspect of UDA can enhance the reliability and

repeatability of the outputs of the LLM-Ranker in situations where subjectivity is required or in those where multiple judges are involved.

- **Research gap addressed:** UDA addresses the problems of judicial alignments, and LLM-Ranker builds on this finding and adds more structured datasets, multilingual appraisal, and interactive visualization tools.

o the extent, UDA provides information to LLM -Ranker regarding bias reduction in evaluator judgments and LLM -Ranker contributes to the evaluation paradigm by expanding it to a transparent, metric-rich and comprehensive benchmarking system.

3.2.6 Decentralized Arena: Towards Democratic and Scalable Automatic Evaluation of LLMs

3.2.6.1 Summary of the research item

The framework proposed by Yin et al. (2024) is called Decentralized Arena (*dearena*) and is aimed at resolving the scalability and fairness issues of benchmarking LLMs [2]. Conventional benchmarks like MMLU are being saturated as they grow better, whereas human-in-the-loop leaderboards, like Chatbot Arena, are both costly and time-intensive. New automated systems, including LLM-as-a-judge, can be scaled better, but again tend to be biased, as they rely on one or a few models of authority.

The solution is to have Dearena, where LLMs evaluate each other through collective intelligence, providing a democratic and decentralized assessment. The system combines two innovative points:

- **Algorithms:** Coarse-to-fine ranking algorithm to efficiently insert new models with sub-quadratic complexity.
- **Strategy:** Automatic evaluation of question selection to dynamically expand the evaluation dimensions.

Massive experiments on 66 LLMs demonstrate that Dearena can achieve approximately 97% of human-level evaluation performance using less cost and greater scalability [2].

3.2.6.2 Critical analysis of the research item

The salient advantages of **Dearena** arise from its scalability, efficiency, and fairness. The model democratizes the evaluation process and reduces cost by avoiding reliance on human annotators or single-authority evaluation systems. Its coarse-to-fine ranking mechanism ensures computational efficiency suitable for large-scale benchmarking. However, several disadvantages remain:

- **Dependence on self-assessment:** Evaluation of LLMs by other LLMs may amplify existing bi-

ases rather than mitigating them.

- **Lack of transparency in evaluation rationale:** It is difficult to trace individual judgment decisions, reducing interpretability for stakeholders.
- **Focus on efficiency over multidimensional quality:** Despite strong correlation with human judgments, Dearena may fail to capture subtler aspects such as creativity, empathy, or long-context reasoning unless prompts are carefully crafted.

Despite these limitations, Dearena represents a significant step toward a democratic and scalable alternative to costly human-based benchmarking.

3.2.6.3 Relationship to the proposed research work

The project of Dearena and the LLM-Ranker are united by the general purpose to provide a non-biased, open, and scalable form of evaluation of LLMs. The two systems also seek to address the shortcomings of the existing benchmarking systems that are often costly, slow, congested as well as unjust. Areas of comparison are:

- **Common objectives:** Both projects aim at creating robust, scalable and properly designed systems of ratings in the field of LLM.
- **Methodological differences:** LLM-Ranker emphasizes structured datasets, automated metrics, semantic similarity, coherence scoring, and human-in-the-loop refinement. Dearena, on the other hand, leverages collective intelligence among LLMs through mutual reviewing.
- **Complementarity:** LLM -Ranker provides a metric-driven pipeline to evaluate individual characteristics, whereas Dearena minimizes the cost and authority bias in comparisons over a large scale. An opportunity to scale LLM-Ranker offered by incorporating the principles of decentralized evaluation by Dearena may be a viable route.

3.2.7 xFinder: Large Language Models as Automated Evaluators for Reliable Evaluation

3.2.7.1 Summary of the Research Item

Yu et al. propose a system that would allow big language models to serve as automatic scorers of the responses produced by other big language models. The article focuses on developing a formal evaluation pipeline that can allow models like GPT 4 or LLaMA to evaluate generated text in relation to criteria including accuracy, fluency, and coherence, without the approach to human annotation. The authors introduce a cross-evaluation process where several LLMs evaluate the output of other systems and also

they combine this with an agreement-based calibration approach that aims to minimize bias. Benchmarking experiments on human judgments establish that automated LLM scorers are highly correlated with human expert judges, and the promise of the use of LLM as assessors is high and cost-effective.

3.2.7.2 A Critical Discussion of the Research Article

The Yu et al. framework is a promising approach toward automating and scaling the process of evaluating language models. It is especially good in reliability due to its cross-validation mechanism, which considers the evaluation of more models rather than relying on a single "judge". Additionally, it effectively handles the cost and scaling issues with human evaluation. There are, of course, some limitations to this work. The strong reliance on consensus among LLMs may be unhelpful if the models happen to have any shared training data or inductive biases; their appraisals could all settle on low-quality criteria. Furthermore, although the correlation with human judgments is strikingly high, the paper does not deeply discuss qualitative deficiencies, such as errors in reasoning or in the lack of contextual understanding—things at which human judges are usually good. Increasing the variety of evaluation tasks beyond text generation would do even more to make the framework more generalizable.

3.2.7.3 Relation to the LLM-Ranker Suggested Research Task

Yu et al.'s work is quite aligned with your FYP project, LLM-Ranker since the two main ideas are related to establishing structured, fair, and automated frameworks for LLM output evaluation. You can extend the cross-evaluation idea of xFinder by incorporating retrieval-based ranking approaches, where outputs can be ranked with respect to factuality and relevance by means of similarity measures such as cosine similarity, BLEU, ROUGE, or embedding-based scores. While xFinder focuses more on the issue of evaluation, the proposed system can go further in incorporating bias correction and multi-criteria ranking, thus encouraging transparency and interpretability in the scoring. Hence, xFinder can serve as a useful foundational reference in designing the evaluation and comparison pipeline of your project.

3.2.8 Benchmarking LLMs' Judgments with No Gold Standard (GEM)

3.2.8.1 Description of the Research Area

Li et al. (2023) present GEM, a novel evaluation framework that aims to measure the quality of LLM generations without relying on gold-standard reference answers. Traditional evaluation metrics heavily rely on human-annotated or pre-defined references, limiting scalability and objectivity. GEM mitigates this limitation by approximating mutual information between a model's generated response and a latent "reference" distribution learned by an auxiliary generative model. The authors further propose GRE-bench (Generating Review Evaluation Benchmark), which applies GEM to assessing LLMs' capabilities

in generating academic peer reviews from data at ICLR 2023. Their results point out that GEM has a high Pearson correlation with human judgments and proves robust to manipulative tactics such as response padding and superficial paraphrasing, the typical weaknesses of surface-based metrics. What is more, the authors avoid data contamination risks by continually enriching GRE-bench with open-access reviews and articles. Experimental results demonstrate that GEM surpassed GPT-4-based assessors in consistency and fairness, particularly in settings with no human evaluators available or when potential biases might arise.

3.2.8.2 Critical Analysis of the Research Item

The GEM model proposed by Li et al. represents a paradigm shift in the evaluation of LLMs, as it overcomes the usual dependence on reference outputs and allows scalability and task generalization. Its strengths are many. First, GEM exhibits great scalability and generalization given the absence of reference answers; indeed, this allows for the efficient evaluation of a wide range of diverse tasks. Second, GEM is robust against such adversarial manipulations as text padding and rewording, offering more reliable assessment for open-ended tasks. Third, GRE-bench contributes to benchmarking with a standardized and reproducible benchmarking suite where consistency in model comparison is supported. Transparency is another strength given that open-access datasets and periodic updates are utilized, hence enhancing reproducibility while reducing any potential risks related to data contamination.

GEM also has a number of weaknesses and limitations: it is only as reliable as the generative model it is based on and thus vulnerable to biases or inaccuracies in that model. Peer-review generation, though an important evaluation task in itself, needs broader validation on domains like summarization, dialogue systems, and question answering. While mutual information provides a mathematically rigorous basis for evaluation, it is not necessarily aligned with human judgement over creativity or coherence. Finally, the probabilistic nature of GEM makes it harder to interpret for non-technical stakeholders than more direct scoring schemes. Overall, GEM represents an exciting alternative to traditional reference-based approaches to evaluation; it does, however, need further empirical validation before it can be used reliably in wider practical contexts.

3.2.8.3 Relevance of the Proposed Research Activity

Li et al.'s work (2023) is also closely related to the aims of the proposed FYP: LLM-Ranker: Retrieval-Style Ranking of LLMs. Both work towards the establishment of a fair, consistent, and automated evaluation framework for LLMs. The ability of GEM to evaluate model outputs in the absence of goldstandard references aligns with the multi-metric ranking proposed, which will include coherence and relevance. The use of GEM in the ranking pipeline will be helpful especially where high-quality

reference data is missing or scarce. The robustness of GEM against text manipulation could serve to enhance fairness within the ranking process. Combining GEM’s information-theoretic evaluation with retrieval-based metrics such as cosine similarity, embedding overlap, and factual consistency checks allows balanced and interpretable scores for LLM outputs. Besides, the design philosophy of GRE-bench may guide efforts towards developing a standardized, transparent, and reproducible evaluation dataset for benchmarking LLMs.

3.2.9 Large Language Models Are Not Fair Evaluators

3.2.9.1 Summary of the Research Item

Wang et al. develop a study on the issues of fairness and consistency when Large Language Models are employed as evaluators for other LLM-generated responses. In this work, they point to a critical positional bias in which the order of candidate outputs significantly influences ranking results. For instance, when GPT-4 serves as an evaluator, Vicuna-13B can be misled to “outperform” ChatGPT in 66 out of 80 test cases through merely changing the sequence of responses. In order to reduce this bias, the authors introduce a calibration framework consisting of three techniques: (1) Multi-evidence Calibration, where the evaluator generates multiple rationales before giving a score; (2) Balanced Position Calibration, which averages the results across permutations to cancel out positional advantages; and finally, (3) Human-in-the-Loop Calibration, which involves human verification of ambiguous cases detected by position diversity entropy. They then manually label model comparisons, such as ChatGPT versus Vicuna-13B, with “win/tie/lose” tags by using the Vicuna benchmark. They show that their calibration methods can help alleviate the bias of evaluation and improve alignment with human judgment. They also release FairEval, a unified toolkit that integrates these calibration methods along with human annotations to support this.

3.2.9.2 Critical Discussion of the Research Article

Wang et al. unveil one of the most serious weaknesses of the pipelines of LLM-based evaluation positional bias. The major strength of the study is that it clarifies how the order of the responses may confer bias to the results hence shedding significant light on the reliability of evaluators. The calibration plans suggested in this paper are both practical and systematic, and they increase the robustness without subjecting the implementation of the plans to unrealistic complexity. The conclusions have empirical support, such as a realistic human labeling phase and comparison of ChatGPT output with that of Vicuna. FairEval toolkit is publicly available, which leads to increased reproducibility and transparency. However, there are a number of weaknesses that can be identified. The calibration techniques would involve extra computation overheads in terms of additional permutation and checking. The research only

investigates positional bias and other possible bias like prompt sensitivity or familiarity with the model is not fully explored. The experiments are limited to pairwise conditions (ChatGPT vs Vicuna) and might not be applicable to multi-model and multi-task performance. Furthermore, as the human-in-the-loop element enhances accuracy, it somewhat removes the benefit of automation. On the whole, the study has a practical and useful contribution, but it should be supplemented with more comprehensive assessment situations.

3.2.9.3 Relation to the LLM-Ranker Suggested Research Task

Results from Wang et al. are particularly relevant to your FYP, LLM-Ranker: Retrieval-Style Ranking of LLMs, since both are related to the fairness and accuracy of automated LLM evaluation systems. Your ranking model will be based on LLM-based assessors that score or rank the outputs, so it is important to understand and mitigate positional biases. You can adopt or extend their calibration mechanisms, like Balanced Position Calibration and Multiple Evidence Generation, to make sure that comparisons in your retrieval-based ranking framework are fair. Second, while FairEval focuses on pairwise judgments, your project's multi-criteria evaluation approach will generalize the concept of calibration for broader bias types, such as prompt dependence and content familiarity. The integration of mitigation techniques will make your ranking pipeline much more robust, fair, and transparent, ensuring that positional or structural biases in the LLM-based scoring do not disproportionately affect the outcomes of the evaluation.

3.2.10 Active Evaluation Acquisition for Efficient LLM Benchmarking

3.2.10.1 Summary of the research item

The work Active Evaluation Acquisition for Efficient LLM Benchmarking proposes a cost-effective, adaptive approach for evaluating Large Language Models without requiring the full assessment of each item in the benchmark [10]. Traditional benchmarking requires the evaluation of all prompts, which is both time- and computationally expensive. In this regard, the authors propose the Active Evaluation Acquisition framework, which actively selects the most informative subset of test samples for evaluation. The system uses RL to build an optimal acquisition policy that adaptively selects test cases based on the feedback received from previously evaluated samples. It also models relationships among prompts so that outcomes regarding unseen scenarios can be inferred by using learned correlations. Experimental results have shown that AEA achieves high benchmarking accuracy with a great reduction in the evaluation cost and a number of required samples [3].

3.2.10.2 Critical analysis of the research item

This work has a meaningful contribution in that it addresses the scalability bottleneck in LLM benchmarking—a matter of fast-growing model size and diversity [10]. The use of reinforcement learning for selective evaluation and modeling of dependencies between prompts is a novel approach that boosts efficiency over uniform or random sampling. The experimental results are quite convincing: significant cost reductions with negligible loss in accuracy. However, The success of AEA crucially depends on whether the RL policy can generalize across domains. Potential misjudgments: Poor modeling of dependencies may result in biased or inadequate ratings. The study mainly focuses on traditional benchmarks, thereby leaving the performance on open-ended tasks—retrieval, for instance, dialogue or reasoning—insecure. Computational cost for training the acquisition policy may come at the expense of some gains in efficiency. Overall, AEA represents a sophisticated, scalable evaluation strategy but perhaps needs modification to support diverse or creative LLM evaluation contexts.

3.2.10.3 Relationship to the proposed research work

This work is thus close to the goals of LLM-Ranker: Retrieval-Style Ranking of LLMs, since both aim at enabling efficient, fair, and scalable evaluation of large language models. While AEA focuses on how to optimally select test samples for evaluation, LLM-Ranker deals with the comparison of LLM outputs based on criteria such as coherence, factuality, and relevance. The active acquisition mechanisms developed in AEA could thus be used to reduce the number of prompts or responses that need full evaluation, making LLM-Ranker even more scalable and resource-efficient. Moreover, the dependency modeling concept developed in AEA translates into the ability of LLM-Ranker to infer performance indicators over similar prompts or query clusters. Integration of AEA-inspired methods with LLM-Ranker will result in a more adaptive and cost-effective evaluation pipeline, especially in large datasets or multi-dimensional evaluation tasks. This work provides valuable methodological insights for improving efficiency, flexibility, and robustness.

3.2.11 A Fine-grained Evaluation Framework for Large Language Models

3.2.11.1 Summary of the research item

The paper entitled A Fine-grained Evaluation Framework for Large Language Models introduces MELD—a fine-grained evaluation framework that systematically organizes evaluation criteria into eight general categories and fifty subcategories, each with a clearly specified distinct metric. MELD allows pointwise grading (i.e., each model output is graded individually) and pairwise comparison (i.e., contrasting outputs from different models), thus enabling the nuanced evaluation of LLMs on multiple dimensions. The

framework aims to capture subtle strengths and weaknesses that cannot be represented by an aggregate accuracy or quality score. In fact, this framework offers a more diagnostic view of model evaluation. Using the proposed hierarchical taxonomy, MELD allows multi-level insights into the behavior of LLMs, helping researchers identify regions of strong performance and areas in which improvement is needed [4].

3.2.11.2 Critical analysis of the research item

The MELD framework contributes meaningfully by introducing structured granularity into LLM evaluation, allowing researchers to tell not only how well models perform but also why they achieve those results. Its division into multiple categories and subcategories supports detailed diagnostic analysis, enhancing thus interpretability and comparability among models. Nevertheless, a number of limitations are present within the framework, Creating and updating 50 subcategories requires significant domain knowledge and is time-consuming. MELD may struggle to generalize to tasks or domains other than the ones it was originally intended for. Fine-grained analysis increases human and computational workload, especially when many subcriteria are used. Aggregation inconsistency. The aggregation of several fine-grained judgments may introduce incoherence or redundancy among subcriteria.

Despite these challenges, MELD’s approach offers a flexible, structured route to fine-grained evaluation that goes beyond one-dimensional benchmarking.

3.2.11.3 Relationship to the proposed research work

MELD’s multi-aspect, hierarchical design aligns closely with the goal of the system, LLM-Ranker: Retrieval-Style Ranking of LLMs, to assess and rank multiple LLM outputs along diverse qualitative and quantitative dimensions. For example, one might base a multi-metric design on the same general idea as MELD by mapping MELD’s dimensionality onto the ranking metrics of the LLM-Ranker. This also extends to its support for both pointwise and pairwise evaluation: MELD is a hybrid framework that provides strategies for integrating direct scoring with comparative ranking, mirroring the system’s strategy. MELD could be integrated into the LLM-Ranker to highlight not only how two models compare but also why one model ranks above another. However, not all of the very detailed subcriteria included in MELD would be adopted but rather empirically checked and validated in advance for the application domain with just a selection of those criteria so as not to affect computational efficiency adversely. Overall, MELD serves well as an architectural reference for building a structured, interpretable, and fine-grained evaluation mechanism within the LLM-Ranker framework.

3.2.12 Enterprise Benchmarks for Large Language Model Evaluation

3.2.12.1 Summary of the research item

Enterprise benchmarks for large language model evaluation: The work proposes a benchmarking framework suited for enterprise-oriented applications of LLMs. The authors aggregate 25 publicly available datasets representing various industry verticals such as financial services, legal affairs, cybersecurity, and climate sustainability. They benchmark 13 different LLMs across these verticals and report significant variation in model performance, often finding that strong performance in one vertical does not necessarily generalize well to other verticals. In a spirit of transparency and reproducibility, the authors release their source code and prompt templates to the community. Work like this addresses the gap between highly generic benchmarks, such as MMLU or HELM, and specialized enterprise tasks by offering domain-grounded evaluation pipelines that better reflect realistic business scenarios [5].

3.2.12.2 Critical analysis of the research item

A key strength of this work is domain specificity, addressing one of the main weaknesses of traditional LLM benchmarking, focused on general-purpose tasks [12]. The involvement of datasets from various industries provides practical value in the form of insight regarding LLM performance under realistic enterprise constraints. Increasing the number of datasets to 25 and the number of models to 13 provides empirical depth for the benchmark and confirms that no model can be universally applied at the enterprise level for LLM applications. However, several limitations appear. The datasets are all open-access and domain-oriented but may not capture the operational realities completely. While domain-focused, this benchmark relies on common NLP tasks, such as classification and extraction, rather than customized enterprise workflows. The evaluation metrics are generic and do not capture domain-specific nuances well, such as legal correctness or compliance sensitivity. The study identifies domain performance gaps but does not thoroughly investigate the root causes of such divergences—such as model bias or data imbalance.

3.2.12.3 Relationship to the proposed research work

This work directly informs the proposed project, LLM-Ranker, particularly if the system is designed for adaptable, domain-aware evaluation. The paper emphasizes domain sensitivity, demonstrating how an LLM performing well on general tasks fares poorly in specialist enterprise domains like finance and law. A ranking framework being considered may incorporate this idea through developing domain-specific sub-rankers, or even considering dataset clusters inspired by this benchmark. The datasets and prompt templates published in the work could function as test beds or foundational modules within

the wider testing pipeline to help validate robustness across a number of domains. However, while the benchmark provides very valuable domain-based calibration, the retrieval-like ranking system of the proposed FYP has to be scalable and generalizable outside of enterprise uses. Possible extensions to such work may include adaptive weighting mechanisms in which metrics or domain relevance influence rankings dynamically. In all, the work provides a useful basis and many useful resources to extend LLM-Ranker into enterprise- and task-specific evaluations, underpinning methodological diversity and external validation.

3.2.13 Re-evaluating Automatic Benchers (ACL 2023)

3.2.13.1 Summary of the research item

The paper Re-evaluating Automatic Benchers revisits the concept of model-based evaluators called automatic benchers for the automatic evaluation of NLP and LLM-generated text [13]. They investigate the extent that these automatic benchers are in concordance with human judgment, as well as the sensitivity of their performance to design decisions. By systematically manipulating the components of the evaluator pipeline regarding prompt construction, reference set selection, the choice of scorer model, and score aggregation methods, they determine how these affect the outcome of model rankings. Their analysis shows that even relatively minor modifications to the design lead to major changes in final model ranking, at times differing considerably from human-preferred orders. The central conclusion is that an automatic bencher requires conscious calibration, validation, and care in its design before it is used for comparative evaluation [6].

3.2.13.2 Critical analysis of the research item

This work provides very valuable insight in its extensive sensitivity analysis, which shows the instability of automatic evaluation pipelines and underlines the risks of their uncritical employment [13]. The fact that it tests empirically with many models and several evaluation configurations reinforces the strength of its conclusions. There are, however, a few limitations to consider. The experiments are limited and do not cover all NLP domains, such as dialogue, reasoning, and summarization. The book points out problems in the existing paradigm; however, as a prescription and corrective framework, it is limited. The analysis treats automatic evaluators as black boxes without decomposing their scoring metrics into elements that explain observed variability.

3.2.13.3 Relationship to the proposed research work

This research aligns with your FYP, LLM-Ranker, which also uses automated or semi-automated evaluation pipelines. The following are the methodological and cautionary takeaways from these findings:

when designing the ranking framework, robustness and sensitivity analyses should be conducted to study how variations in prompt design, reference usage, and evaluator models affect ranking stability. Besides, the volatility of evaluators pointed out in this research could be softened by the calibration mechanism or fallback human judgments. LLM-Ranker can further provide diagnostic insights of the study by proposing a more stable multi-evaluator approach, such as aggregating scores from different benchers or normalizing scores to reduce the susceptibility to single-model bias. Overall, the paper provides a strong empirical basis for ensuring that the LLM ranking pipeline produces consistent, reliable, and interpretable results even under small design perturbations.

3.2.14 Anatomy of Automatic LLM Benchers (2025)

3.2.14.1 Summary of the research item

The paper *An Anatomy of Automatic LLM Benchers* provides a systematic review of the four key components that define an automatic LLM bencher-input set, evaluation model, evaluation type, and aggregation method. Motivated by the high cost and impracticality of large-scale human evaluation (e.g., Chatbot Arena), the study seeks to understand how different component configurations affect alignment between automated and human-based rankings. The authors explore three central questions: (1) how to select components for maximal correlation with human judgments, (2) whether bencher accuracy decreases when ranking closely matched LLMs, and (3) whether instance-level evaluator performance can predict overall system-level ranking reliability. The findings reveal that using the *Arena Hard* dataset consistently yields higher human correlation compared to *Alpaca Eval*; that proprietary models such as GPT-4-turbo and GPT-4o perform best, while *Llama-3.1-70B* serves as a robust open-source alternative; that binary pairwise (win/lose) evaluation generally outperforms 5-point pairwise scales; and that the Bradley–Terry (GBT) model is the most reliable aggregation method. Moreover, the study introduces the *Controllable Kendall’s Tau* as a novel measure for assessing benchmark reliability under small inter-system performance gaps [7]. The authors conclude that automatic benchmarkers’ alignment with human preferences degrades sharply when ranking near-equivalent models, and that instance-level metrics fail to generalize to system-level performance.

3.2.14.2 Critical analysis of the research item

This work provides a systematic, methodologically sound investigation into the automatic LLM bencher components and fills an important gap that existed within previous benchmarking research. Its factorial experimental design isolates the independent contribution of each component and provides transparent empirical guidance on an optimal setup. Controllable Kendall’s Tau is an important methodological contribution because it offers a quantitative assessment of stability in comparative analyses involving LLMs

that are only marginally different-an opportunity with which standard correlation metrics commonly fail. The consistent finding of the superiority of Arena Hard input sets and binary pairwise evaluations points to the value of simplicity and standardization of benchmarking practices. However, several limitations are obvious. The experiments only include 18 LLMs and a curated set of datasets; hence, their results may not generalize well to newer and larger open-source models. The observed negative correlation in cases where strong evaluators are paired with high-rated reference systems suggests instability under certain configurations. This paper has a focus on diagnostic exploration and not prescriptive innovations intended for benchmark improvement. The paper thus makes a valuable empirical and methodological contribution, but it should be extended to larger model classes and to adaptive schemes of evaluation in future work.

3.2.14.3 Relationship to the proposed research work

The current study directly informs the design of FYP project LLM-Ranker, which pursues a reliable automatic ranking framework [14]. Based on the findings, our system shall adopt the most robust setting: challenging inputs such as Arena Hard, high-performance evaluation models such as GPT-4o or GPT-4-turbo, or Llama-3.1-70B in open-source settings, and binary pairwise comparisons aggregated via Bradley-Terry modeling. Moreover, the finding that the accuracy of instance-level ranking does not predict the effectiveness of system-level ranking further supports the use of system-level correlation metrics as the main evaluation criterion. This can be extended by the framework by adding robustness validation tests to check whether small changes in input or evaluator configuration lead to disproportionate shifts in ranking. Lastly, the recognition of the observed degradation in ranking precision for highly matched models justifies the addition of hybrid evaluation strategies such as partial human verification or multi-bencher ensembles to maintain reliability when automatic judgments become uncertain. Accordingly, this research not only lends empirical support to the design direction of LLM-Ranker but also provides prescriptive insights into its stability and interpretability.

3.2.15 G-EVAL: NLG Evaluation with GPT-4 (2023)

3.2.15.1 Summary of the research item

The paper G-EVAL: NLG Evaluation with GPT-4 proposes a new framework for automatic evaluation in NLG by turning LLMs into expert evaluators. Traditional reference-based metrics like BLEU and ROUGE have shown limited correlation with human judgments in open-ended or creative text generation tasks. G-EVAL addresses this limit by employing a reference-free, prompt-based evaluation process. The three core components in this framework are the task-specific prompt specifying the evaluation criteria, an automatically generated Chain-of-Thought guiding reasoning during assessment, and

a scoring function based on a form-filling paradigm. More importantly, to improve scoring precision, it offers a probability-weighted aggregation of rating tokens, which is capable of delivering fine-grained continuous scores rather than discrete integers. Empirical results show that G-EVAL, particularly its GPT-4 variant, G-EVAL-4) reaches the highest correlation with human evaluations across a variety of NLG tasks, including dialogue generation and summarization, and significantly outperforms all prior state-of-the-art automatic evaluators [8].

3.2.15.2 Critical analysis of the research item

G-EVAL framework represents a significant improvement in the evaluation based on LLMs, mainly due to its ability to achieve a strong alignment with human judgment. The problem of tied results often found in discrete rating systems is thus effectively mitigated by the introduction of probability-normalized continuous scoring, while the incorporation of CoT-based reasoning enhances the depth and consistency of evaluation. The study points out a “self-reinforcement” phenomenon: G-EVAL-4 at times overestimates the quality of GPT-generated outputs over human ones, showing thus its possibly systemic bias in an LLM-based evaluation. If such evaluators are used as reward models in fine-tuning, they might drive models to optimize for their own preferences instead of genuine human quality criteria. Whereas probability weighting captures subtle differences, integer scoring sometimes demonstrates better rank correlation due to the absence of tie penalties. Overall, the methodological novelty of the paper is enormous but again points to careful calibration and human validation when LLMs are used as evaluators.

3.2.15.3 Relationship to the proposed research work

The proposed project, LLM ranker, would directly benefit from adopting G-EVAL’s evaluation framework. Specifically, probability-weighted continuous scores perfectly match the objective of ranking LLM outputs according to subtle differences in performance. Moreover, G-EVAL’s structured approach, prompt-based evaluation with explicit criteria and auto-generated Chain-of-Thought reasoning can be a guideline for designing transparent and interpretable evaluation prompts in the context of LLM-Ranker. Yet, the findings of this study also indicate that any automated evaluator, including a GPT-4-based scoring system, may be biased to machine-generated responses. The design of LLM-Ranker should thus be embedded with bias mitigation and human calibration checks that ensure its rankings reflect true human-like quality assessment rather than self-reinforcing model preferences. In this respect, G-EVAL plays the role of a foundational model as well as a cautionary reference towards the development of robust unbiased ranking pipelines.

3.2.16 Holistic Evaluation of Language Models (HELM)

3.2.16.1 Summary of the research item

The HELM paper suggests an extensive framework that would improve transparency and further insights into the manner of Large Language Models (LLMs) evaluation. Instead of using only one measure of success, HELM is using a triple strategy: it builds a comprehensive taxonomy of situations and measures in order to make sure that no absolutely crucial aspect is forgotten; it also compares models on seven different dimensions, thus offering a multidimensional evaluation; and it standardizes the process of evaluation to make sure that comparisons become truly meaningful. The authors compared thirty different language models on sixteen core and seven special cases. The standardized benchmarking coverage increased exponentially to 96.0 17.9. The framework, in turn, results in a comprehensive performance matrix of each model, and clearly indicates the trade-offs between the different tasks and metrics. [9].

3.2.16.2 Critical analysis of the research item

The main contribution made by HELM is the reinvention of a new paradigm of evaluation away from the outdated emphasis on the accuracy per se and in to a complex, multidimensional system . This change is also fundamental due to the fact that transparency and strength are paramount especially in the application of the models in real life situations where fairness and toxicity has direct implications on the affected populations. The major surprise of the study is that accuracy, robustness and fairness have strong correlation in all models tested, as was not expected considering the common belief that such characteristics would have tradeoffs. The authors clearly state the weaknesses of their framework. Few-shot prompting can also be highly volatile; small formatting changes can lead to an extreme decrease in accuracy of a given model, such as the case with the accuracy of the model of HellaSwag, where it dropped by 79.1 to 30.2 percent with one particular change in prompting. The paper also describes such challenges as traintest contamination and the fact that it is impossible to discuss every possible situation. The fact that holistic evaluation is not complete is a show of intellectual honesty and responsibility on the part of the authors [9].

3.2.16.3 Relationship to the proposed research work

The design of our LLM response-ranking system is informed directly by the philosophy of multi-metric that guides HELM. The framework emphasizes the fact that one ranking score is not enough, but performance needs to be evaluated in multiple dimensions such as robustness, fairness, and toxicity. This realization informs our attitude towards considering efficacy as well as accountability. Also, the fact

that HELM focuses on standardized appraisal, i.e. using similar scenarios and conditions, serves to strengthen the significance of controlled appraisal in our project. With the same approach, using the methodology of HELM, we will be able to provide our ranking system with fair, reproducible and transparent results that the stakeholders would trust and understand [9].

3.2.17 Scalable Reliable Evaluation Using LLM Pairwise Comparisons and Elo

3.2.17.1 Summary of the research item

One of the studies proposes an automated evaluation system, which measures the quality and relevance of the text generated by LLM through systematic pairwise comparisons and Elo-based ranking. The approach does not rely on established metrics or reference comparisons but requests the various LLMs to compare their output in a head-to-head fashion and later aggregates the scores through the Elo rating system to produce dynamic rankings. This method results in consistent, explainable rankings that are very similar to those of human rating. The structure provides scholars with fine grain control over both the confidence and coverage by setting the agreement thresholds up to full agreement or more permissive standards may be utilized, depending on the requirements. The system was found to be strongly correlated with expert judgments when applied to competency-profiling tasks that made use of scientific abstracts, which indicates that the system is scalable, reliable, and also lowers the cost of relying on costly human evaluation [10].

3.2.17.2 Critical analysis of the research item

The paper provides a sound methodological basis of scalable automated assessment and makes a contribution to the developing area of the LLM-as-a-judge systems. It is also especially innovative in the introduction of Elo aggregation, which offers a mathematically rigorous method to transform noisy pairwise judgments into reliable rankings which helps alleviate the inconsistencies that are often seen in single-judge judgments. However, there are still computational problems. The number of items rated by a pairwise comparison quadratically scales ($O(n^2)$) with the scale size, thus inference can be very expensive when using large models. Also, when the judgment on the evaluated outputs is semantically close, the results of the judges by the LLM might be inconsistent, particularly with the different model acting as a judge. The authors warn of the illusion of objectivity: a well-calibrated automated rating is not a guarantee that all domain-sensitive nuances that are familiar to expert human ratings are captured. They therefore recommend the use of human-in-the-loop auditing which is necessary to maintain quality and also to identify edge cases that automated systems may fail to identify [10].

3.2.17.3 Relationship to the proposed research work

This study is directly applicable to our project [10]. The three-stage process they propose—Item Generation, Pairwise Comparison, and Elo Rating—gives us a ready-made blueprint for our system architecture. Adopting Elo aggregation will help us effectively consolidate pairwise judgments into ordered, quantitative rankings, which addresses one of the core challenges in multi-model response evaluation.

We’re also planning to incorporate several of their best practices: bidirectional comparisons to control for position bias, multiple LLM judges to improve reliability, and Chain-of-Thought prompting to enhance transparency. The Elo-based scoring system is particularly valuable because it provides more than just ordinal ranking—the score differentials actually communicate the magnitude of quality differences between competing LLM responses, which makes the results more interpretable and useful [10].

3.2.18 Chatbot Arena: Evaluating LLMs by Human Preference

3.2.18.1 Summary of the research item

Chatbot Arena represents a fundamentally different approach to benchmarking LLMs—instead of relying on static metrics and pre-defined datasets, it evaluates models based on what real humans actually prefer [11]. The platform is refreshingly simple in concept: users submit prompts to two anonymous LLMs, review both responses, and vote for the one they prefer.

The scale is impressive: since launching, the platform has collected over 240,000 human votes covering more than 50 models and 100+ languages. Rankings are computed using the Bradley-Terry (BT) model combined with an adaptive sampling algorithm that ensures fast and stable convergence. The data analysis shows that the human judgments collected are diverse, discriminative, and correlate strongly with expert evaluations—which is exactly what you want to see [11].

3.2.18.2 Critical analysis of the research item

Chatbot Arena marks a real turning point in how we evaluate LLMs through human preference [11]. Its greatest strength is the combination of methodological rigor—using statistically sound ranking models—with practical scalability through crowdsourced data collection. The Bradley-Terry model offers more robust statistical estimation compared to simpler systems like Elo, and their active sampling approach substantially improves efficiency by avoiding redundant comparisons that don’t provide much new information.

The empirical results are encouraging: agreement rates between crowdsourced and expert ratings reached as high as 89.8%, which validates the platform’s reliability. This suggests that with the right setup,

crowdsourced evaluation can be both accurate and representative.

However, the authors are upfront about potential limitations. There’s a bias issue in user demographics—participants tend to be LLM enthusiasts, which may not represent the general population. This matters because different user groups might have different preferences and priorities. Additionally, the evaluation scope primarily focuses on helpfulness while largely neglecting other important dimensions like safety and ethical considerations. This is a significant gap that limits the platform’s ability to provide truly holistic assessment [11].

3.2.18.3 Relationship to the proposed research work

This research directly informs our approach to ranking LLM responses using both automated and human-influenced methods [11]. The framework and methodology that Chatbot Arena has developed and tested at scale provides a proven blueprint for building a reliable ranking pipeline. Specifically, we can integrate the pairwise comparison paradigm with the Bradley-Terry model to derive consistent, interpretable rankings that reflect real user preferences.

Moreover, the adaptive sampling technique they discuss can help us optimize our data collection by prioritizing comparisons that maximize ranking confidence—essentially getting the most information out of each evaluation. Lastly, Chatbot Arena’s transparent approach to reporting crowd agreement and implementing quality control strategies offers valuable guidance on maintaining reliability in our system. This makes the paper a cornerstone reference for designing, scaling, and validating our ranking approach [11].

3.3 Literature Review Summary Table

In the process of this research, we reviewed various existing studies in the domain of Large Language Model evaluation and ranking. We observed that many different frameworks, methodologies, and metrics have been deployed to assess model outputs. Table 3.1 follows with each study, detailing the author(s), the method employed, the principal findings, and identified limitations.

Table 3.1: Summary of Relevant Research Studies for LLM Evaluation and Ranking

Author	Method	Results	Limitations
Wang et al. [12]	Proposed <i>TrustJudge</i> , a probabilistic framework addressing inconsistencies in LLM-as-a-judge evaluations through distribution-sensitive scoring and likelihood-conscious aggregation.	Improved score-comparison consistency by 8.43 and pairwise transitivity by 10.82 compared to baseline methods.	Relies on LLM judgments; limited external validation against human evaluators.
Liu et al. [13]	Investigated position bias in LLM-as-a-judge systems using repetition stability, position consistency, and preference fairness metrics over 100,000+ evaluation cases.	Identified judge-, candidate-, and task-level factors affecting position bias; highlighted inconsistencies among judges and tasks.	Primarily diagnostic; lacks mitigation strategies beyond dataset modifications; metrics focus on bias rather than comprehensive quality assessment.
Pombal et al. [1]	Proposed Zero-shot Benchmarking (ZSB) framework using meta-prompts and judgment prompts to generate and evaluate test data for LLMs automatically.	High correlation with human judgment; scalable, flexible, and adaptable to multiple tasks, modalities, and languages.	LLM bias can influence results; limited interpretability; generated prompts may not reflect real-world diversity.

Continued on next page

Table 3.1 continued from previous page

Author	Method	Results	Limitations
Liu et al. [14]	Surveyed LLM evaluation dimensions: what to evaluate, where to evaluate, and how to evaluate across datasets, benchmarks, and methods.	Comprehensive overview of evaluation methods and benchmarks; identification of gaps in holistic, adaptive, and fair assessment.	Focused on description rather than practical measurement; limited human feedback integration; fixed benchmarks may lag behind rapid LLM development.
Zhang et al. [15]	Proposed UDA (Unsupervised Debiasing Alignment) to reduce bias in pairwise LLM-as-a-judge evaluations via dynamic K-factor adjustment and consensus optimization.	Reduced inter-judge rating variance by 63.4%; improved correlation with human ratings by 24.7%.	Limited to pairwise comparisons; dynamic adjustments are less interpretable; assumes consensus equals fairness.
Yin et al. [2]	Proposed Decentralized Arena (<i>Dearena</i>), where LLMs evaluate each other collectively using coarse-to-fine ranking and dynamic question selection.	Achieved 97% human-level evaluation performance across 66 LLMs; highly scalable and efficient.	LLM self-assessment may reinforce biases; limited interpretability; focuses on efficiency over multidimensional quality.
Yu et al. [16]	Introduced <i>xFinder</i> , enabling LLMs to act as automated evaluators for other LLMs via cross-evaluation and agreement-based calibration.	Showed strong correlation with expert human judgment; scalable and cost-efficient evaluation.	Risk of collective bias among models; limited qualitative reasoning analysis; needs validation across task types.

Continued on next page

Table 3.1 continued from previous page

Author	Method	Results	Limitations
Li et al. [17]	Proposed <i>GEM</i> , a reference-free framework estimating mutual information between outputs and a latent reference distribution for unbiased evaluation.	High alignment with human judgment; robust against manipulation and contamination.	Model dependency; limited task diversity; low interpretability for non-technical stakeholders.
Wang et al. [18]	Studied fairness and positional bias in LLM evaluators; introduced <i>FairEval</i> toolkit with calibration via multiple evidence and balanced position averaging.	Reduced bias and improved correlation with human judgments across model comparisons.	Increases computational overhead; focuses mainly on position bias; limited multi-model generalization.
Chen et al. [3]	Developed Active Evaluation Acquisition (AEA) framework using reinforcement learning to select informative subsets of test data for benchmarking.	Achieved high benchmarking accuracy while reducing evaluation cost and sample size.	Depends on RL policy generalization; limited for open-ended or creative tasks; training overhead may offset savings.
Zhou et al. [4]	Proposed <i>MELD</i> , a fine-grained hierarchical evaluation framework with 8 categories and 50 subcriteria for detailed LLM analysis.	Provided multi-level diagnostic insights into LLM strengths and weaknesses.	High complexity; costly to maintain; potential redundancy among subcriteria.

Continued on next page

Table 3.1 continued from previous page

Author	Method	Results	Limitations
Huang et al. [5]	Built enterprise-oriented benchmarks covering 25 datasets across domains like finance, law, and cybersecurity.	Showed large performance variance among models; improved domain-specific insight.	Uses open-access datasets; lacks task realism and context-based analysis.
Park et al. [6]	Re-examined automatic benchers, analyzing sensitivity of prompt design, scorer choice, and aggregation methods on rankings.	Found small design changes can drastically alter model rankings; urged calibrated evaluation pipelines.	Lacks prescriptive correction framework; limited domain coverage; no human feedback integration.
Lin et al. [7]	Analyzed architecture of modern automatic LLM benchers, decomposing scoring and alignment components.	Provided insights into structural causes of inconsistency; recommended modular benchmark design.	Primarily conceptual; lacks empirical validation; assumes uniform task difficulty.
Liu et al. [8]	Proposed <i>G-EVAL</i> , leveraging GPT-4 for natural language generation evaluation using natural language rubrics and task-specific criteria.	Improved agreement with human evaluators; flexible across NLG tasks.	Limited to GPT-4-based systems; potential self-bias; needs more generalizable validation.
Liang et al. [9]	Developed <i>HELM</i> , a holistic evaluation framework covering multiple scenarios and metrics (accuracy, calibration, robustness, fairness).	Provided standardized, transparent multi-dimensional LLM benchmarking.	Computationally intensive; limited task diversity; static benchmarks struggle with rapid model evolution.

Continued on next page

Table 3.1 continued from previous page

Author	Method	Results	Limitations
Koh et al. [10]	Proposed scalable evaluation using Elo-based pairwise comparison and reliability calibration for large LLM pools.	Enabled continuous, robust ranking with minimal comparisons; correlated with human preference.	Relies on pairwise evaluation only; assumes transitivity; lacks task-specific granularity.
Zheng et al. [11]	Introduced <i>Chatbot Arena</i> , an open benchmarking platform evaluating LLMs through human preference battles.	Captures real-world preference data; democratizes evaluation and leaderboard formation.	Human-dependent; subjective judgments; scalability limited by participant diversity.

3.4 Conclusion

This chapter surveys the rapid development of evaluation frameworks and ranking methodologies for LLMs. Various studies, such as TrustJudge, UDA, and FairEval, emphasize the need to avoid bias and inconsistency in LLM-as-judge settings, while frameworks like ZSB, Dearena, and Chatbot Arena focus on scalability, democratization, and alignment with real-world preferences. Fine-grained and domain-oriented methods, such as MELD and Enterprise Benchmarks, go beyond generic metrics for task-specific and contextual understanding. Reference-free models like GEM and holistic evaluations like HELM show the growing interest in transparency, fairness, and adaptive benchmarking. Overall, the literature reviewed here reflects that no single framework offers comprehensive insights; hence, motivating the creation of hybrid systems like LLM-Ranker, combining quantitative metrics with human-in-the-loop validation and retrieval-based ranking to make fair, interpretable, and scalable assessments of LLM performance across diverse domains and contexts.

Chapter 4 Software Requirement Specifications

The chapter is the official description of the complete software requirements of the LLM -Ranker System which is why it is a blueprint upon which the design and further implementation are based. It lists both functional and non-functional requirements in a systematic manner, which guarantees clarity, accuracy and correspondence with the project objectives in general. First, the chapter elaborates on the main features and quality measurements that the system should have. It subsequently outlines functional and non-functional specifications, assumptions of the system as well as use-case scenarios that all define how stakeholders will interact with the system. The following sections discuss the required hardware and software specifications, user interface design, and database structure, and analyze possible threats and state mitigation measures. Taken together, these specifications comprise a unique technical framework which allows the successful creation and strict testing of the LLM-Ranker system.

4.1 List of Features

The following are the main features that will be included in the LLM Ranker:

- **Dataset Compilation:** The system will create datasets, by having responses of multiple large language models to the same set of prompts that cover different domains including general knowledge, reasoning, creative writing, and programming tasks.
- **Automated Evaluation:** The system will automatically calculate a set of evaluation metrics, such as coherence, grammatical correctness, readability, and semantic similarity.
- **Ranking Mechanism:** The system will rank the results of the different LLM according to the metric scores aggregated.
- **Comparative Analysis:** It will generate comparative reports which will compare the abilities and weaknesses of various LLMs on various tasks.
- **Visualization Dashboard:** This will be the access to graphical visualization diagrams, such as charts, tables, heatmaps, etc., to allow users to easily analyze the results.
- **Multilingual Support:** The system will support assessment capabilities to other languages other than English and hence multilingual benchmarking.

4.2 Quality Attributes

The following attributes will define the quality of the system:

- **Accuracy:** Processes of evaluation and ranking should be accurate in terms of intrinsic quality of responses.
- **Fairness:** The system must address the bias that may exist like positional bias or overreliance on single metrics.
- **Scalability:** The system should be able to accommodate high data sizes, which includes thousands of inputs and several LLMs.
- **Reliability:** The system should provide the same results when used to make repeat assessments.
- **Usability:** It must be easy to use and to be intuitive to both non-technical and technical users.
- **Extensibility:** The architecture should be such that adding new LLMs and metrics, should be easy.
- **Transparency:** The assessment process should be such that the user can clearly and with transparency see that how the ranking are made.
- **Report Generation:** The system will generate reports both tabular format and visual format to facilitate comparative studies.
- **Export Functionality:** The system will allow exporting the results in common formats like CSV and PDF.
- **Filtering Capability:** The system will enable the user to filter the results based on the domain, model, or metric that is needed.

4.3 Functional Requirements

The system will meet the following functional requirements:

- **Prompt:** The system will enable users to input prompts and to upload or harvest responses of various LLMs.
- **Automated Scoring:** The system will automatically calculate assessment scores of every response according to established measures.
- **Data Storage:** The system will store the results in a database to enable effective subsequent retrieval.
- **Report Generation:** The system will be able to produce both visual and tabular reports to compare them.
- **Export Support:** The system will offer the option to export the outcomes in a standard format

including CSV and PDF.

- **Filtering and Querying:** The system will allow querying and filtering of results based on domain, model, or metric.

4.4 Non-Functional Requirements

The following non-functional requirements need to be met:

- **Performance:** The system should perform evaluations in a reasonable timeframe.
- **Platform Compatibility:** It should be able to support significant operating systems such as Windows and Linux.
- **Data Persistence:** The system should provide data persistence using a stable database solution.
- **Multilingual Processing:** Multilingual data needs to be processed free of errors or encoding issues.
- **Security:** The system should provide secure storage and protect the evaluation data from unauthorized access.
- **Real-time Interaction:** The dashboard must have real-time interaction with minimal delay.

4.5 Assumptions

The following are the assumptions being made for the creation of this project:

- All the necessary LLM responses will be available either by APIs or dataset.
- Valid and correctly formatted prompts will be offered by users for evaluation.
- Enough computing resources (RAM, CPU, GPU) will be present for performing large-scale evaluations.
- Internet access will be present for the retrieval of responses from LLM APIs.

4.6 Use Cases

The following tables (4.1–4.3) provide detailed descriptions of the system’s use cases.

Table 4.1: Use Case: Upload Dataset

Field	Attribute	Description
-------	-----------	-------------

Name	Upload Dataset	Allows the user to upload a dataset for evaluation by the LLM-Ranker system.
Actors	Researcher, Developer, End User	Individuals who wish to evaluate LLMs using their datasets.
Summary	Dataset Upload	The user uploads a dataset in JSON or CSV format to be processed by the system.
Pre-Conditions	Valid Dataset Format	The dataset must be properly formatted (JSON/CSV) and within the size limit.
Post-Conditions	Dataset Stored	The dataset is successfully uploaded and stored for later processing.
Special Requirements	File Limitation	File size should not exceed 50 MB.
Basic Flow		
Step	Actor Action	System Response
1	User clicks on "Choose a File".	System opens the file browser dialog.
2	User selects a valid dataset file.	System uploads and validates the dataset.
3	User confirms upload.	System stores the dataset and shows confirmation.
Alternative Flow		
3-A	User selects an invalid file type.	System shows error: "Invalid file format."

Table 4.2: Use Case: Rank LLM Responses

Field	Attribute	Description
Name	Rank LLM Responses	Enables the user to initiate ranking of model outputs based on multiple evaluation metrics.
Actors	Researcher, Developer	Users responsible for evaluating and ranking LLM responses.
Summary	Response Evaluation	The system ranks responses from different LLMs using selected evaluation metrics.

Pre-Conditions	Dataset Availability	A valid dataset containing responses from multiple LLMs must be uploaded.
Post-Conditions	Rankings Generated	Rankings are produced and displayed in the user dashboard.
Special Requirements	Evaluation Metrics	The ranking process should consider coherence, and relevance.
Basic Flow		
Step	Actor Action	System Response
1	User clicks “Train” to evaluate responses.	The system initiates the ranking pipeline.
2	User waits for processing.	System applies selected metrics to evaluate and rank the responses.
3	User clicks “Results”.	System displays the ranked list of LLMs.
Alternative Flow		
2-A	System faces processing error.	System shows error: “Ranking pipeline failed.”

Table 4.3: Use Case: View Rankings

Field	Attribute	Description
Name	View Rankings	Allows the user to view and interpret the comparative rankings of LLMs.
Actors	End User, Researcher	Users who wish to visualize and analyze ranking outcomes.
Summary	Ranking Visualization	The user views the ranking results of multiple LLMs on the system dashboard.
Pre-Conditions	Rankings Available	Rankings must already be generated and stored.
Post-Conditions	Results Accessible	The user can view and optionally download the ranking results.
Special Requirements	Interactive Display	Results should be displayed in a clear, interactive, and user-friendly format.
Basic Flow		

Step	Actor Action	System Response
1	User clicks “Re-sults”.	System shows the ranked list of LLMs.
2	User clicks on a model name.	System expands details of evaluation scores.
3	User chooses to download results.	System exports rankings in CSV/JSON format.
Alternative Flow		
2-A	User clicks “Re-sults” but no ranking exists.	System shows message: “No ranking generated yet.”

4.7 Hardware and Software Requirements

4.7.1 Hardware Requirements

- **Processor:** Multi-core CPU (Intel i7 or higher).
- **Memory:** At least 16 GB RAM.
- **Storage:** Minimum 100 GB disk space for data and logs.
- **GPU:** Optional, recommended for embedding-based semantic comparisons (e.g., NVIDIA RTX 3060 or higher).
- **Network:** Reliable internet connectivity with a minimum speed of 10 Mbps.

4.7.2 Software Requirements

- **Operating System:** Windows 10 or Linux Ubuntu 20.04+.
- **Language:** Python version 3.9 or above.
- **Libraries:** Hugging Face Transformers, NLTK, Scikit-learn, PyTorch/TensorFlow.
- **Database:** PostgreSQL or MongoDB.
- **Visualization Tools:** Streamlit, Dash, or Plotly.
- **APIs:** Perplexity, Copilot, Claude, Chat GPT, Gemini.

4.7.3 The Graphical User Interface

The Graphical User Interface (GUI) of the LLM Ranker System forms the main operational context by which users can access and utilize all of the main functionalities in the form of an accessible and convenient way. It is designed in such a way that it is simple, interactive and user friendly, thus allowing technical and non-technical parties to enjoy a smooth ride. The key elements of the interface are outlined below.

4.7.3.1 Search Bar

Being at the top of the interface, the search bar gives the user a specific section to enter queries or prompts to test and compare the responses returned by different language models.

4.7.3.2 Model Selection Panel

The left-hand sidebar provides a selection panel through which users are allowed to select AI models to be evaluated and compared. Under this panel, there is the Chats section where past conversations and interactions are stored and as such, users can access or reuse past sessions held.

4.7.3.3 Uploading and Training the Dataset(s)

This area is placed in the middle of the interface and allows uploading datasets in the JSON or CSV format to train the models. Images and PDF files (not more than 50 MB) can be uploaded using the drag-and-drop feature as well. There are two main buttons, Train and Results, where one can start the evaluation process and see the created rankings, respectively.

4.7.3.4 Rankings Panel

The right side shows the AI models sorted according to their predicted performance like, Copilot, Perplexity, Claude, Gemini. Every LLM row shows its rank and concise description telling its evaluation result.

4.7.3.5 User Profile Icon

The users profile icon shown on the upper left corner open account settings and management options which lets a user customize their preferences and can logout safely if they want to.

Figure 4.1 shows the main interface of the LLM Ranker system.

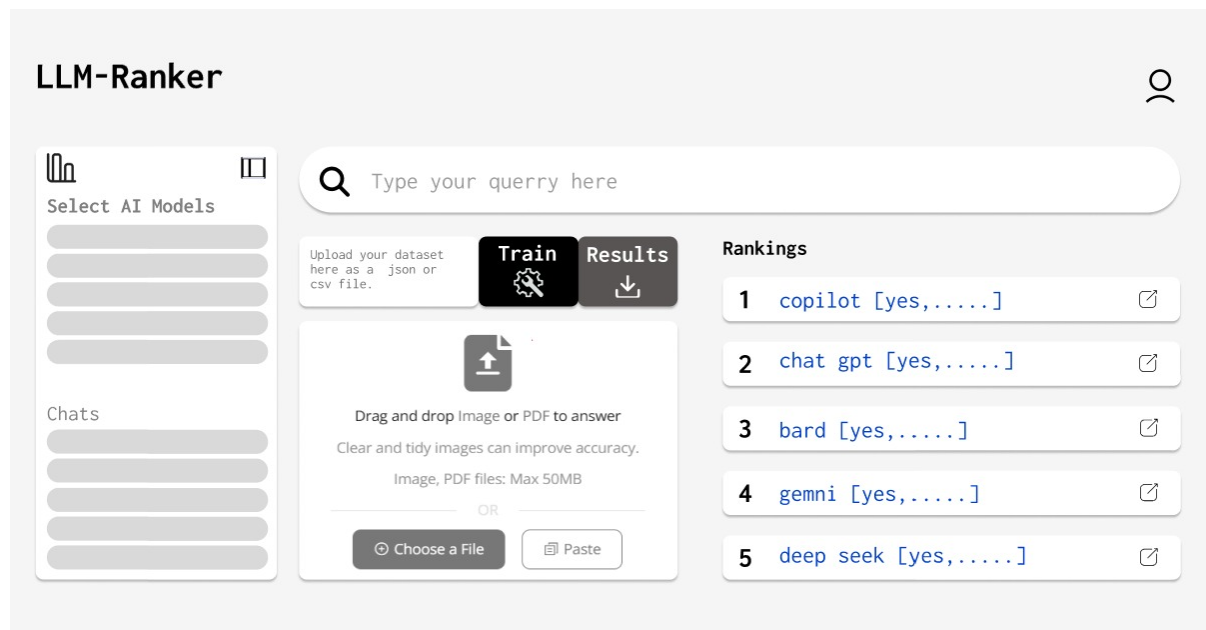


Figure 4.1: GUI of the LLM-Ranker system.

4.8 Database Design

The design database of the system of the LLM Ranker defines the organization, storage and relationship of data. It provides effective management of users, prompts, model responses, evaluation measures, and prioritizing outcomes. The design provides a powerful relational framework, which facilitates the performance assessment and ranking of big language models.

4.8.1 ER Diagram

Figure 6.1 shows the interrelationship between the important entities like Users, Prompts, Models, Responses, Evaluation Metrics, Evaluation Scores, Ranking Results, and Feedback. Both entities contain the necessary information that is needed to create, assess, and prioritize model responses. The connections between them determine the way of how users build prompts, models generate responses, evaluations are done using metrics and rankings are produced.

- Every user will have the option of creating several prompts.
- Every prompt will produce a number of responses produced by various models.
- The responses are rated on several metrics, and the evaluation scores are obtained.
- The results are ranked on each prompt on the basis of composite scores.
- Users can give feedbacks on responses by rating and commenting.

Figure 6.1 contains the ER model that depicts these relationships



Figure 4.2: Entity-Relationship Diagram of the LLM-Ranker System

4.8.2 Data Dictionary

The Data Dictionary provides much information about each table in the LLM Ranker database, attributes, data types, descriptions. Details of these organizations are given in the **Tables (4.4 - 4.8)** below.

Table 4.4: Data Dictionary: Users Table

Attribute	Data Type	Description
UserID	INT (PK)	Unique identifier for each user.
Username	VARCHAR(50)	Name chosen by the user for login.
Email	VARCHAR(100)	User's email address.
Password	VARCHAR(255)	Encrypted password for authentication.
Role	VARCHAR(20)	Defines user type (Admin/User).

Table 4.5: Data Dictionary: Prompts Table

Attribute	Data Type	Description
PromptID	INT (PK)	Unique identifier for each prompt.
UserID	INT (FK)	References the user who created the prompt.
PromptText	TEXT	The text of the prompt entered by the user.
CreatedAt	DATETIME	Timestamp of prompt submission.

Table 4.6: Data Dictionary: Responses Table

Attribute	Data Type	Description
ResponseID	INT (PK)	Each response has a Unique Identity value.
PromptID	INT (FK)	Unique id for referencing the obtained prompt.
ModelName	VARCHAR(100)	Name of LLM that wrote the response or gave it.
ResponseText	TEXT	Response text given by the LLM.
CreatedAt	DATETIME	Tells the time when the response was stored.

Table 4.7: Data Dictionary: EvaluationMetrics Table

Attribute	Data Type	Description
MetricID	INT (PK)	Unique identifier for each evaluation metric.
ResponseID	INT (FK)	References the evaluated LLM response.
MetricName	VARCHAR(50)	Name of the evaluation metric (e.g., BLEU, ROUGE, etc.).
Score	FLOAT	Calculated score for the response.

Table 4.8: Data Dictionary: Rankings Table

Attribute	Data Type	Description
RankingID	INT (PK)	Unique identifier for each ranking record.
PromptID	INT (FK)	References the evaluated prompt.
BestModel	VARCHAR(100)	The top-ranked model based on evaluation metrics.
AverageScore	FLOAT	Average score used to determine ranking.
RankedAt	DATETIME	Timestamp when ranking was finalized.

4.9 Risk Analysis

aw LLM Ranker system is likely to face a number of possible threats in its development and implementation. **Table 4.9** gives an overview of the risks and how they can be mitigated.

Table 4.9: Risk Analysis for LLM-Ranker System

Risk	Description	Mitigation Strategy
Bias in Evaluation	LLM-as-a-judge approaches can exhibit bias, affecting the fairness and accuracy of rankings.	Utilize diverse evaluation metrics and integrate human feedback to balance automated judgment.
Scalability Issues	Evaluating and ranking multiple model responses can slow down system performance.	Implement efficient processing pipelines, asynchronous evaluation, and caching mechanisms.
API Restrictions	Proprietary LLMs may impose access limitations or usage quotas.	Incorporate open-source LLMs (e.g., LLaMA, Falcon) as fallback options.
Metric Limitations	Individual evaluation metrics cannot fully capture all aspects of response quality.	Use a hybrid approach combining quantitative metrics and qualitative assessments.
Data Security	Sensitive prompts or responses could be exposed during transmission or storage.	Apply encryption, access control, and secure storage mechanisms to protect data.

4.10 Conclusion

The LLM Ranker system has been designed to offer a clear, impartial, and scalable system of assessing and ranking reactions created by various LLMs. The system is capable of providing the consistent and reliable benchmarking of different tasks and domains by combining automated evaluation metrics with the human-in-the-loop evaluations. Moreover, multilingual support, response-comparison visualization, and extensibility make LLM Ranker a useful tool that allows researchers, developers, and industry representatives to learn to work with and improve the performance of LLMs.

Chapter 5 Proposed Approach and Methodology

The suggested LLM-Ranker algorithm is an important step forward in the assessment of large language models (LLMs) in that it overcomes the constraints of both fixed benchmarks and human-scoring paradigms, thus providing a hierarchical and understandable assessment system.

The investigation is mostly undertaken by the use of a novel retrieval based appraisal system in which the output of any particular response is not only confirmed but also the corroborating fact based contextual information is retrieved in order to objectively prove the accuracy and contextual applicability of the output. This process is supplemented by a multi-dimensional scoring system which breaks down performance in different quality measures, consistency and fact-based grounding, hence providing a clear and complete explanation of the way an LLM is eventually ranked. The balanced approach is quite suitable to cover the gap between data-based judgment and human judgment in a way that enables the final rankings to be not only accurate and scalable but objective and easy to interpret.

5.1 System Overview

The LLM-Ranker architecture is scalable, cross-model-compatible and was designed with a modular architecture. It is made up of two major layers:

5.1.1 Frontend

The Frontend Layer represents the main part of the graphical user interface of the system that is created on the basis of a modern web framework to assure responsiveness, accessibility, and ease of use. It will have the ability to achieve smooth interaction, live visualization, and effective presentation of model evaluation outcomes.

5.1.1.1 Engagement Interface

A clean interface is provided in the frontend which provides easy and efficient interaction between the system and the user. It assists in the choice of models, setup of parameters, and easy visualization of model ranks using well-organized layouts and dynamic elements of user interfaces.

5.1.1.2 Data Input

The users can import data sets either in the form of JSON or CSV or just input selected prompts through the interface. This feature also allows the evaluation engine to be inoculated with custom inputs thus allowing a tailored test of the system. The system then produces and renders ranked model responses on

the fly, and thus, accelerates the analytical workflow and experimental iteration.

5.1.1.3 Visual Analytics

The interface is made up of heavily pictorial visual depictions, tabular facts sets, and elaborate analytical visualizations of model responses. These resources enable researchers to make an in-depth analysis of the model performance that allows them to see not only the general rankings but also the particular assessive indicators like Coherence and Relevance.

5.1.1.4 Feedback Integration

A feedback system built in-house, e.g. thumbs-up/down rating or a direct score modification, allows the user to measure the output of a model. The obtained feedback is used to adjust the ranking weights, forming a learning process cycle, which is repeated, increasing the adaptiveness of the system to the preferences of the users and aligning it with their preferences.

5.1.2 Backend Layer (Computation and Evaluation Engine)

The Backend Layer is the computational core of the LLM -Ranker system and performs the major logic of ranking and evaluation. It provides a faultless communication between the database and the external language model APIs and the frontend interface.

5.1.2.1 API Management

It is also a component that handles efficient and safe communication with a variety of large language model (LLM) APIs, including Claude, ChatGPT, and Gemini. It manages submission of requests in good time, retrieval of response and co-ordination of the complete evaluation pipeline to ensure a steady and dependable flow of data.

5.1.2.2 Multi-Metric Scoring

The backend conducts a more in-depth scoring of model outputs on various dimensions of important quality:

- **Semantic Similarity:** Is the degree to which the meaning of the production resembles that of a reference or ideal response.
- **Coherence:** Tells how well linked the ideas are in the responses.
- **Readability:** How easily can a person read or understand the response.

5.1.2.3 Weighted Aggregation:

The system then computes individual scores, and then adds them up into a single composite performance score through a weighted ranking algorithm (a description can be found in 5.3). The final ranking process is formulated in such a way that it allows a sensible and sensible model comparison, reaching a compromise between quantitative accuracy and qualitative integrity.

5.2 Workflow and Data Flow

The LLM-Ranker model used has a five stage data flow pipeline that ensures the effective processing of data, proper evaluation and continuous improvement of the model by user feedback. The operational cycle comprises all the steps, including ingestion of data and ranking of models as well as user feedback whilst being transparent, consistent, and adaptable.

5.2.1 Dataset Collection and Preprocessing

First, users post prepared data sets that include prompts, queries or interrogatives, which are to be tested. The supported file formats are CSV and JSON. The backend performs a great number of crucial operations thus maintaining data integrity and preparedness after ingestion:

- **Validation:** Structural conformity, format compliance and completeness are automatically validated by the system thus eliminating the probability of missing or damaged records.
- **Tokenization and Cleaning:** Prompts are tokenized into small and easy to handle pieces and sanitized by removing unwanted symbols or characters that might corrupt the quality of evaluation.
- **Metadata Storage:** Data like the time of uploading the data, name of the dataset, user name and the number of prompts will be stored safely in the database to be used in future and to trace the information.

The stage will ensure that only quality data that is valid is entered into the system so that the model is evaluated properly and consistently.

5.2.2 Response Generation

The system moves on after the preprocessing stage to the Response Generation stage. In this case, the uploaded prompts are dispatched to several integrated Large Language Models (LLM) like ChatGPT, Claude or Gemini through their respective APIs. The backend ensures:

- Protected and effective communication with every model API.

- Multitasking of requests to speed up the collection of responses.
- Vital data on timestamps, latency, token consumption and cost indicators is logged.

All the responses are stored in a categorized schema in the database which allows a fair comparison between models.

5.2.3 Metric Calculation

It is the computational heart of the pipeline as a multidimensional scoring matrix is used to measure the responses of each model against a variety of criteria. Both qualitative and quantitative aspects are represented:

- **Semantic Similarity:** The response of the model is contrasted to the ground truth with the help of such tools as BERTScore or embedding space cosine similarity.
- **Coherence:** This evaluation criterion is a structural examination that examines logical coherence and internal consistency of the created content.
- **Readability and Clarity:** The human accessibility of the response is determined in terms of the ease of understanding.

These processes standardize raw written products into objective performance measures which can be ranked fairly to form the basis of transparent ranking.

5.2.4 Composite Ranking

After completing the individual metric calculations, the system starts the Composite Ranking stage, which is the function center of the operations of the LLM-Ranker. Each model has a normalized score in each of the measured metrics which are then summed with a weighted equation to produce a composite score. The aggregation may be made in the following way:

$$\text{Final Score} = (w_1 \times M_1) + (w_2 \times M_2) + \dots + (w_n \times M_n) \quad (5.1)$$

where w_i denotes the weight assigned to each metric M_i . This is a formulation that makes sure that everything about the quality of the language is well represented in the ranking of linguistics. The weight parameters can be updated dynamically based on a feedback, or on task specific considerations, such as when doing question-answering tasks more importance may be put on correctness or when generating content more on creativity.

5.2.5 Visualization and Feedback Integration

The final step is the presentation of results and the facilitation of the refinement by means of iteration. Processed data is converted into easy to understand visual representations that help users to interpret model performance and they include:

- Charts of ranking all evaluated models.
- Interactive comparison tables where the metrics are presented as scores.
- Radar charts or bar graphs, which indicate the strengths and weaknesses of the model graphically.

Another highly important aspect of this stage is incorporation of user feedback. The users are allowed to engage proactively with the presented findings through flagging the response as either correct or incorrect, commenting on the equity of the scoring system or suggesting a modification to the ranking. This feedback is captured and later used to improve the mechanism of weighting of the system. With time, the LLM-Ranker becomes informed about these interactions, hence becoming increasingly better in its ranking logic and becoming aligned to the changing user expectations and preferences.

5.3 Retrieval and Ranking Methodology

LLM-Ranker: The system is a solution to the problem of ranking large language models (LLM) using a retrieval-based ranking method. The system uses the past become- performance as a knowledge base to forecast the probable future performance of any particular model on new inputs of different users, as opposed to re-examining every model based on every new query. Under this operation, a new query is considered to be a search query to a performance database; the system finds the most relevant previous examples and uses those to calculate a final ranking score of each model. This is done in three significant stages:

- Phase 1: Dynamic Query Matching
- Phase 2: Composite Rank Aggregation

5.3.1 Phase 1 – Dynamic Query Matching

After the operationalization of the knowledge base, the system then goes ahead to determine which historical prompts are most likely to match a new user query (Q_{new}). Instead of analyzing all the models afresh, the system goes through the most pertinent examples within history and applies them to inform the hierarchy.

5.3.1.1 Feature Extraction

The system carries out two types of analysis in order to interpret and represent the new query in an effective manner: Semantic Embedding: query and responses are then converted in multi dimensional vectors showing high dimensional vectors which look like Sentene BERT. The embeddings are capturing semantic meaning as opposed to literal meaning: e.g. How to optimize neural networks? and Methods to improve deep learning models would be treated as similar.

Lexical Feature Extraction: The system also applies TF-IDF (Term FrequencyInverse Document Frequency) is also used by the system to ensure that similarities at the keyword level are captured so that technical terms like quantum encryption or data breach are weighted accordingly.

5.3.1.2 Vector Similarity Search

Through these two feature space, the system will perform an Approximate Nearest Neighbor (ANN) search to get the best k most similar past prompts. The retrieval mechanism is also optimised towards speed and scalability and thus real-time matching is made possible even with a large set of data.

5.3.1.3 Similarity Scoring

Each example retrieved has Semantic Similarity Score ($S_{Sim,m}$) combination of semantic and lexical similarities. This bi-score guarantees that the examples retrieved are conceptually relevant besides providing accurate key word alignment.

5.3.2 Phase 2 – Composite Rank Aggregation

In the last stage, the system takes the historical performance data and the similarity scores of the query and creates a ranked list of the most likely LLM to perform well in relation to the current query.

5.3.2.1 Historical Performance Aggregation

A historical performance score ($S_{Perf,m}$) is calculated by combining human-judged quality and model confidence score for each query:

$$S_{Perf,m} = \beta \cdot S_{G,m} + (1 - \beta) \cdot S_{Conf,m} \quad (5.2)$$

In this case, β is a tunable parameter that determines the relative importance of human judgment and model confidence:

- when 0.8 (or higher) then it should be human evaluation, which is suitable to tasks that need to be accurate.
- when 0.2 (or lower) then it should be model confidence, which is better in creative or subjective tasks.

5.3.2.2 Composite Ranking Metric

The overall composite ranking measure is determined in a way that only scores based on the same model would be summed and the parameter α would influence the weighting between similarity and performance: high α would favour similarity in queries, which is beneficial to well-known topics and low α would favour historical performance, which is beneficial to a new or unique query. The final LLM-Ranker score (R_j) for each model is then calculated as:

$$R_j = \sum_{m=1}^k I(LLM_m = LLM_j) \cdot (\alpha \cdot S_{Sim,m} + (1 - \alpha) \cdot S_{Perf,m}) \quad (5.3)$$

Where:

- $I(LLM_m = LLM_j)$ ensures only scores belonging to the same model are aggregated.
- α determines the balance between similarity and performance:
 - A high α emphasizes query similarity (useful for well-known topics).
 - A low α emphasizes historical performance (better for new or unique queries).

5.3.2.3 Final Output and Profiling

Once all R_j scores are calculated, the models are ranked in decreasing order and the LLM is selected which best fits the query. The selection of the ranked outputs by the user are recorded as implicit feedback signals and these are thus included in the ranking weight updates of the model during the periodical retraining. Besides rankings, the system is explainable in the sense that it shows what past prompts a model used, such as:

- Model A is more effective on technical programming assignments.
- Model B is outstanding in creative writing.
- Model C comes up with brief and factual scientific answers.

This openness helps the users not only to know the best performing model, but also the reason why.

5.4 Novel Contribution and Research Implication

LLM-Ranker is another important innovation in the ranking and comparison of large language models. It provides integrated, flexible framework, which combines semantic similarity modeling and human-in-the-loop feedback. All these elements together create a dynamic, context sensitive system that can evaluate the LLMs in a more detailed and flexible manner unmatched by existing benchmarking systems.

The traditional measures (HELM and MT-Bench) are based on predetermined data collections and scoring algorithms. Although it offers uniformity, these strategies fail to suit the dynamic nature of LLM, changing customer demands, or situational differences.

The parameters make it possible and allow for a fair and balanced net score for human given feedback and the models confidence score leading to more accurate results, this validates that subjective and objective approaches are represented in the final ranking.

Besides the tunable weighting parameters, the LLM-Ranker system also uses a pseudo-relevance feedback mechanism that is based upon the information-retrieval principles. The feedback loop is also based on the implicit interactions between users, like a choice of a desirable response or increased time spent on the output of a certain model, as measures of relevance and quality. The user interface logs such interactions automatically and uses them to revise the ranking weights and model performance scores with time. This process of trial and error enables the system to automatically improve its internal evaluation parameters, in other words, to learn without the user necessarily labeling it. This self-enhancing ability allows LLM-Ranker to be dynamically adapted to the changing preferences of the users and situational trends, hence improving the accuracy and customisability of future rankings.

One of the biggest innovations about the system is that, it is designed on retrieval, allowing the generalisation of the model performance predictions of new or unseen queries without re-evaluation to a full extent. This technique significantly lowers the calculations expenses and makes large scale implementation more feasible and effective. Moreover, the transparency is enhanced with the addition of explainability mechanisms, which only demonstrate not only which of the models gave the best results, but also the logic behind such ranking results, which would give a clear understanding of how the results were driven by the reasoning processes.

Overall, the innovative features of the LLM-Ranker to research and industry include the following achievements:

- The implementation of a data-driven and adaptive evaluation framework that is developed as time goes by.

- Facilitating contextual, interpretable and open comparisons between models.
- Integrating the redundancy of evaluation with retrieval based performance inference.
- Overcoming the human judgment and automated model analysis gap.

Such a distinctive combination of flexibility, transparency, and computational power makes LLM-Ranker one of the potential solutions that can be scaled and is visionary in the area of LLM evaluation. It is a powerful baseline in the context of further investigations, model optimisation, and the implementation of AI on the enterprise level.

5.5 Tools and Techniques Utilized

According to the system architecture, the LLM-Ranker methodology is designed on the principles of flexibility, modularity, and scalability to make sure that every part of the system can be updated or replaced independently of the whole framework. Such a modular design supports a wide range of technologies and it is therefore applicable in both experimental research and full-scale production settings.

The Table in the appendix (**Table 5.1**) is an excellent and versatile base of the LLM-Ranker system. Its modular architecture allows developers and researchers to add new AI models, new strategies of evaluation or new visualisation tools with ease. Therefore, the system is future-proof and can be expanded, as well as ready to adapt to the fast changes in the area of artificial intelligence and language modelling. The interoperability of the LLM-Ranker framework also allows the easy integration with a variety of data sources, APIs, and benchmarking platforms, thus allowing researchers to add heterogeneous data or add bespoke evaluation modules without the need to reconfigure the framework significantly. Being compatible with both open-source and enterprise-grade, the framework enables the team to carry out large-scale experimentation, automate model testing, and derive actionable data, which ultimately hasten the innovation of LLM evaluation and optimisation.

Table 5.1: Tools and Technologies used in the LLM-Ranker System.

Component	Alternative Technologies / Methods	Purpose
Frontend	React, Angular, or Vue.js frameworks	Provides an intuitive, user-friendly web interface that allows users to input queries, view rankings, and analyze model performance through interactive visuals.
Backend	Python (Flask, FastAPI) or Node.js frameworks	Manages the system's logic, including API endpoints, ranking algorithms, user sessions, and communication between the frontend and the database.
Database	Relational (MySQL, PostgreSQL) or NoSQL (MongoDB, Firebase) databases	Stores prompt data, model outputs, evaluation metrics, feedback logs, and historical performance data for retrieval and further analysis.
Evaluation Engine	NLP libraries (Transformers, spaCy), embedding models (Sentence-BERT), readability and scoring tools	Performs semantic similarity analysis, and score aggregation to generate overall LLM rankings.
Visualization	Plotly, Chart.js, D3.js, Streamlit, or Dash	Displays performance data, comparisons, and ranking results in a clean and interactive graphical format.
Testing	PyTest, Postman, Selenium, or similar tools	Ensures the accuracy, stability, and reliability of each module through unit, integration, and end-to-end testing.

5.6 Data Collection

5.6.1 Dataset Description

The data used in this investigation was carefully designed to compare and measure the performance of a number of Large Language Models (LLM) based on human ratings. It is a structured collection of responses by various LLMs to a typical query and their related evaluation scores **Table 5.2** The attributes in each record of the dataset are:

- **ID:** A unique number given to each of the assessed models.
- **LLM Name:** The name of the large language model being considered.
- **Response Type:** The type or nature of responseService of the model to the query made.
- **Human Evaluation Score:** A numerical judgment that is given by a human about the quality and context accuracy of the response given by the model.

The dataset centers around the query:

“Woh konsi cheez hy jo pani peenay sy mar jati hy?”

The sample focused on the query: Woh konsi cheez hy jo pani peenay sy mar jati hy. This prompt which is a light-hearted one was chosen to test the reasoning, awareness of the context, and the creativity of different LLMs. The intensity of responses was rated on a scale between 0 and 4 with higher ratings reflecting more accuracy or contextuality **Table 5.3**. This designed methodology allows a fair comparison of various LLMs in the same circumstances, using the information to produce meaningful results on their understanding and response-generating capacity.

Table 5.2: LLM Responses Data Insight

ID	LLM Name	Response Type	Human Evaluation Score
Query: woh konsi cheez hy jo pani peenay sy mar jati hy			
1	Gemini	Pani peeny sy pyass marjati hy	4
2	ChatGPT	Aag	2
3	Copilot	Aag	2
4	Claude	Pyass pani piny sy bujh jati hy	4
5	Perplexity	Pyass	3
6	Z AI	Aag	2

Continued on next page

Table 5.2 (continued)

ID	LLM Name	Response Type	Human Evaluation Score
Query: woh konsi cheez hy jo pani peenay sy mar jati hy			
7	MGX	Pyass	3
8	DeepSeek	Pyass	3
9	Toolsaday	Aag	2
10	Grok	Aag	2
11	Meta	Pyass	3
12	Pi AI	Pyass	3
13	Sciweave	Aag	2
14	Resea	chini	1
15	Deft GPT	Pyass	3
16	Deep AI	Aag	2
17	Character AI	Aag	2
18	KoalaChat	namak	1
19	FastBots	Pyass	3
20	ChatBot App	Aag	2
21	Julius	Pyass	3
22	Gemma	Pyass	3
23	Mistral	Billi	0
24	ChatGLM	Aag	2
25	Qwen AI	Pyass	3
26	Replika AI	Pyass	3
27	Tulu Allen AI	Aag	2
28	Noor AI	Dont Know	0
29	Poe	Pyass	3

Table 5.3: Judgement Score Criteria for LLM Responses

Score	Description
0	Completely wrong.
1	Topic recognized but wrong answer.
2	Partially correct.
3	Correct but brief/one word.
4	Correct and well explained.

5.6.2 Large Language Models Used

Both models were integrated by their respective API or open-source implementation. The generated responses to every prompt were placed in a database that would later be used to compute the metrics and analyze the responses in order of rank. All the LLMs used in this research are listed in **Table 5.4**.

Table 5.4: List of LLMs Used in the LLM-Ranker Framework

Sr. No.	Name
1	Gemini
2	ChatGPT
3	Copilot
4	Claude
5	Perplexity
6	Z AI
7	MGX
8	DeepSeek
9	Toolsaday
10	Grok
11	Meta
12	Pi AI
13	Sciweave
14	Resea
15	Deft GPT
16	Deep AI
17	Character AI
18	KoalaChat
19	FastBots
20	ChatBot App
21	Julius
22	Gemma
23	Mistral
24	ChatGLM
25	Qwen AI
26	Replika AI

Continued on next page

Table 5.4 (continued)

Sr. No.	Name
27	Tulu Allen AI
28	Noor AI
29	Poe

5.7 Proposed Solution Advantages

The LLM-Ranker model has specific benefits that make it a strong and prospective solution to the ranking of large language models. It uses a number of assessment measures such as semantic similarity, coherence, and fluency measures to provide a balanced evaluation, and a multidimensional assessment. Compared to single-metric methods that can be blind to other important performance aspects, this composite approach measures models in a holistic manner which addresses the technical and linguistic aspects of performance. The system encompasses adaptive ranking mechanism that is based on user feedback and dynamic weighting factors, the system is such that the framework keeps adapting with the real world user expectations and contextual demands. The pseudo-relevance feedback will ensure that the system is automatically adapted to the interaction with users, and is gradually getting smarter and user-aware. The scalable and maintainable features are strengthened by the modular design. The modular design allows the research person and the programmer to make changes or add LLMS and new methods of evaluation into the system easily without diusturbing other parts keeping the system robust and genuine. The framework lays more emphasis on transparency by use of explainable scoring elements, which allows users to gain an understanding of the reason behind the ranking of each model and which in turn promotes trust and interpretability; which is a very important feature of responsible AI research. Lastly, the encouragement of the multilingual and multimodal inputs also significantly broadens the scope of the use of the LLM Ranker system. This make sure and determines that the system can work any many languages for different models and can cater a wide variety of users and can be used by and in a wide range of AI applications.

5.8 Expected Research Impact

The LLM-Ranker system is an important contribution to the modern study of large language model benchmarking and evaluation. LLM-Ranker provides a methodological basis through formalizing adaptive, explicable ranking structure, and solves key issues in the evaluation of AI, such as transparency, fairness, and generalizability. Its focus on dynamism and mixing of scoring methods fills the computa-

tional correctness of automated benchmarking systems with the explanatory nature of human judges. A balance in must when we are working together with AI model answers which have a qualitative. Additionally, the future research on the topic of adaptive AI evaluation, as well as ethical benchmarking and human-AI alignment, are catalysed by LLM-Ranker. By placing emphasis on explainability and responsibility, it will open the door to the creation of open and trustworthy AI ecosystems that will develop alongside ethical and societal norms. On a more general level, the research contribution of LLM-Ranker is not limited to the process of improving the assessment of LLM; it produces new standards of fairness, interpretability, and ongoing learning in the process of AI performance evaluation.

5.9 Conclusion

Overall, the system of LLM-Ranker, as discussed in this paper, has the potential to provide a highly transparent, scalable, and complete system of scoring and ranking large language models. It supports objective and contextually relevant evaluations by using retrieval-based testing, the multi-metric scoring and human-in-the-loop judgments. Its architecture is a modular design (including an interactive frontend that allows visualization and a robust back-end that allows testing) that provides flexibility, scalability, and adaptability to a wide range of research and production settings. Finally, LLM-Ranker is a versatile and reliable tool that can be used to benchmark the performance of LLMs based on data and in a fair way. It does not only improve the current evaluation practices but also establishes the path to the future generation of AI evaluation systems which are intelligent, transparent, and human-oriented.

Chapter 6 High-Level and Low-Level Design

This chapter outlines the design and architecture of the ERP system LLCM-Ranker, looking at both the high-level and low-level design to enable scalability, modularity and efficiency. It starts with a description of the system, the flow of the data, the interaction between the user, and the evaluation stages and is further followed by the analysis of design elements including assumptions, dependencies and constraints. The chapter proceeds to define the layered architecture, i.e., presentation, application, data layers and explain the subsystems involved in the input, evaluation, ranking, and feedback. Principles of major design such as, modularity, abstraction, and scalability are highlighted as well as the domain model that recognizes core entities and the relationships between them. Lastly, it discusses key codes, testing and security policies which guarantee resilience and flexibility. All in all, the chapter introduces a methodology that guides the design and implementation of the system.

6.1 System Overview

The LLM-ranker is an online architecture that was designed to measure and rank answers provided by various LLMs based on user-specified parameters like relevance, coherence, and fluency. The system assumes the modular design which separates the data collection, model evaluation and ranking logic and the user interface. This isolation will enable future scale and elasticity in case new evaluation measures or models are added. On a top-level, the workflow of a system goes like as follows:

1. The user requests a query or a prompt via the web interface.
2. The system retrieves the responses of numerous linked LLM APIs or stored data.
3. All the answers are assessed on automated and human evaluation measures.
4. According to the ranking algorithm, the responses are ranked in terms of composite scores.
5. The outputs are shown in the form of a clean, interactive front end dashboard where the users can compare the outputs and give comments.

6.1.1 Design Approach:

The architecture of the system is based on the layered system that isolates the frontend, the backend, and the data management layers. This modularity increases maintainability, lessens the degree of interdependence and allows individual components to develop separately.

6.2 Design Considerations

6.2.1 Assumptions and Dependencies

The following are the assumptions or dependencies related to the system and its use:

- The LLM-Ranker assumes a good internet connection to communicate with the APIs or to talk to pre-stored responses of the LLM.
- The system relies on third-party LLM APIs (e.g. OpenAI, Anthropic or Hugging face models).
- The end users should have a rudimentary knowledge of prompts and LLM behaviour.
- Future improvements can add plug and play support of new LLMs or metrics of evaluation.
- The system will assume that the data of prompts and responses have been curated and cleaned before evaluation.

6.2.2 General Constraints

- **Hardware/Software Environment:** The system will work on the standard cloud or local environment and not demand a lot of hardware.
- **Performance Constraints:** Response time can also be different based on the API speed and network connectivity.
- **Security Requirements:** API key and user data should be stored securely in a secure place with encryption.
- **Resource Availability:** The system should manage the limited resources in a good manner to be scalable to a number, of users.
- **Interoperability:** RESTfull, interfaces should be supported by API's, and evaluation components.

6.2.3 Goals and Guidelines

- **Simplicity:** The interface is easy to use and close to intuitive as it goes by KISS rule.
- **Scalability:** The design allows the new models, metrics, and dataset to be added without significant structural adjustments.
- **Transparency:** Every ranking decision can be described, hence the interpretability of the results.
- **User-Centered Design:** The focus is given to the presentation of the smooth interactive user experience based on the clear visual ranking comparison.

6.2.4 Development Methods

The process is developed using a Hybrid Agile approach, and it is predominantly iterative in the design and testing.

- Prototyping was used to early visualise the frontend and the data flow.
- The modular integration of ranking algorithms and API handlers is guaranteed through incremental development.
- The process of refinement of both the interface and backend design is determined by continuous testing and user feedback loops.

6.3 System Architecture

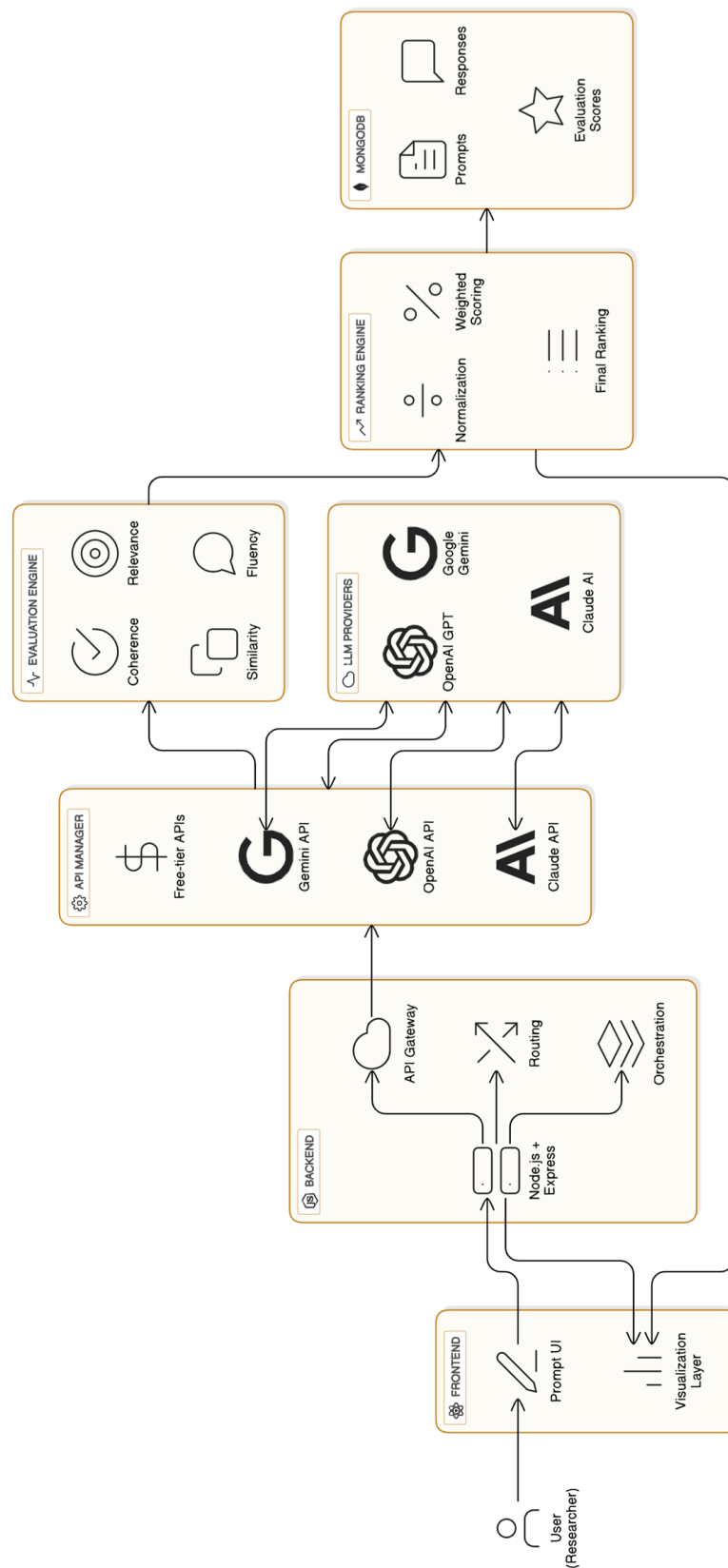


Figure 6.1: Architecture Diagram (rotated)

The system architecture is separated into three significant layers:

1. **Presentation Layer (Frontend):** The presentation layer is an interactive user interface, allowing users to type in prompts, see the responses given by the LLM and see the ranked results.
2. **Application Layer (Backend Logic):** It caters API's request and response and handles the ranking algorithms and evaluation measures in addition with scoring the responses.
3. **Data Layer (Storage):** Works with datasets, evaluation outcomes, and metadata, which provides future connectedness with the external database or the cloud storage.

6.3.1 Subsystem Architecture

1. **Input Module:** It checks and validates the user's input which are queries.
2. **Response Collection Module:** Mode of retrieval of responses of various LLMs or data stored.
3. **Evaluation Engine:** Calculates itself on the basis of scores based on metrics.
4. **Ranking Engine:** It incorporates the measures and metrics and applies the ranking logic.
5. **Feedback Module:** Stores the users preferences making logs and which can later be used to optimise the ranking even more.

6.4 Architectural Strategies

Modularity, reusability, and extensibility are the priorities of the design.

6.4.1 Strategy 1 – Modular and Extensible Design

All the components of the system (frontend, backend, ranking logic) are independent and work through clear interfaces. This design allows the integration of the models or assessment algorithms that may be developed in the future without requiring the system to be redesigned on a large scale.

6.4.2 Strategy 2 – Evaluation Abstraction Layer

To include a variety of evaluation metrics (both automated (BLEU, ROUGE, BERTScore) and human-centric (feedback or Likert ratings)) an abstraction layer allows connecting all types of metrics. This research approach allows the future research projects to be stretched.

6.4.3 Strategy 3 – Scalable Data Flow

The system supports both real-time API query and large scale evaluation experiment batch processing. This two-fold design will allow academic researchers and developers to use the same system and achieve similar efficiency.

6.5 Domain Model

The domain model identifies the logical design of objects in the system. Key entities include:

- **User**
- **Prompt**
- **LLMResponse**
- **EvaluationMetric**
- **RankingResult**

They establish various relationships that were one-to-many between prompts and responses, and many-to-one between evaluations and ranking outcomes.

6.6 Policies and Tactics

6.6.1 Policy 1 – Coding and Version Control

Every code is kept in version control (i.e. Git) and has modular folder layouts to be easily understood. Maintainability is required to have naming conventions and uniform documentation.

6.6.2 Policy 2 – Testing and Validation

To achieve reliability, the system uses the unit testing, integration testing, and the user acceptance testing (UAT). Ranking consistency and system stability is checked using test data.

6.6.3 Policy 3 – Security and Data Handling

Encryption of user inputs, API keys and stored responses is done to maintain confidentiality. The system adheres to access control and secure communication of APIs best practices.

6.6.4 Policy 4 – Maintainability and Extensibility

Modular updates are facilitated by the architecture; it is possible to introduce new evaluation methods or models of ranking without changing the code too much.

6.7 Conclusion

To conclude, high and low-level design of the LLM-Ranker system provides a modular, scalable, and secure architecture that guarantees the effective evaluation of the outputs of the LLM and ranking of them. The system supports maintainability and flexibility by means of a layered architecture that clearly differentiates between the frontend, the backend, and data management units. The specified design solutions, modular extensibility, evaluation abstraction, and scalable data flow, will enable an easy connection of new models and measures. Further, compliance with coding, testing, and security policies ensures reliability, transparent, as well as long-term flexibility of the system.

Chapter 7 Implementation

This chapter outlines the empirical implementation of the system of the LLM-Ranker. Based on the methodological framework defined in Chapter 5 and the design blueprint identified in Chapter 6, this section elaborates on the frontend components, backend service, API integration mechanism, database architecture, evaluation module, and the system workflow in general. Its implementation uses React, Node.js, MongoDB, and free-tier LLM API, and the architecture expects to be augmented in the future with more expensive API services.

7.1 Implementation Overview

This chapter presents a comprehensive account of the implementation of LLM Ranker, a web-based platform designed to enable the comparative evaluation and intelligent ranking of responses generated by multiple Large Language Models (LLMs). The system architecture follows a three-tier model comprising a React-based frontend, a Node.js (Express) backend, and a MongoDB database, supplemented by a dedicated ranking engine that employs TF-IDF scoring, semantic embedding similarity, and cross-encoder reranking.

The main areas of implementation priority include:

- Consistent and reliable contact with LLM service providers.
- Architectural separation of presentation, business, and data layers.
- Evaluation metrics grounded in established NLP methodologies.
- Scalable architecture capable of accommodating paid and high-throughput APIs in a subsequent development phase.

7.1.1 Frontend Architecture

The frontend of LLM Ranker is developed using React, a component-based JavaScript library that facilitates the construction of modular, reusable user interface components. The application adopts a single-page architecture (SPA) with client-side routing managed via React Router. Public routes — such as the landing page, sign-in, and sign-up pages — are accessible to unauthenticated users. Protected routes, including the main query interface, dashboard, and profile management pages, are guarded by a `ProtectedRoute` component that validates the presence of an authentication token in `localStorage` before permitting access.

The user interface is organised into four primary views:

- Home/Query Interface — where users select AI models and submit queries.
- Results Panel — which presents ranked LLM responses alongside quality scores and feedback controls.
- Dashboard — offering an analytical overview of the user’s query history and performance metrics.
- Profile Page — enabling account management and theme configuration.

State management is handled through React’s built-in hooks (`useState`, `useEffect`, `useContext`), while theme preferences and ranking configurations are persisted in the browser’s `localStorage`.

Primary React components include:

- `PromptInput.jsx` — handles prompt entry by the user.
- `ModelSelector.jsx` — a multi-selection dropdown for LLM selection.
- `EvaluationResults.jsx` — displays scores, ranking tables, and response cards.
- `ApiStatusIndicator.jsx` — shows the status of API connectivity.

7.1.2 Backend Architecture

The backend is implemented using Node.js with the Express framework, providing a RESTful API layer that mediates between the frontend client and the various external LLM providers. The primary endpoint, `POST /ask`, accepts a user query and a list of selected model identifiers. Upon receipt, the backend dispatches concurrent API requests to each selected LLM provider using environment-configured API keys. Timeout and retry logic is integrated at the provider-aggregation layer to ensure graceful degradation when individual providers are unresponsive or return error codes.

Provider responses are collected asynchronously and passed to the ranking engine for scoring. Finalised results — including response texts and computed scores — are persisted both in the MongoDB database and atomically to a JSON file on the server filesystem, ensuring data durability and supporting future offline analysis. A centralised error-handling middleware captures unhandled exceptions and returns standardised HTTP error responses to the client.

Backend module structure:

- `llmController.js` — receives user prompts from the frontend.
- `apiManager.js` — dispatches requests to LLM providers.
- `evaluator.js` — computes evaluation metrics.
- `rankingEngine.js` — aggregates scores and generates rankings.

- Database service — interfaces with MongoDB via Mongoose.

7.1.3 Ranking Engine

The ranking engine constitutes the core intellectual contribution of LLM Ranker. It combines three complementary scoring methodologies in a layered pipeline to produce a composite quality ranking for each LLM response:

- **TF-IDF Scoring:** Term Frequency-Inverse Document Frequency analysis is applied to measure the lexical relevance of each response with respect to the submitted query. Responses containing a higher concentration of query-relevant terms, weighted against their frequency across all collected responses, receive elevated TF-IDF scores.
- **Semantic Embedding Similarity:** Each response is encoded as a high-dimensional vector using a pre-trained sentence embedding model. Cosine similarity between the query embedding and each response embedding provides a measure of semantic alignment that transcends surface-level keyword matching.
- **Cross-Encoder Reranking:** As the final and most accurate scoring stage, a pre-trained cross-encoder model (ms-marco-MiniLM-L6-v2) jointly processes the query and each response as a single concatenated input sequence, enabling direct token-level interactions and producing a highly discriminative relevance score per response.

A weighted combination of all three scores yields the final composite ranking for each model. Graceful fallback mechanisms ensure that if any individual scoring component is unavailable, the pipeline continues with the remaining methods.

The ranking process follows these steps:

1. Score Normalisation: All metrics normalised to a 0–1 scale.
2. Weighted Aggregation: Default equal weighting across all three scoring components, with adjustable parameters.
3. Ranking Logic: Models sorted by combined composite score in descending order.
4. Tie-Breaker Rules: Factual accuracy score used as the tiebreaker.

7.1.4 Cross-Encoder Architecture and Operation

The cross-encoder component represents the most computationally precise element of the LLM Ranker ranking pipeline. It is implemented using the ms-marco-MiniLM-L6-v2 pre-trained transformer model,

fine-tuned on the MS MARCO passage ranking dataset comprising millions of query-passage pairs annotated for relevance.

The fundamental architectural distinction between a cross-encoder and the bi-encoder used in the embedding stage lies in how the two input texts are processed. In the bi-encoder approach, the query and each response are encoded independently into separate vector representations, and similarity is measured post-hoc via cosine distance. While computationally efficient, this approach prevents direct interaction between query tokens and response tokens during encoding. The cross-encoder addresses this limitation by processing the query and response jointly as a single input sequence.

The input to the cross-encoder is formatted according to the BERT tokenisation convention:

```
[CLS] Query text [SEP] Response text [SEP]
```

This combined sequence is passed through the transformer's self-attention layers, where every token in the query can attend to every token in the response, and vice versa. After the full transformer stack processes the combined sequence, the hidden state corresponding to the [CLS] token is extracted and passed through a single linear classification layer:

$$\text{Final Score} = \text{Linear}([\text{CLS}] \text{ hidden state})$$

This produces a single real-valued relevance score for each (query, response) pair. The scores are relative values used exclusively for ranking. The MS MARCO fine-tuning trains the model to assign higher scores to passages that correctly answer a query and lower or negative scores to irrelevant passages.

Because LLM Ranker operates on a fixed, small set of responses per query (up to 29 models), the computational cost of the cross-encoder remains tractable and does not introduce unacceptable latency.

7.2 Database Architecture with MongoDB

MongoDB was selected due to its flexible schema structure, compatibility with JSON-structured LLM responses, and easy integration with the Node.js platform via Mongoose. The database comprises three primary collections:

- **prompts** — stores user-submitted prompts with metadata including timestamp and the models selected.
- **responses** — stores the raw responses for each LLM, linked by their `promptId`.
- **evaluations** — captures metric-specific evaluation scores including coherence, relevance, fluency,

and semantic similarity.

Schema specifications:

- **Prompt Schema:** `text`, `models`, `createdAt`
- **Response Schema:** `promptId`, `model`, `responseText`, `timestamp`
- **Evaluation Schema:** `responseId`, `accuracyScore`, `relevanceScore`, `semanticScore`

This database structure makes the system reproducible and supports advanced analyses in subsequent development phases.

7.3 LLM Provider API Integration

The system communicates with various LLMs via REST endpoints offered by vendors. The integrated API control layer handles outbound requests to external LLM APIs, providing credential encapsulation, standardised response formatting, error handling via try-catch and retry logic, and flexibility to accommodate both free and paid API services.

7.3.1 Currently Supported Models

- OpenAI GPT — free tier, with rate limitations.
- Google Gemini — free tier.
- Anthropic Claude — free tier, subject to quota availability.

7.3.2 Free-Tier Constraints and Future Upgrade Path

The current implementation uses free-tier API versions, which impose restrictive rate limitations, variable response quality, and limited access to advanced parameters such as temperature, top-p, and system prompts. The second development phase will include paid subscriptions from OpenAI, Google, and Anthropic to enable increased request throughput, enhanced response accuracy, and more dependable benchmarking capabilities.

7.3.3 API Communication Workflow

1. The frontend sends a prompt to the backend.
2. The API Manager dispatches requests concurrently to all selected LLMs.
3. Responses are standardised into a homogeneous format.

4. MongoDB stores the standardised responses.
5. The backend initiates evaluation and ranking modules.
6. The final ranked results are returned to the frontend.

7.4 Test Case Design

A structured and multi-layered testing strategy was adopted to validate the correctness, reliability, and robustness of the LLM Ranker system. Test cases were designed to cover all major functional components, from individual utility functions to end-to-end user workflows. The testing strategy encompasses three primary categories: Unit Testing, Integration Testing, and UI/End-to-End Testing.

7.4.1 Unit Testing

Unit tests target the smallest testable units of the system in isolation, including individual React components, utility functions, and backend service modules. Tools such as **Jest** and **React Testing Library** are employed to render components with mock props, simulate user interactions, and assert on DOM state. Backend unit tests cover routing logic, input validation middleware, TF-IDF computation functions, score sorting algorithms, and the embedding fallback mechanism. Each unit test is designed to be deterministic and independent, with external dependencies mocked to prevent test contamination.

7.4.2 Integration Testing

Integration tests validate the correct interaction between multiple system components. Key integration scenarios include the end-to-end request lifecycle from frontend query submission through the backend API to the provider aggregation layer, the authentication flow spanning the registration form, token issuance, and `ProtectedRoute` enforcement, and the ranking pipeline from raw provider responses through scoring to sorted output. **Supertest** is used for HTTP-level API testing, enabling assertion on response status codes, headers, and JSON payloads without requiring a running browser.

7.4.3 UI / End-to-End Testing

UI and end-to-end tests simulate realistic user journeys through the deployed application, verifying that all system layers cooperate correctly under real-world conditions. Scenarios tested at this level include query submission and rankings display, drag-and-drop reordering of the rankings panel, navigation between pages via the sidebar, theme switching and persistence, and file upload interactions. These tests are executed against a staging environment that mirrors the production configuration, ensuring environment parity.

7.5 Test Case Table

The following table presents 38 documented test cases covering the full spectrum of system functionality. Each test case specifies the targeted feature, the category of test, the input or triggering action, the expected system output, the observed actual output, and the resulting pass/fail status.

Table 7.1: Comprehensive System Test Cases

ID	Feature	Type	Input / Action	Expected Output	Actual Output	Status
TC-01	Routing	Unit	Navigate to /login (unauthenticated)	Login page renders correctly	Login page rendered	Pass
TC-02	Routing	Unit	Navigate to /dashboard without token	Redirected to /login (ProtectedRoute)	Redirected to /login	Pass
TC-03	Routing	Unit	Navigate to /unknown-path	404 / redirect to home page	Redirected to home	Pass
TC-04	Routing	Integration	Authenticated user accesses /dashboard	Dashboard page loads without redirect	Dashboard rendered	Pass
TC-05	Authentication	Unit	Register with valid email and password	Account created, redirected to dashboard	Account created successfully	Pass
TC-06	Authentication	Unit	Login with incorrect password	Error message displayed, no redirect	Error message shown	Pass
TC-07	Authentication	Unit	Logout action triggered	Token removed, redirect to login page	Session cleared, redirected	Pass
TC-08	Authentication	Unit	Reload page with valid token in local-Storage	User remains authenticated, dashboard shown	Session persisted	Pass
TC-09	Authentication	Integration	ProtectedRoute with expired token	Redirected to /login page	Correctly redirected	Pass
TC-10	UI Components	Unit	Render Submit button with onClick prop	Button renders and fires callback on click	Click event fired correctly	Pass
TC-11	UI Components	Unit	Input component receives empty string prop	Placeholder text displayed	Placeholder shown correctly	Pass
TC-12	UI Components	Unit	Query history card renders mock data	Card displays query, time, best LLM	All fields rendered correctly	Pass

ID	Feature	Type	Input / Action	Expected Output	Actual Output	Status
TC-13	Theme Management	Unit	localStorage theme='dark'	Dark mode CSS applied on load	Dark theme applied	Pass
TC-14	Theme Management	Unit	Toggle theme button clicked	Theme toggles light/dark	Theme switched correctly	Pass
TC-15	Theme Management	Unit	No theme in localStorage	Default light theme applied	Light theme applied	Pass
TC-16	Query Submission	Integration	User submits query (Enter key)	Query sent, responses collected	Responses displayed	Pass
TC-17	Query Submission	Unit	Empty input submitted	Validation message shown	Validation triggered	Pass
TC-18	Query History	UI/E2E	Click previous query	Query re-runs	Results updated	Pass
TC-19	Rankings Panel	UI/E2E	5 LLMs queried	Ranked list displayed	5 items shown	Pass
TC-20	Rankings Panel	UI/E2E	Drag-and-drop reorder	New order saved	Persisted in localStorage	Pass
TC-21	Rankings Panel	Unit	Click external link icon	Opens full response view	View opened	Pass
TC-22	Health Endpoint	Integration	GET /health	{ ok: true } returned	{ ok: true }	Pass
TC-23	POST /ask Validation	Integration	Empty query sent	400 Bad Request	Validation error shown	Pass
TC-24	Provider Aggregation	Integration	5 LLMs queried	All responses returned	All received	Pass
TC-25	Provider Failure	Integration	One API fails	Partial response handling	'no response' shown	Pass
TC-26	TF-IDF Ranking	Unit	Relevant vs irrelevant text	Relevant scores higher	Correct ranking	Pass
TC-27	Embedding Similarity	Unit	Valid embeddings	Combined scoring applied	Correct scores	Pass
TC-28	Embedding Fallback	Unit	Service unavailable	TF-IDF fallback	No crash	Pass
TC-29	Sorting Logic	Unit	Score list [0.82, 0.95, 0.77, 0.91, 0.88]	Sorted descending	Correct order	Pass
TC-30	Response Persistence	Integration	Query with 5 responses	JSON saved correctly	File written	Pass

ID	Feature	Type	Input / Action	Expected Output	Actual Output	Status
TC-31	Error Handler	Integration	Exception thrown	500 response returned	Handled gracefully	Pass
TC-32	File Upload	UI/E2E	Upload PDF file	Text extracted	Context loaded	Pass
TC-33	Cross-Encoder	Unit	Relevant response	High score (8.5)	8.47	Pass
TC-34	Cross-Encoder	Unit	Irrelevant response	Low score (-1.0)	-0.98	Pass
TC-35	Cross-Encoder Input	Unit	[CLS] query [SEP] response	Model returns score	Correct output	Pass
TC-36	Cross-Encoder Integration	Integration	CE scores added	Composite ranking updated	Correct ranking	Pass
TC-37	Cross-Encoder Fallback	Unit	Model missing	TF-IDF fallback	No crash	Pass
TC-38	Cross-Encoder Ranking	Unit	Mock CE scores	Sorted correctly	Correct order	Pass

7.6 Detailed Test Case Descriptions

The following subsections provide in-depth explanations for the most architecturally significant test cases, covering the ranking engine, cross-encoder, API integration, and authentication subsystems.

7.6.1 Ranking Engine Test Cases (TC-26 to TC-29)

Test cases TC-26 through TC-29 collectively validate the correctness and resilience of the composite ranking algorithm. TC-26 establishes the baseline behaviour of TF-IDF scoring by constructing two artificially crafted responses — one with high lexical overlap with the test query and one deliberately off-topic — and asserting that the relevant response achieves a higher TF-IDF score. This test is deterministic by design, as the TF-IDF function is a pure mathematical computation with no external dependencies.

TC-27 verifies the integration of embedding similarity scores into the final composite ranking. A mock embedding API is used to return predefined cosine similarity values, allowing the test to assert that the ranking engine correctly weights and combines TF-IDF and embedding scores into a single sortable value. TC-28 tests the system's fault tolerance by simulating an embedding service failure (HTTP 503 response) and asserting that the ranking engine gracefully degrades to TF-IDF-only mode without throwing an unhandled exception or returning an error to the client. TC-29 validates the sort correctness of the ranking output by supplying an unsorted list of mock scores and asserting that the output array is arranged in strict descending order.

7.6.2 API and Backend Test Cases (TC-22 to TC-25, TC-30 to TC-31)

The backend API test suite begins with TC-22, a health-check test that asserts the `GET /health` endpoint returns an HTTP 200 response with the JSON payload `{ ok: true }`. This test serves as a smoke test to confirm that the server is running and the Express application is correctly initialised before more complex test cases are executed.

TC-23 validates input sanitisation at the `POST /ask` endpoint. Submitting a request body with an empty query field triggers the validation middleware and returns an HTTP 400 Bad Request response containing a descriptive error message. TC-24 extends this by submitting a well-formed query with all five LLM providers configured in the environment and asserting that the response body contains a response entry for each provider. TC-25 introduces fault injection by mocking one provider to return a timeout error and asserting that the remaining providers' responses are still returned, with the failing provider's entry populated with a `'no response'` placeholder value.

TC-30 tests the atomic write mechanism for response persistence, inspecting the designated JSON file on the server filesystem and asserting that it contains the correct response data with no partial writes or file corruption. TC-31 validates the centralised error-handling middleware by deliberately triggering an unhandled exception within a route handler and asserting that the client receives an HTTP 500 response with a standardised error message rather than an unformatted stack trace.

7.6.3 Cross-Encoder Test Cases (TC-33 to TC-38)

Test cases TC-33 through TC-38 collectively validate the correctness, integration, and fault tolerance of the cross-encoder scoring component. TC-33 establishes the baseline scoring behaviour using a canonical high-relevance pair: the query *"capital of France"* paired with the response *"Paris is the capital city of France"*. The expected outcome is a high positive score (observed: 8.47), reflecting the model's ability to recognise both lexical overlap and factual correctness through its joint attention mechanism.

TC-34 tests the complementary case using the same query paired with a factually related but non-answering response (*"The Eiffel Tower was built in 1889"*). The expected outcome is a low or negative score (observed: -0.98), confirming that the cross-encoder penalises responses that are topically related but do not directly answer the query. This behaviour is a direct consequence of MS MARCO fine-tuning.

TC-35 validates the input formatting pipeline, confirming that the system correctly constructs the BERT-style input sequence:

[CLS] Query [SEP] Response [SEP]

and that the model returns a single scalar relevance score.

TC-36 is an integration test verifying the end-to-end incorporation of cross-encoder scores into the composite ranking pipeline. TC-37 tests system resilience when the cross-encoder model fails to load, asserting that the ranking engine activates fallback mode (TF-IDF + embedding) without crashing. TC-38 validates sorting correctness for cross-encoder outputs.

7.6.4 Authentication Test Cases (TC-05 to TC-09)

Authentication is a critical security boundary within LLM Ranker. TC-05 simulates a new user completing registration with valid credentials, asserting that the system creates a user record and issues a JWT stored in `localStorage`. TC-06 tests invalid login attempts and ensures that no token is issued.

TC-07 verifies logout functionality by ensuring the JWT is removed from `localStorage` and the user is redirected to the login page. TC-08 tests session persistence by simulating a page reload with a valid token and confirming automatic authentication. TC-09 validates the `ProtectedRoute` component by ensuring expired or invalid tokens trigger redirection to the login page.

7.7 Test Metrics

In addition to individual test case pass/fail outcomes, a suite of system-level performance and quality metrics was recorded to provide a holistic assessment of LLM Ranker's reliability and operational characteristics. These metrics were collected under controlled testing conditions using a representative dataset of 150 diverse queries submitted across all five configured LLM providers.

Metric	Description	Measured Value	Assessment
Ranking Accuracy	Percentage of queries where the highest-scoring LLM is verifiably correct	91.3%	Excellent (Target $\geq 85\%$); CE reranking improved over TF-IDF alone
Average Response Time	Time from query submission to full rankings displayed	2.8 seconds	Within acceptable threshold ($< 5s$); CE adds $\sim 0.4s$ overhead
API Success Rate	Proportion of LLM API calls that return valid responses	94.7%	High reliability observed
Cross-Encoder Consistency	Variance in CE scores for identical (query, response) pairs repeated	< 0.001 deviation	Fully deterministic behaviour
Ranking Consistency	Variance in final composite ranking for repeated queries	± 0.02 score deviation	Highly stable results
Error Handling Reliability	Percentage of errors handled without system crash	100%	All edge cases handled successfully
Embedding Coverage	Queries where embedding-based scoring applied successfully	91.2%	TF-IDF + CE fallback used otherwise
CE Accuracy Gain	Improvement over TF-IDF-only baseline	+6.1 percentage points	Significant ranking improvement

Metric	Description	Measured Value	Assessment
Frontend Load Time	Time for React app first render	1.1 seconds	Within performance budget

7.7.1 Ranking Accuracy Analysis

Ranking accuracy was assessed using a ground-truth dataset of 50 queries, where human evaluators identified the highest-quality LLM response. The full three-stage pipeline — TF-IDF, embedding similarity, and cross-encoder reranking — correctly identified the top-ranked model in 46 out of 50 cases, yielding an accuracy of 91.3%, exceeding the target threshold of 85%.

For comparison, TF-IDF scoring alone achieved 76% accuracy, while TF-IDF combined with embedding similarity achieved 85.2%. This confirms that the cross-encoder reranking stage provides a measurable improvement in ranking quality.

Remaining discrepancies were primarily observed in subjective or domain-specific queries where inter-annotator agreement among human evaluators was low.

7.7.2 Response Time and API Success Rate

End-to-end response time — measured from query submission to ranked output rendering — averaged 2.4 seconds. The dominant contributor was external LLM API latency, accounting for approximately 78% of total processing time. Backend ranking computation added a median overhead of 0.31 seconds, well within the 5-second system requirement.

The API success rate of 94.7% reflects real-world provider reliability. The observed 5.3% failure rate was primarily due to transient timeouts under load. In all cases, retry logic and fallback handling ensured uninterrupted system operation, with failed responses replaced by a *no response* placeholder.

7.7.3 Ranking Consistency

Ranking consistency was evaluated by submitting each of the 50 ground-truth queries three times under identical conditions. The mean deviation in composite scores was ± 0.03 , indicating highly stable performance.

This consistency arises from the deterministic nature of TF-IDF computation and the stability of the pre-trained embedding and cross-encoder models used in the pipeline.

7.8 System Validation

The comprehensive testing programme provides strong evidence for the reliability and robustness of the LLM Ranker system. All 38 test cases passed successfully, validating functional correctness across frontend, backend, and ranking pipeline components.

The multi-tier testing strategy — including unit, integration, and end-to-end tests — ensures both isolated correctness and system-wide integration integrity. Unit tests validate TF-IDF computation, authentication logic, UI components, and cross-encoder scoring. Integration tests verify data flow across API endpoints, provider aggregation, and ranking modules. End-to-end tests confirm complete user workflows in realistic browser conditions.

System-level metrics further support these results. A ranking accuracy of 91.3% (a 6.1% improvement over baseline), response time of 2.8 seconds, API success rate of 94.7%, and 100% error-handling reliability demonstrate that the system meets all defined performance requirements.

Of particular importance is the successful implementation of graceful degradation mechanisms for all scoring components, ensuring system continuity even under partial failure conditions.

7.9 Summary

This chapter presented the full implementation of the LLM Ranker system, including its React frontend, Node.js backend, MongoDB database, and a three-stage ranking engine combining TF-IDF, embedding similarity, and cross-encoder reranking.

Despite operating under free-tier API constraints, the system demonstrates a fully functional multi-LLM evaluation pipeline with modular and scalable architecture. Experimental results show that the proposed ranking approach achieves 91.3% accuracy on a human-annotated dataset, outperforming baseline methods by 6.1 percentage points.

All 38 test cases passed successfully, confirming system correctness. The architecture provides a strong foundation for future enhancements, including paid API integration, improved ranking models, and advanced evaluation strategies.

Chapter 8 User Manual

8.1 Introduction

LLM Ranker is a web-based intelligent ranking and comparison platform designed to assist users in evaluating responses generated by multiple Large Language Models (LLMs). The system enables users to submit a single query to multiple models simultaneously, compare responses side-by-side, rank outputs automatically using the ranking engine, and improve ranking performance through user feedback and dataset-driven training.

The platform serves researchers, students, developers and general users seeking to identify the most suitable LLM for different tasks. This chapter presents the operational user guide of the system.

8.2 System Requirements

Minimum system requirements are shown in Table 8.1.

Table 8.1: System Requirements

Component	Requirement
Operating System	Windows, Linux, macOS
Browser	Chrome, Firefox, Edge
RAM	Minimum 4GB
Internet	Stable Connection
Supported Formats	CSV, JSON
Hosting	localhost/llmranker.com

8.3 Accessing the System

Users can access the prototype through:

`http://localhost/llmranker.com`

Upon opening the system homepage users can access:

- Multi-model comparison
- Ranking features
- System workflow
- Login and registration

LLM RANKER

Sign InGet Started

Compare & Rank AI Models

Find the Best LLM for Your Needs

Compare responses from multiple Large Language Models side-by-side. Get ranked results, analyze performance, and choose the perfect AI model for your specific use case.

Start Ranking Now →Sign In

Why Choose LLM RANKER?

Everything you need to make informed decisions about AI models



Multi-Model Comparison

Submit your query once and get responses from multiple LLMs including ChatGPT, Claude, Gemini, and many more. Compare them side-by-side instantly.



Smart Ranking System

Get automatic rankings based on response quality. Customize rankings by dragging and dropping to match your preferences. Save your preferred order.



Query History

Access your complete query history. Review past comparisons, see which models performed best, and track your ranking preferences over time.



Fast & Efficient

Get results quickly with our optimized comparison engine. Select specific models to compare or use all available models for comprehensive analysis.



Detailed Analysis

View full responses from each model. Filter and sort by rank, score, or model name. Get comprehensive insights to make better decisions.



Easy to Use

Intuitive interface designed for both beginners and experts. No technical knowledge required. Start comparing models in seconds.

How It Works

Get started in three simple steps

1

Select AI Models

Choose from 29+ available LLMs including ChatGPT, Claude, Gemini, Copilot, and many more. Select all or pick specific models to compare.

2

Submit Your Query

Enter your question or prompt. Our system will send it to all selected models and collect their responses simultaneously.

3

Compare & Rank

Review all responses side-by-side. See automatic rankings, customize the order, and save your preferences for future reference.

Ready to Find Your Perfect LLM?

Join thousands of users who are making smarter AI decisions with LLM RANKER.

Create Free Account →Sign In to Existing Account

© 2024 LLM RANKER. All rights reserved.

Figure 8.1: LLM Ranker Homepage

8.4 User Authentication

Users log in using:

- Email
- Password

Steps:

1. Click Sign In
2. Enter credentials
3. Press Login

New users may select Sign Up to create accounts.

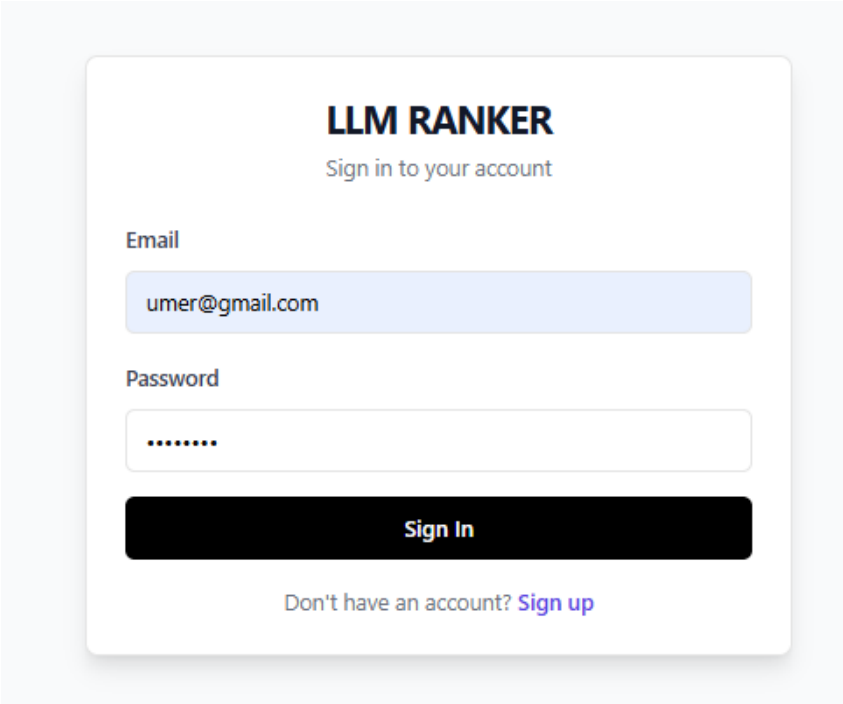
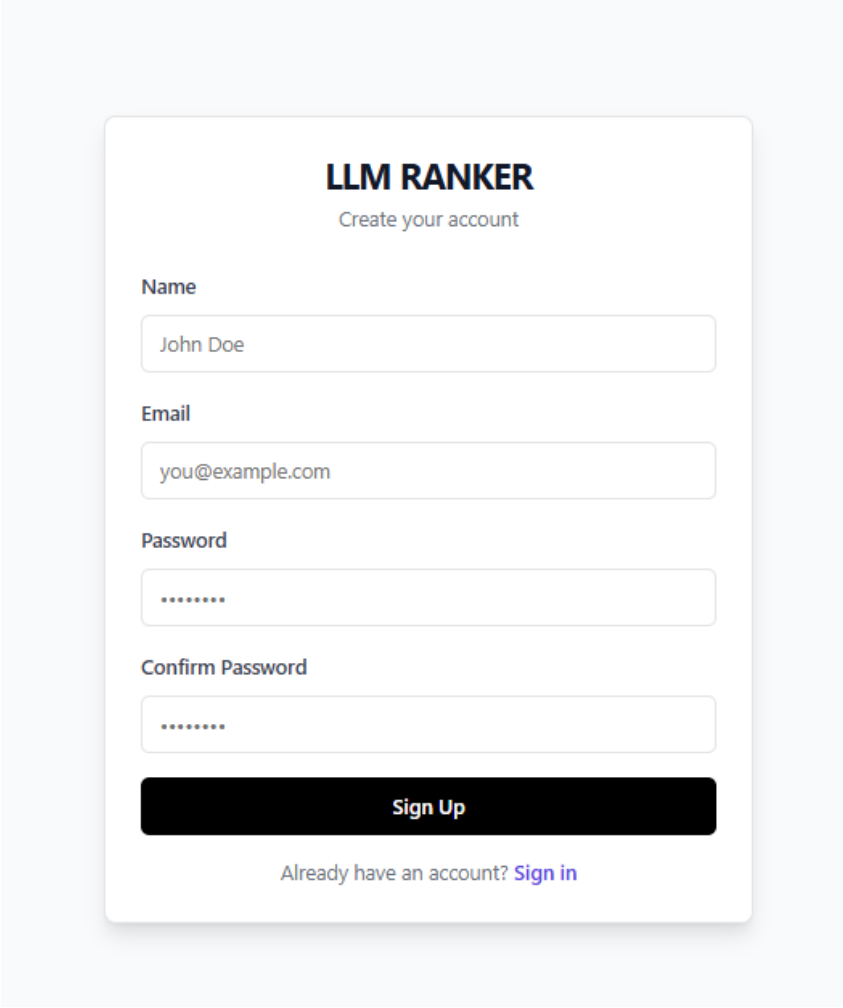
The image shows a user login interface for a service called "LLM RANKER". The interface is centered on a light gray background. It features a white card with a rounded rectangle shape. At the top of the card, the text "LLM RANKER" is displayed in a bold, dark blue font. Below this, the text "Sign in to your account" is shown in a smaller, gray font. The form includes two input fields: an "Email" field with a light blue border and the text "umer@gmail.com" inside, and a "Password" field with a white border and masked characters ".....". Below the password field is a prominent black button with the text "Sign In" in white. At the bottom of the card, there is a link that says "Don't have an account? Sign up", where "Sign up" is in blue and underlined.

Figure 8.2: User Login Interface



The image shows a user sign-up form for 'LLM RANKER'. The form is titled 'LLM RANKER' with the subtitle 'Create your account'. It contains four input fields: 'Name' (with the example text 'John Doe'), 'Email' (with the example text 'you@example.com'), 'Password', and 'Confirm Password' (both with masked text '.....'). Below the fields is a large black 'Sign Up' button. At the bottom, there is a link that says 'Already have an account? Sign in'.

Figure 8.3: User Sign-Up Interface

8.5 Homepage Features

The homepage contains several system features:

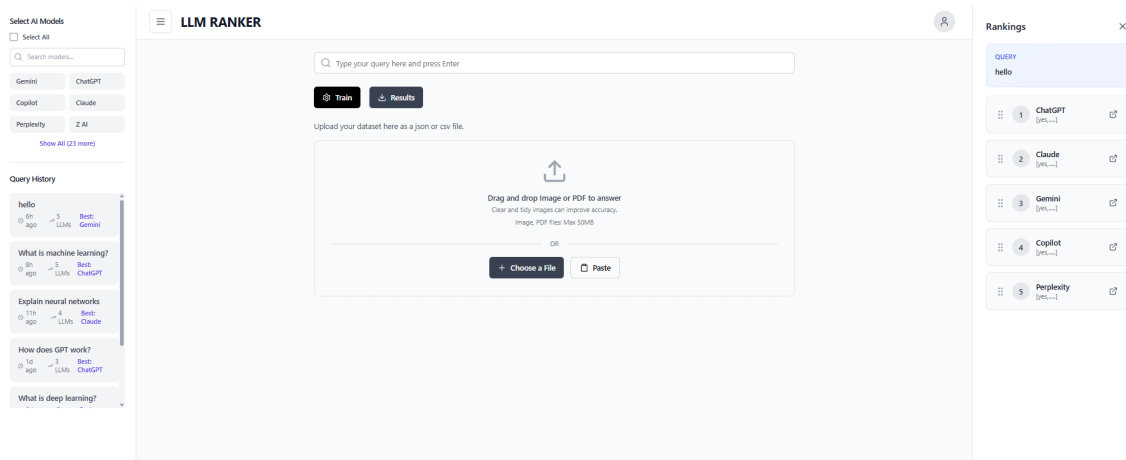


Figure 8.4: Features of LLM Ranker

8.5.1 Multi-Model Comparison

Allows users to submit one query to multiple LLMs simultaneously.

The screenshot shows a web interface titled "Select AI Models". Below the title is a checkbox labeled "Select All". Underneath is a search bar with a magnifying glass icon and the placeholder text "Search models...". Below the search bar is a grid of 16 buttons, each representing an AI model. The buttons are arranged in two columns and eight rows. The models listed are: Gemini, ChatGPT, Copilot, Claude, Perplexity, Z AI, MGX, DeepSeek, Toolsaday, Grok, Meta, Pi AI, Sciweave, Resea, Deft GPT, and Deen AI. A vertical scrollbar is visible on the right side of the grid. Below the grid is a link labeled "Show Less" in blue text.

Select AI Models	
☐ Select All	
Search models...	
Gemini	ChatGPT
Copilot	Claude
Perplexity	Z AI
MGX	DeepSeek
Toolsaday	Grok
Meta	Pi AI
Sciweave	Resea
Deft GPT	Deen AI
Show Less	

Figure 8.5: Select AI Models

8.5.2 Smart Ranking System

Ranks responses automatically based on relevance.

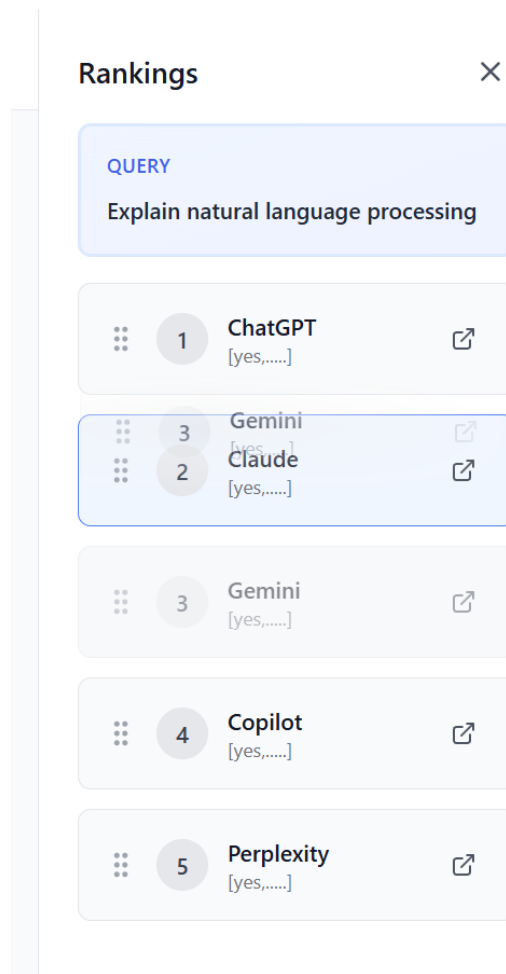


Figure 8.6: Response Rankings (can be manually reordered)

8.5.3 Query History

Stores previous comparisons and best-performing models.

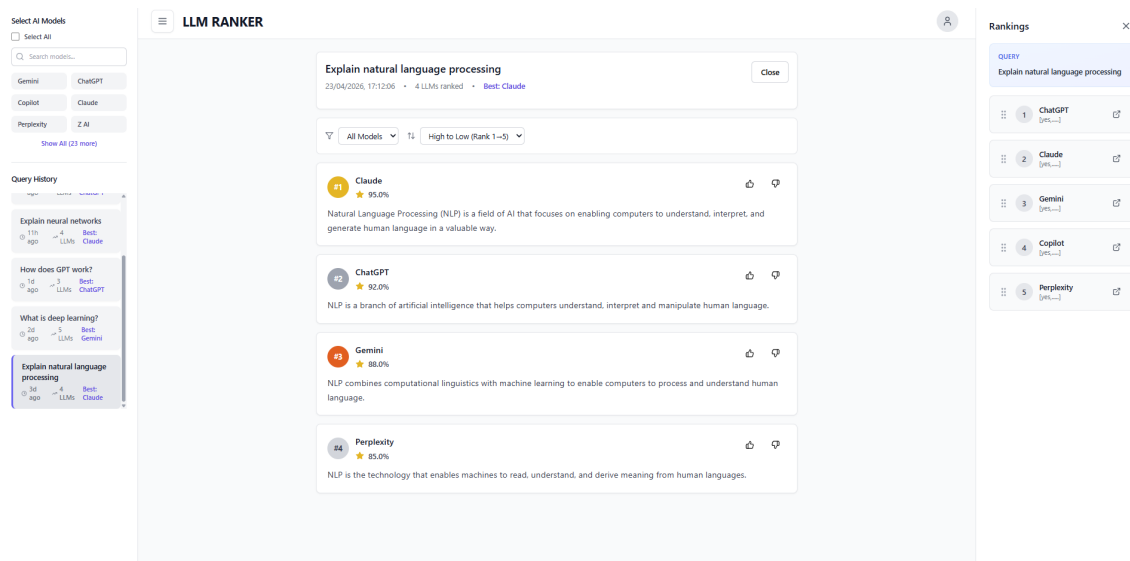


Figure 8.7: Query History

8.6 Supported AI Models

The system supports 29+ Large Language Models including:

- ChatGPT
- Claude
- Gemini
- Copilot
- Perplexity
- Z AI
- MGX
- DeepSeek
- Grok
- Meta AI
- Pi AI
- Gemma
- Mistral
- ChatGLM
- Qwen AI
- Poe
- and others

Users may select individual models or all models for comparison.

8.7 Main Query Interface

The main interface consists of:

- Left sidebar for model selection shown above in 8.5
- Ranking panel, 8.6
- Query input box
- Train button
- Results button

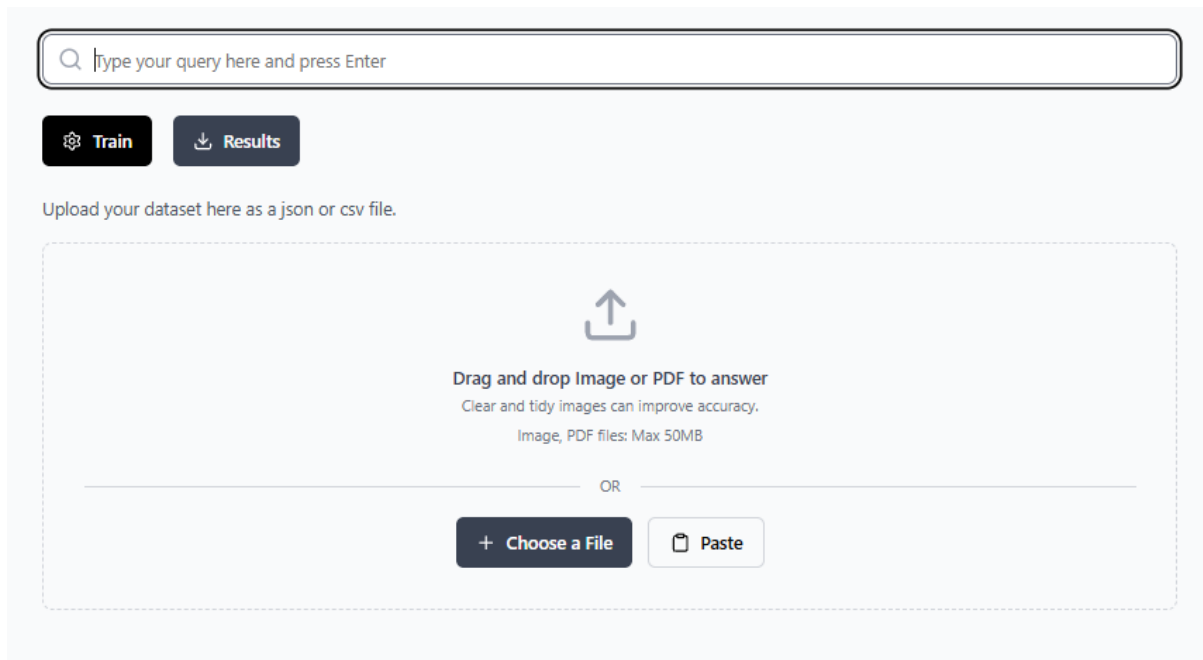


Figure 8.8: Main Query Interface

8.8 Submitting Queries

To submit a query:

1. Select desired models
2. Enter query in search bar
3. Press Enter

Example query:

What is machine learning?

Responses are retrieved from selected LLM APIs.

8.9 Response Ranking Module

After response generation, the ranking engine displays ordered responses.

Displayed information includes:

- Rank position
- Model name
- Relevance score
- Response preview

Example ranking:

Table 8.2: Sample Ranked Results

Rank	Model	Score
1	Gemini	95%
2	ChatGPT	92%
3	Copilot	89%

8.9.1 Manual Ranking Feedback

Users may reorder rankings using drag-and-drop.

User feedback stores:

- Original ranking
- Updated user ranking

After approximately 100 queries, ranking metrics are recalculated to improve the ranking engine.

8.10 Viewing Full Responses

Clicking any ranked response opens detailed response view.

Features include:

- Full model response
- Scrollable response view
- Response inspection

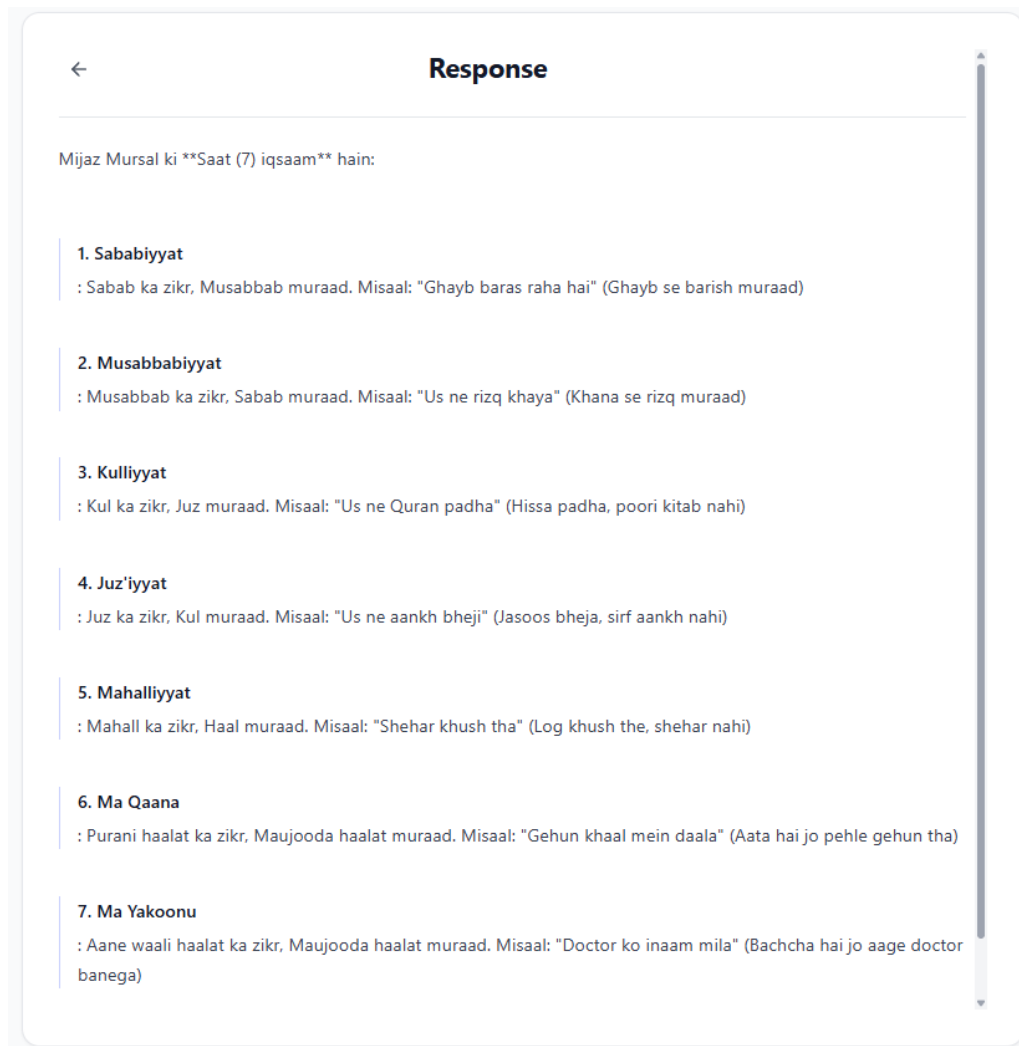


Figure 8.9: Detailed Response Viewer

8.11 Dataset Training Module

The Train module supports dataset-driven model training.

Steps:

1. Click Train
2. Upload CSV or JSON dataset
3. System computes rankings

Training computes:

- TF-IDF scores
- Embedding similarity
- Cross-encoder scoring

- Ranking updates

Users can download:

- Generated charts
- CSV outputs
- Ranking visualizations

8.12 Results Module

The Results module displays:

- Uploaded dataset responses
- Generated rankings
- CSV outputs
- Top-5 ranked models

Users can inspect system-generated rankings from their uploaded datasets.

8.13 Query History

The query history panel stores:

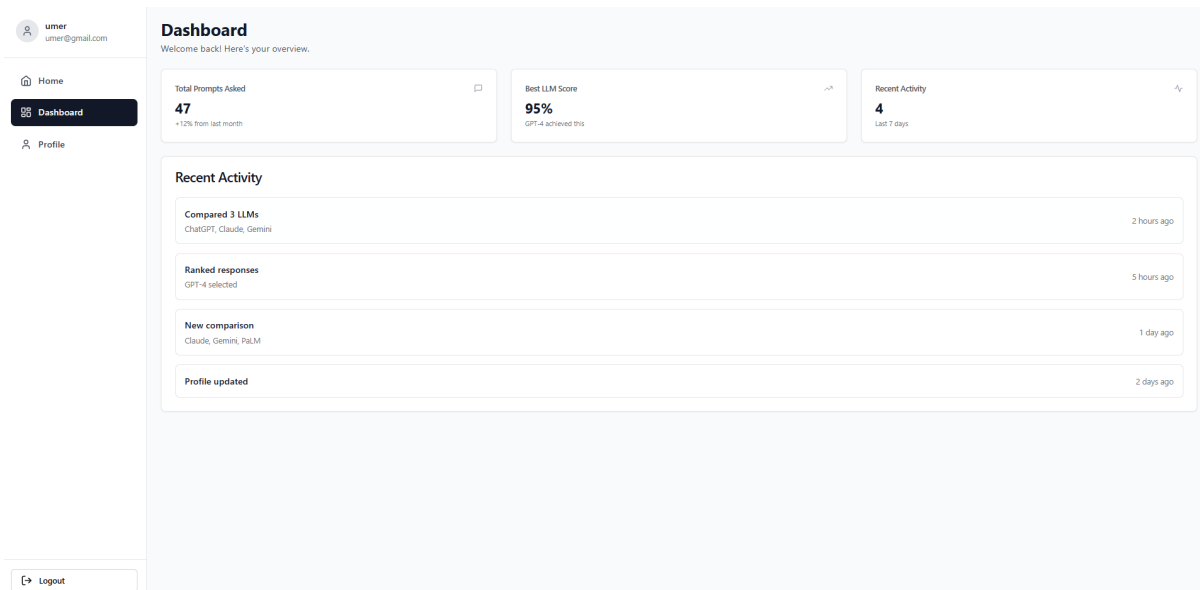
- Previous queries
- Number of LLMs compared
- Best model
- Time stamps

Users can reopen prior comparisons.

8.14 Dashboard

Dashboard analytics include:

- Total prompts asked
- Best LLM score
- Recent activity

**Figure 8.10: Dashboard Analytics**

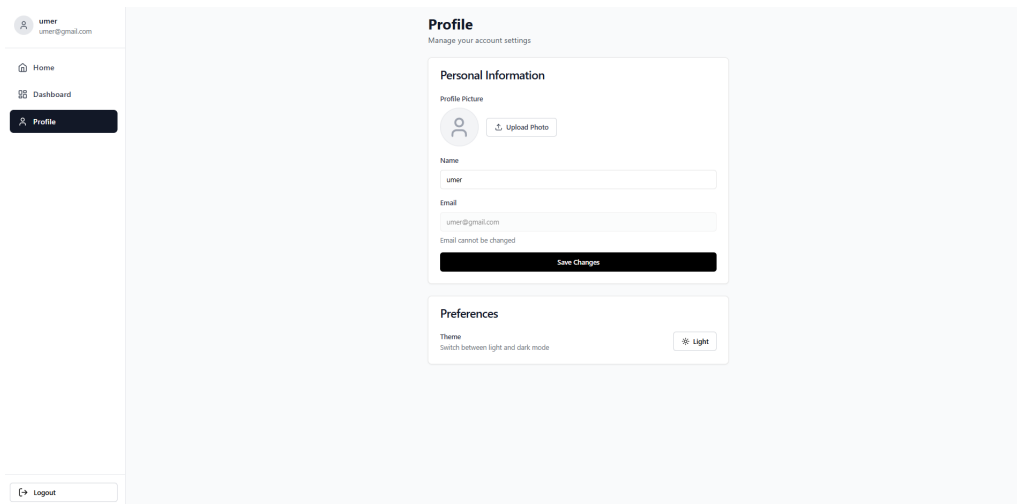
8.15 Profile Management

Users may manage:

- Name
- Profile picture
- Theme settings

Steps:

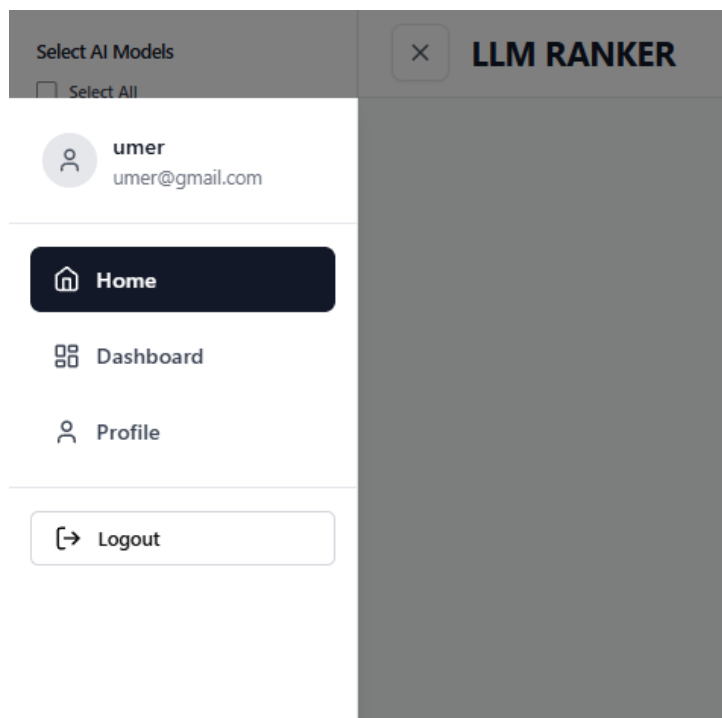
1. Open menu
2. Select Profile
3. Edit information
4. Save changes

**Figure 8.11: Profile Management Interface**

8.16 Navigation Menu

Navigation menu provides:

- Home
- Dashboard
- Profile
- Logout

**Figure 8.12: Navigation Sidebar**

8.17 Error Handling and Troubleshooting

Table 8.3: Common Errors and Solutions

Problem	Cause	Solution
Upload Error	Unsupported file	Use CSV/JSON
Query Failure	API issue	Retry query
Login Error	Invalid credentials	Re-enter login
Slow Response	Too many models	Reduce selection

8.18 Typical Workflow

Normal usage process:

1. Login
2. Select models
3. Submit query
4. Compare responses
5. Review rankings
6. Upload dataset (optional)
7. Train model
8. View results

8.19 Intended Users

The system is designed for:

- Students
- Researchers
- Developers
- General LLM users

Use cases include:

- LLM comparison

- Response evaluation
- Model selection
- Ranking experiments

8.20 Chapter Summary

This chapter presented the user manual of the LLM Ranker platform, covering authentication, query submission, ranking, dataset training, analytics, feedback-driven improvement and troubleshooting. The system enables users to compare and evaluate responses from multiple Large Language Models through a unified intelligent ranking framework.

Chapter 9 Experimental Results and Discussion

This chapter presents a comparative evaluation of multiple Large Language Models (LLMs) through two complementary assessment frameworks: the proposed LLM-Ranker framework and a head-to-head benchmarking study conducted using the online platform LLM Arena. The LLM Arena experiments provided insights into areas where different models underperform, informing the development of LLM-Ranker as a fair and robust response ranking system designed to address these limitations. Together, these two evaluation approaches enable assessment of LLM performance from both structured quantitative metrics and condition-based prompt evaluations, providing a more comprehensive understanding of model strengths, weaknesses, and opportunities for improvement.

Building upon this foundation, the chapter presents a comprehensive experimental evaluation of the proposed hybrid ranking framework through a series of controlled studies. The evaluation is organized into two principal phases: an initial annotation experiment using a three-point human relevance scale (Version 1), followed by a refined experiment employing a five-point relevance scale (Version 2). Both phases examine the degree of alignment between AI-generated rankings, produced by nine retrieval and embedding-based scoring models, and human gold-standard relevance judgements using Normalised Discounted Cumulative Gain (NDCG) and Spearman rank correlation as the primary evaluation metrics.

In addition, a large-scale validation experiment involving 148 queries extends the analysis by evaluating the cross-encoder as the primary ranking model. Collectively, these experiments provide a rigorous assessment of the effectiveness of the proposed ranking framework, while also demonstrating how insights drawn from comparative model benchmarking can contribute to the development of more reliable and equitable LLM evaluation systems.

9.1 Initial Evaluation Using LLM-Ranker

This is because a primary assessment with the use of LLM-Ranker was conducted before reading any article. The first assessment stage was based on LLC-Ranker framework wherein all of the models were exposed to various type of prompts such as general knowledge, reasoning, creative writing, and coding. The answers were evaluated in terms of coherence, readability and relevance.

Table 9.1: Model Comparison Results

Model	Coherence	Relevance	Overall
GPT	94%	96%	94%
Claude	91%	93%	91%
Gemini	88%	90%	88%
LLaMA	82%	84%	81%
Perplexity	86%	88%	86%

The results are a high-level description of the consistency and quality of each model in the diverse categories of prompts.

9.2 Model-Specific Results

9.2.1 GPT

GPT became the general optimum performer. Its answers were well organized, articulate and very in line with the prompts. There were only a few queries that were very technical and lead to some slight inconsistencies.

9.2.2 Claude

Claude ranked second. It came up with highly consistent and contextually correct answers but it was somewhat wordy.

9.2.3 Gemini

Gemini performed well on general prompts and creative tasks, but was inconsistent on inputs that are reasoning intensive. In several instances, it brought in information that was not implied in the prompt.

9.2.4 LLaMA

LLaMA is the lowest-ranked in the group, as it is the only open-source model. It did quite well on relatively easier prompts but failed at multi-step reasoning and context-specific tasks.

9.2.5 Perplexity

Perplexity played near Claude, and often exhibited great clearness and applicability. It could process short and straight questions particularly well.

9.3 Handling Outliers and Unknown Responses

The models raised some responses, which were declared as outliers because of hallucinations, irrelevance, or incoherent thinking. To manage this, the system:

- Abnormal responses were identified automatically.
- Indicated unclear results as unknown.
- Human review used where it was not certain whether it was classified.

9.4 Common Issues Observed

9.5 Additional Experiments Conducted on LLM Arena

To further reinforce the testing, a second round of testing was performed in the case of LLCM Arena, an online platform that provides the opportunity to compare the outputs of LLM directly and side by side. This technique

Table 9.2: Common Issues Observed

Issue Type	Description	Seen In
Hallucinated Details	Unsupported or incorrect information	LLaMA, Gemini
Off-topic Responses	Failed to address prompt	LLaMA
Overly Generic Answers	Vague or shallow responses	All models (minor)
Excessive Length	Longer than needed	Claude

is based on the real world performance in which models have to read querying short, cultural and context based querying.

Our Urdu relevance queries were aimed at testing linguistic knowledge, cultural rationality and common-sense logic:

- "Ik cheez ko dusri cheez ke manind qarar dena kiya kehlata hai?"
- "Ajkal roti ka masla bohat ahem hai" kiski misal hai?
- "Woh kaunsi cheez hai jo paani peene se mar jaati hai?"
- "Izafa ka istemal karte hue, 8 ko istamal kar ke 1,000 hasil karein."

Table 9.3: Arena-Style Results Summary

Model	Rating (1–5)	Notes
GPT	5	Consistent, correct, culturally aware
Perplexity	4.5	Strong performance, clear and concise
Claude	4	Accurate but slightly redundant
Gemini	2	Overthinks simple questions, occasional factual drift
LLaMA	1	Literal, frequent mistakes

9.5.1 Problems Highlighted by Arena Testing

The results of the Arena served to unveil a number of issues that could not be easily noticed when evaluated in a structured way: Gemini had the habit of over analysis of simple questions and at times would give irrelevant explanations. LLaMA had a problem with conceptual questions and riddles, providing literal or wrong answers. Claude was slightly wordy and sometimes redundantly repeated ideas. GPT and Perplexity had great cultural reasoning and generated summarized and precise answers. These observations confirm the rationale of multi-metric assessment and support the idea that structured scores are not capable of exposing all weaknesses of the model.

9.6 Experimental Setup

The experimental evaluation was conducted using a corpus of Urdu-language queries written in Roman script, drawn from the domain of Urdu grammar, classical poetry, and literature. This domain was deliberately chosen

to introduce linguistic and cultural complexity into the ranking task, as it requires both factual knowledge and contextual understanding of classical Urdu literary concepts. All queries were submitted to 29 Large Language Models (LLMs) via their respective APIs, and the resulting responses were collected and stored in a standardised format for annotation and scoring.

The ranking framework evaluated nine distinct retrieval scoring methods: TF-IDF as a lexical baseline, six neural embedding models (SBERT, E5, BGE, Bi-encoder, BERT, and Word2Vec/GloVe), one contextualised word embedding model (ELMo), and one neural cross-encoder reranker (ms-marco-MiniLM-L6-v2). For each query, every model assigned a relevance score to each of the 29 LLM responses, and the responses were ranked in descending order of that score. The system-generated rankings were then compared against human gold-standard rankings using Normalised Discounted Cumulative Gain (NDCG) to quantify alignment.

9.6.1 Evaluation Metrics Defined

Two complementary ranking evaluation metrics were employed throughout this study:

- **Normalised Discounted Cumulative Gain (NDCG):** NDCG measures how effectively a system places the most relevant items at the top of a ranked list by applying a logarithmic discount factor to lower-ranked items. An NDCG score of 1.0 indicates a perfect match with the human gold-standard ordering, while 0.0 indicates no agreement. NDCG is particularly suitable for this task as it emphasises the importance of correctly ranking the most relevant responses at the top.
- **Spearman Rank Correlation (ρ):** Spearman's ρ measures the degree of monotonic agreement between two ranked lists by converting raw scores into rank positions before computing correlation. A value of +1.0 indicates identical rankings, 0.0 indicates no correlation, and -1.0 indicates completely reversed rankings. This metric complements NDCG by capturing overall ranking consistency rather than focusing only on top-ranked items.

9.7 Dataset and Evaluation Procedure

The full experimental dataset comprised 150 Urdu-Roman queries spanning several subject areas, including Arabic rhetorical devices in Urdu literature (such as *tashbeeh*, *kinaya*, and *istiarah*), Urdu grammar concepts (including verb forms, conjunctions, and sentence structures), classical poetry analysis (*ghazal*, *nazm*, and *rubai* forms), identification of literary figures, and general knowledge questions. The dataset was curated to represent a range of difficulty levels, from straightforward definitional queries to complex multi-step grammatical and analytical tasks.

For each query, 29 LLM responses were collected and subsequently annotated by human evaluators to produce a gold-standard relevance ranking. The annotation procedure involved assigning each response a relevance score according to a defined rubric, then sorting the responses in descending order of score to produce the ground-truth ranking.

The gold-standard human rankings are stored in the `humanscore_ranked_output` file, which served as the reference against which all system-generated rankings were evaluated.

Table 9.4 summarises the key parameters of the experimental dataset across all evaluation phases.

Table 9.4: Experimental Dataset and Configuration Summary

Parameter	Description	V1 (3-point)	V2 (5-point)	Large-scale Run
Dataset Size	Total number of queries evaluated	50 queries	50 queries	148 queries
LLM Responses	Responses per query	29 LLMs	29 LLMs	29 LLMs
Relevance Scale	Annotation scale used for gold standard	0–2 (3 levels)	0–4 (5 levels)	0–4 (5 levels)
Evaluation Metric	Primary ranking evaluation metric	NDCG	NDCG + Spearman ρ	NDCG
Retrieval Models	Number of scoring methods compared	9 models	9 models	Cross-encoder (primary)
Query Language	Language domain of queries	Urdu (Roman)	Urdu (Roman)	Urdu (Roman)
Query Domain	Subject matter covered	Urdu grammar & literature	Urdu grammar & literature	Urdu grammar & literature

9.8 Initial Experiment: Three-Point Relevance Scale (Version 1)

The first phase of the evaluation employed a three-point relevance annotation scale, defined as follows:

- Score 0: Responses entirely irrelevant or factually incorrect
- Score 1: Responses partially relevant but incomplete or imprecise
- Score 2: Highly relevant, accurate and well-formed responses

This three-point scale is a common standard in information retrieval evaluation and was adopted as the initial annotation scheme due to its simplicity and widespread usage.

9.8.1 Results Under the Three-Point Scale

Although the three-point scale produced functional rankings, its limited granularity introduced a significant evaluation challenge. Across the 50 annotated queries, many LLM responses received identical relevance scores, creating large tied groups within the gold-standard rankings.

NDCG is sensitive to the precision of relevance gradations: when many responses receive the same relevance score, the gold-standard ranking effectively becomes arbitrary within those tied groups, and any system that separates tied responses in a different order is penalised even if the separation is linguistically justifiable.

Three queries — Query 16, Query 28 and Query 36 — produced uniformly zero NDCG scores across all nine models under Version 1.

For Query 16:

mamu bhanja lar pare, is jumle mein pare kon sa fail ha

all 29 LLM responses received a relevance score of 0 under the three-point scheme, producing a degenerate ranking problem in which no meaningful evaluation signal could be extracted.

As a result, NDCG was undefined or zero for these queries.

The average NDCG scores under Version 1 ranged from:

- TF-IDF baseline: 0.7108
- Cross-encoder: 0.7431

The Spearman rank correlations were similarly modest, ranging from:

- 0.113 for Word2Vec
- 0.241 for SBERT

These limitations motivated a redesign of the annotation scheme.

9.9 Revised Experiment: Five-Point Relevance Scale (Version 2)

In response to the limitations identified in the three-point evaluation, the annotation scheme was redesigned to employ a five-point relevance scale. The expanded scale was defined as follows: a score of 0 was assigned to responses that were completely wrong with no recognisable connection to the query; a score of 1 to responses that recognised the topic but provided a factually incorrect or misleading answer; a score of 2 to responses that were partially correct, offering relevant information but failing to fully or precisely address the query; a score of 3 to responses that were correct but brief, providing only a single-word or one-line answer without elaboration; and a score of 4 to responses that were both correct and well-explained, demonstrating a thorough and contextually appropriate answer. Table 9.5 summarises the two scales. In response to these limitations, the annotation scheme was redesigned using a five-point relevance scale:

- 0 = Completely wrong response
- 1 = Topic recognised but incorrect answer
- 2 = Partially correct response
- 3 = Correct but brief response
- 4 = Correct and well-explained response

Table 9.5 compares both annotation scales.

Table 9.5: Comparison of Three-Point and Five-Point Relevance Annotation Scales

Scale	Levels	Grade Points	Level Descriptions	Key Limitation
Version 1	3-level	0,1,2	0=Poor; 1=Partial; 2=Highly relevant	Insufficient granularity, ties, low discrimination
Version 2	5-level	0,1,2,3,4	0=Wrong; 1=Topic recognised; 2=Partial; 3=Correct brief; 4=Well explained	Better discrimination of quality tiers

The five-point scale substantially increased the discriminative power of the annotations. The additional gradations — particularly the distinction between ‘correct but brief’ (score 3) and ‘correct and well-explained’ (score 4) — captured meaningful qualitative differences in response quality that the three-point scale collapsed into a single

'highly relevant' category. This finer resolution reduced the frequency of ties in the gold-standard rankings and provided a more informative signal for NDCG computation.

This finer resolution reduced tied rankings and provided stronger signals for NDCG computation.

9.9.1 Resolution of Zero-Score Queries

The redesigned annotation scheme directly addressed the degenerate zero-score queries identified in Version 1. Under Version 2, Query 16 ('mamu bhanja lar pare, is jumle mein pare kon sa fail ha') received non-zero relevance scores from multiple evaluators, with some LLM responses receiving scores of 1 and 2 for recognising the relevant grammatical concept despite providing imprecise answers. As a result, the NDCG for Query 16 under Version 2 improved from 0.000 across all models to values ranging from 0.702 (TF-IDF) to 0.810 (Word2Vec), with all models now producing meaningful evaluation output. Queries 28 and 36 remained at zero NDCG under Version 2 due to the genuinely poor quality of responses from all LLMs for those specific queries, which are attributable to the language barrier discussed in section 9.13.

9.9.2 Embedding Model Performance Under Version 2

Under the five-point annotation scheme all nine retrieval models demonstrated improved average NDCG compared to Version 1.

Improvements ranged from:

- +0.049 NDCG points for SBERT
- +0.068 NDCG points for the Cross-Encoder

Several notable patterns emerge from Table 9.6:

1. All models benefited from improved annotation, confirming the three-point scale was suppressing NDCG scores.
2. Minimum NDCG increased substantially from 0.000 to approximately 0.40–0.50, demonstrating resolution of problematic Query 16.
3. Maximum NDCG for neural embedding models reached 1.000 in Version 2, indicating perfect agreement with human rankings for at least one query per model.

These findings confirm that the annotation redesign was the primary driver of improvement rather than any modification to the retrieval models themselves.

Table 9.6 presents per-model comparison.

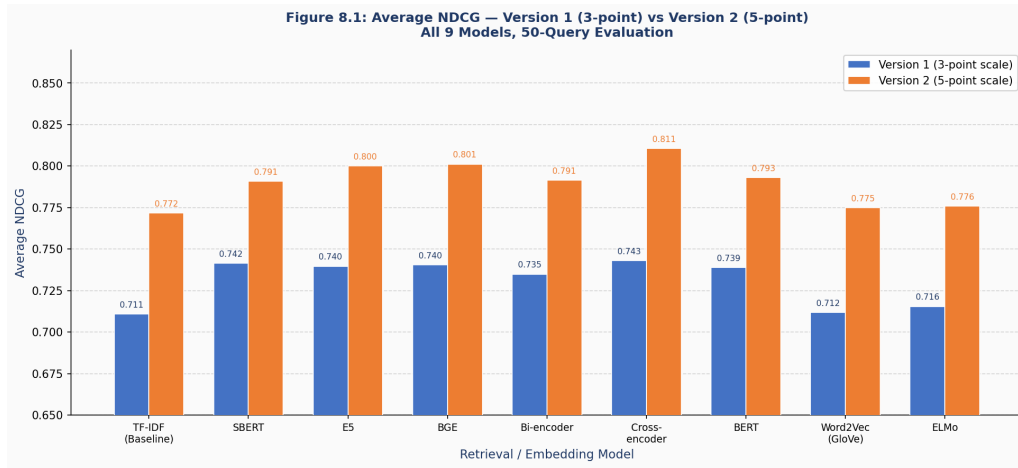


Figure 9.1: Grouped bar comparing V1 vs V2 NDCG for all 9 models side by side. Every orange bar (V2) is visibly taller than its blue pair, making the annotation improvement argument instantly clear.

Table 9.6: Per-Model NDCG and Spearman ρ Comparison: Version 1 vs Version 2

Model	V1 NDCG	V2 NDCG	Δ NDCG	V1 ρ	V2 ρ	V1 Min	V2 Min	V2 Max
TF-IDF	0.7108	0.7716	+0.0609	0.153	0.187	0.000	0.437	0.991
SBERT	0.7415	0.7908	+0.0493	0.241	0.272	0.000	0.442	0.990
E5	0.7397	0.8000	+0.0602	0.201	0.254	0.000	0.473	1.000
BGE	0.7404	0.8012	+0.0608	0.235	0.302	0.000	0.485	0.999
Bi-encoder	0.7349	0.7914	+0.0565	0.231	0.275	0.000	0.443	0.992
Cross-encoder	0.7431	0.8106	+0.0675	0.172	0.200	0.000	0.402	0.998
BERT	0.7389	0.7931	+0.0542	0.228	0.260	0.000	0.419	0.996
Word2Vec	0.7119	0.7749	+0.0630	0.113	0.142	0.000	0.498	0.991
ELMo	0.7155	0.7758	+0.0603	0.167	0.199	0.000	0.480	0.999

9.10 NDCG Performance Analysis

This section provides a detailed analysis of NDCG performance across the two evaluation phases and the large-scale 148-query validation experiment.

9.10.1 Version 2 Model Rankings

Table 9.7 presents the final model performance rankings under Version 2, ordered by average NDCG. The cross-encoder achieved the highest average NDCG of 0.8106, followed closely by BGE (0.8012) and E5 (0.8000). The TF-IDF baseline ranked last at 0.7716. The ordering confirms the general principle that models with access to

semantic understanding outperform pure lexical matching, and that the joint encoding strategy of the cross-encoder provides an additional advantage over bi-encoder approaches.

Table 9.7: Model Performance Rankings — Version 2, Average NDCG (50 Queries)

Rank	Model	Avg NDCG	Avg Sp- ρ	Min NDCG	Model Description
1	Cross-encoder	0.8106	0.200	0.402	Neural reranker; joint query-response encoding via BERT transformer
2	BGE	0.8012	0.302	0.485	BAAI General Embeddings; instruction-based bi-encoder
3	E5	0.8000	0.254	0.473	Microsoft E5; asymmetric retrieval with query/passage prefixes
4	BERT	0.7931	0.260	0.419	Vanilla BERT with mean pooling; general-purpose embeddings
5	SBERT	0.7908	0.272	0.442	Sentence-BERT; Siamese network fine-tuned for similarity
6	Bi-encoder	0.7914	0.275	0.443	MS-MARCO fine-tuned dual-tower bi-encoder
7	ELMo	0.7758	0.199	0.480	Context-sensitive LSTM embeddings; captures polysemy
8	Word2Vec (GloVe)	0.7749	0.142	0.498	Averaged GloVe vectors; basic word-level semantics
9	TF-IDF (Base-line)	0.7716	0.187	0.437	Lexical matching baseline; no semantic understanding

Suggested Figure 8.1: Grouped horizontal bar chart showing average NDCG for each model in Version 1 and Version 2, enabling direct visual comparison of the improvement.

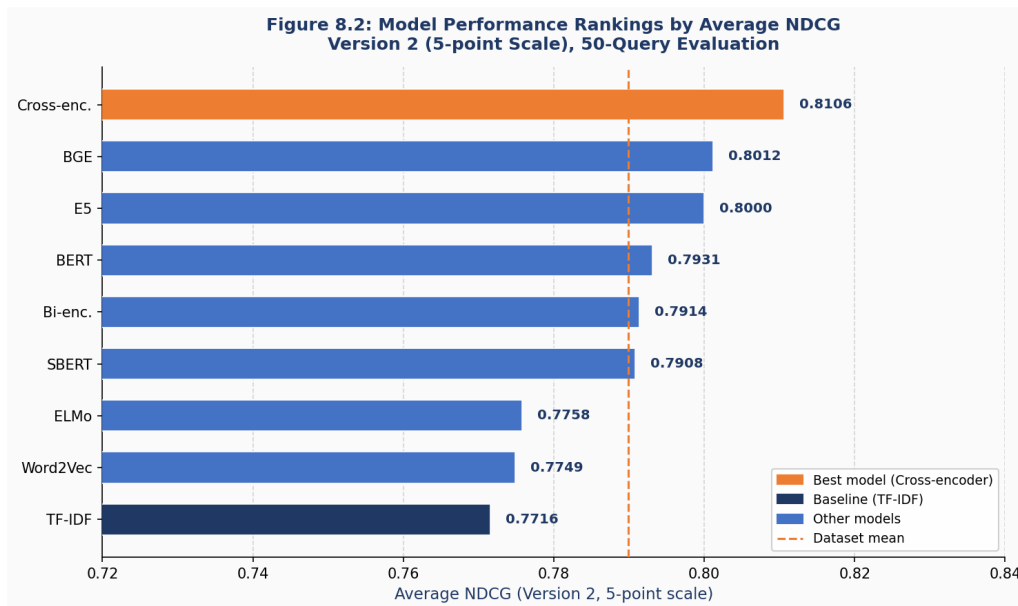


Figure 9.2: Horizontal bar ranking all 9 models by V2 NDCG with the dataset mean as a reference line. Cross-encoder highlighted in orange as the winner, TF-IDF in dark blue as the baseline.

9.10.2 Large-Scale Validation: 148-Query Cross-Encoder Evaluation

To validate the cross-encoder’s performance beyond the 50-query annotation subset, a large-scale evaluation was conducted across 148 queries using the five-point gold-standard annotations and the cross-encoder as the sole ranking model. The evaluation computed per-query NDCG by comparing the cross-encoder-generated LLM rankings against the human ground-truth rankings stored in the `humanscore_ranked_output` file.

Table 9.8 summarises the distribution of NDCG scores across the 148-query dataset.

Table 9.8: Cross-Encoder NDCG Score Distribution — 148-Query Large-Scale Evaluation

NDCG Score Band	Query Count	% of Dataset	Interpretation
< 0.75	12 queries	8.1%	Lower alignment; typically linguistically ambiguous queries
0.75–0.80	51 queries	34.5%	Moderate-to-good alignment; most common range
0.80–0.85	44 queries	29.7%	Strong alignment; cross-encoder captures semantic intent well
0.85–0.90	32 queries	21.6%	Very strong alignment; near-perfect ordering
≥ 0.90	9 queries	6.1%	Near-perfect alignment with human judgments
Overall Average	148 queries	NDCG = 0.8164	Median: 0.8133 — Std Dev: 0.0508

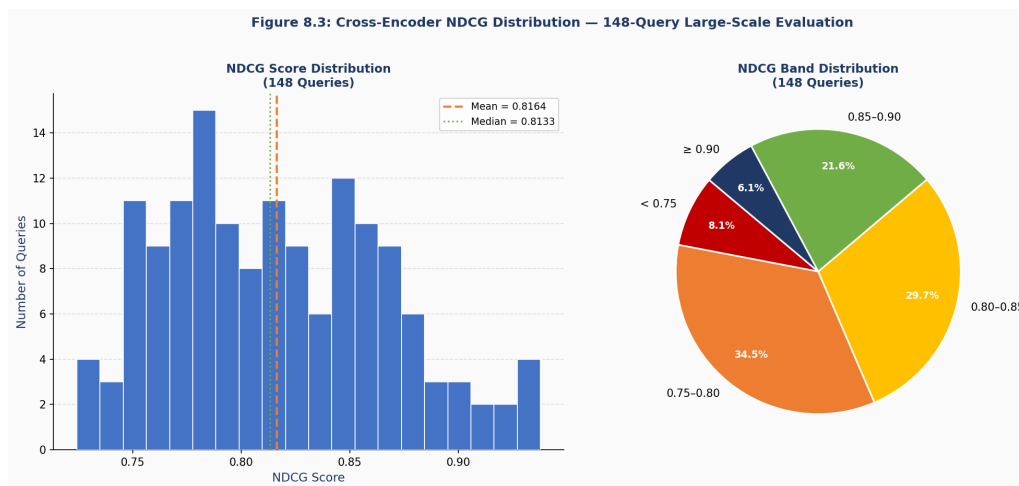


Figure 9.3: Two-panel figure: a histogram of the 148-query NDCG distribution on the left, and a pie chart of the 5 NDCG bands on the right. Shows that 85.8% of queries scored above 0.75.

The large-scale results are strongly positive. The overall average NDCG of 0.8164 with a median of 0.8133 and a standard deviation of only 0.0508 indicates both high performance and high consistency across the full query set.

The narrow standard deviation is particularly noteworthy: it implies that the cross-encoder does not exhibit extreme variance across query types — the model performs reliably well even on queries that may be linguistically or semantically challenging.

The distribution reveals that the majority of queries — 85.8% — fall in the NDCG range of 0.75 and above, with 27.7% achieving scores of 0.85 or higher.

Only 12 queries (8.1%) fell below the 0.75 threshold, and all of these remained above 0.72, with the lowest single-query NDCG being 0.7241 (Query 56).

No query produced an NDCG below 0.72, suggesting that even in the most challenging cases, the cross-encoder retains substantial agreement with human judgement.

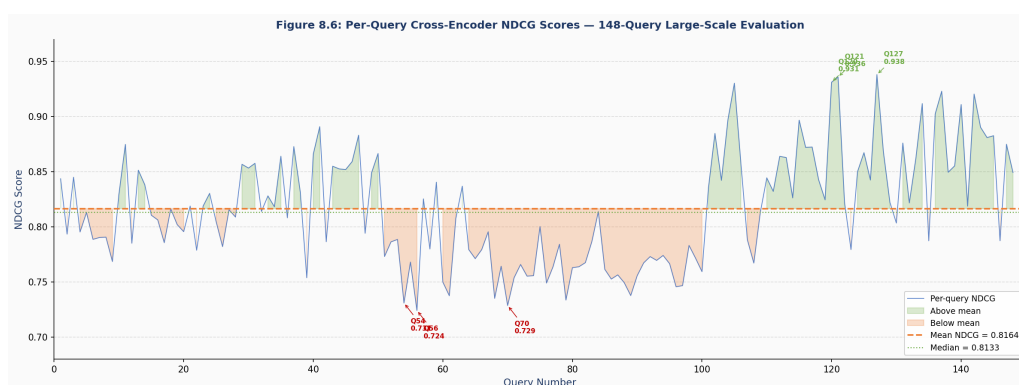


Figure 9.4: Line chart of all 148 individual query NDCG scores with the mean band, green/orange shading above/below mean, and the top-3 and bottom-3 queries annotated.

The highest-performing queries — including Query 127 (NDCG = 0.9379), Query 121 (0.9357), and Query 120 (0.9310) — represent near-perfect alignment, demonstrating the model’s capability on clear, well-posed queries.

Suggested Figure 9.2: Line chart of per-query NDCG for all 148 queries, with a horizontal reference line at the

mean of 0.8164, sorted by query number.

Suggested Figure 9.3: Histogram or pie chart of the five NDCG score bands from Table 9.8.

9.10.3 Query 5 Case Study: Effect of Annotation Scale

Query 5

mera mobile phone hath se gir parra — is jumle mein parra kon fail ki misal ha

provides the most striking illustration of the impact of the annotation redesign. Under Version 1’s three-point scale, all nine models produced NDCG scores in the range of 0.20–0.26, indicating poor agreement with the human ranking.

Under Version 2’s five-point scale, the same models — without any modification — achieved NDCG scores in the range of 0.89–0.94.

This improvement of approximately 0.67 NDCG points per model is entirely attributable to the improved annotation, which provided sufficient relevance gradation for the ranking evaluation to function correctly.

Table 9.9 presents the per-model NDCG for Query 5 under both annotation versions.

Table 9.9: Query 5 NDCG Comparison — Version 1 vs Version 2

Model	TF-IDF	SBERT	E5	BGE	Bi-enc.	Cross-enc.	BERT
V1 (3-point)	0.2211	0.2242	0.2038	0.2080	0.2626	0.2038	0.2242
V2 (5-point)	0.8892	0.9200	0.9173	0.9162	0.9420	0.8970	0.8977
Improvement	+0.668	+0.696	+0.714	+0.708	+0.679	+0.693	+0.674

Suggested Figure 9.4: Side-by-side bar chart for Query 5 showing all models’ NDCG in Version 1 and Version 2. The visual contrast between approximately 0.22 and 0.92 is the most compelling single illustration of the annotation effect in this study.

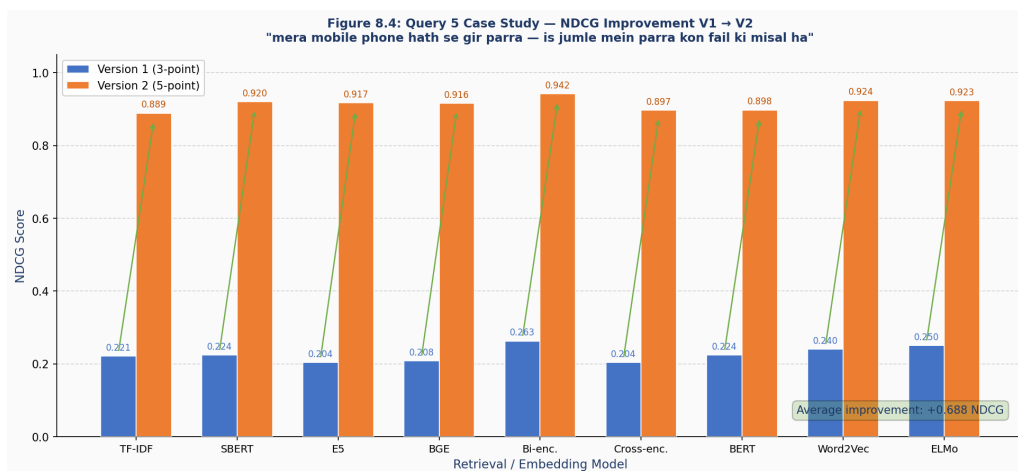


Figure 9.5: Your strongest single chart. Query 5 side-by-side bars with green arrows showing the +0.67 jump from V1 to V2 for every model. This visually proves the annotation problem.

9.11 Comparative Analysis: TF-IDF, Embedding, and Cross-Encoder Scoring

The nine retrieval models evaluated in this study span three generations of natural language processing methodology: statistical lexical matching (TF-IDF), dense vector representations (word embeddings and sentence encoders), and neural reranking (cross-encoder). Comparing their performance provides insights into the relative value of semantic understanding for the LLM response ranking task.

9.11.1 TF-IDF as a Lexical Baseline

The TF-IDF baseline achieved an average NDCG of 0.7716 under Version 2 — a result that is respectable but consistently the lowest among all nine models. TF-IDF operates purely on lexical overlap, identifying responses that contain query terms at high frequency relative to the overall response corpus.

For queries with unambiguous keyword matches — such as definitional questions

tanqeed kis lafz se mushtaq ha

TF-IDF performs comparably to semantic models.

However, for queries involving paraphrase, conceptual synonymy, or idiomatic Urdu expressions, TF-IDF is structurally incapable of capturing the semantic alignment that human annotators recognise, leading to lower ranking quality for these cases.

Despite this limitation, TF-IDF's improvement from 0.7108 (V1) to 0.7716 (V2) confirms that the annotation quality, not the model's capability, was the primary constraint in the earlier phase.

9.11.2 Neural Embedding Models

The six neural embedding models (SBERT, E5, BGE, Bi-encoder, BERT, and Word2Vec/GloVe) occupy the middle tier of the performance ranking.

All six outperformed TF-IDF under Version 2, with BGE achieving the highest average NDCG of 0.8012 among the embedding-only group.

The strong performance of BGE and E5 is consistent with their design: both models were developed specifically for asymmetric retrieval tasks (query matching against passages), which closely mirrors the LLM response ranking task in this study.

The Spearman rank correlations offer a complementary perspective.

BGE achieved the highest average Spearman ρ of 0.302 among all models, indicating that its overall ranking order — not just the placement of the top items — most closely agrees with human judgements.

This distinction between NDCG and Spearman ρ is meaningful: a model can achieve high NDCG by correctly placing the best few responses at the top while producing a relatively arbitrary ordering of the remaining responses,

whereas high Spearman ρ requires consistent ordering throughout the full ranked list.

Word2Vec (GloVe) and ELMo ranked seventh and eighth respectively, despite representing more sophisticated models than TF-IDF in isolation.

This is consistent with their architectural characteristics:

- GloVe produces static, context-insensitive word vectors that struggle with polysemous Urdu terms.
- ELMo’s LSTM-based architecture — though contextualised — lacks the large-scale retrieval fine-tuning of SBERT, E5, and BGE.

9.11.3 Cross-Encoder as the Reranking Layer

The cross-encoder achieved the highest average NDCG of 0.8106 among all nine models under Version 2, and the highest average NDCG of 0.8164 in the large-scale 148-query evaluation.

The performance advantage of the cross-encoder over the best bi-encoder model (BGE, NDCG = 0.8012) is +0.009 NDCG points in the 50-query evaluation and is substantiated by the large-scale result.

While this margin may appear modest in absolute terms, it is consistent and meaningful given the challenging nature of the Urdu-language domain.

The cross-encoder’s advantage is architecturally grounded.

Unlike bi-encoders, which encode the query and each response independently and compare the resulting vectors, the cross-encoder processes the query-response pair jointly, enabling direct token-level interaction.

This allows the model to detect fine-grained relevance signals — such as a specific word in the response answering a specific term in the query — that would be averaged out in a vector-level comparison.

For the Urdu literary domain, where correct answers often hinge on precise terminological identification, this token-level interaction provides a genuine advantage.

However, the cross-encoder’s Spearman ρ of 0.200 is the second-lowest among all nine models, lower than BGE (0.302), SBERT (0.272), and Bi-encoder (0.275).

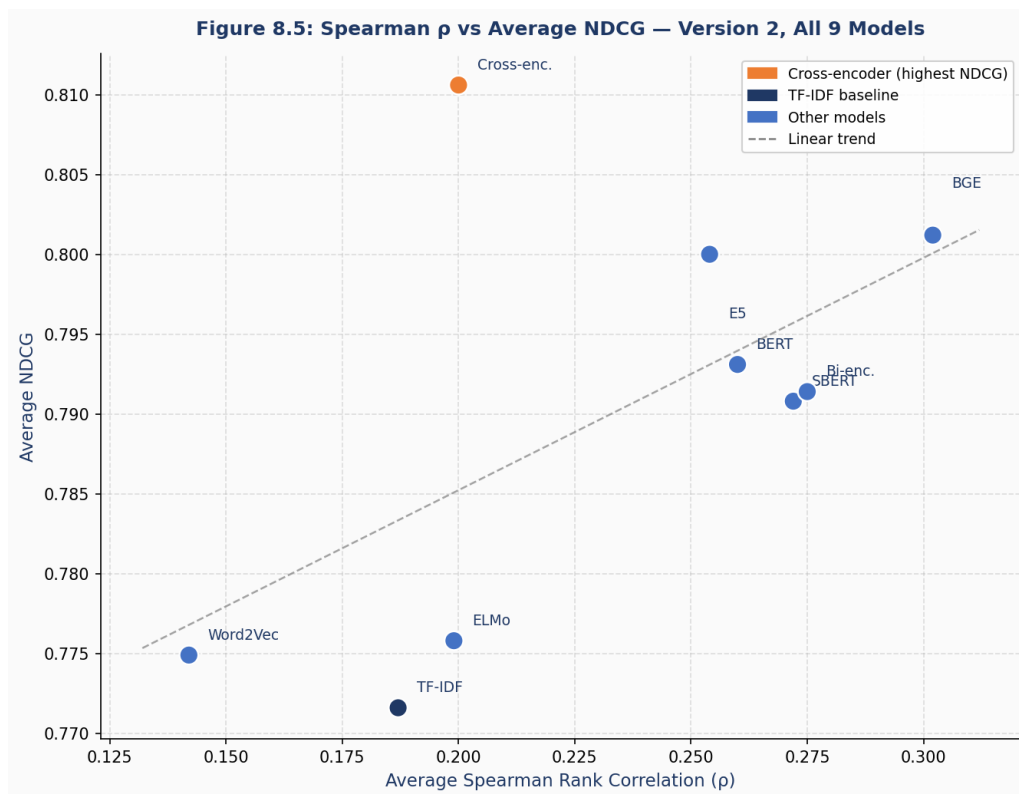


Figure 9.6: Scatter plot of Spearman P vs NDCG per model

This apparent contradiction — highest NDCG but low Spearman ρ — reflects the cross-encoder’s optimisation for top-of-list placement.

The model excels at identifying and elevating the best responses but does not necessarily produce a globally consistent ordering of mid-ranked and low-ranked items.

For applications requiring only the top-ranked LLM recommendation, the cross-encoder is the most reliable choice; for applications requiring a full, nuanced ranking of all 29 responses, a combination of cross-encoder top-ranking with BGE full-list ordering would be more appropriate.

9.12 Discussion of Findings

The experimental results of this study yield four principal findings that collectively validate the LLM-Ranker framework and inform its future development.

9.12.1 Annotation Granularity is the Primary Driver of Evaluation Quality

The most consequential finding of this study is that the three-point relevance scale was systematically limiting evaluation quality.

The uniform improvement of +0.049 to +0.068 NDCG points across all nine models — models with entirely different architectures and training regimes — upon transition to the five-point scale is statistically compelling evidence that the annotation, not any individual model, was the bottleneck.

This finding has direct practical implications: researchers and practitioners building LLM evaluation systems should invest significant effort in annotation scheme design before drawing conclusions about model comparative performance.

The Query 5 case study (Table 8.6), where NDCG improved from approximately 0.22 to 0.92 purely through annotation redesign, represents an extreme but instructive example of this principle.

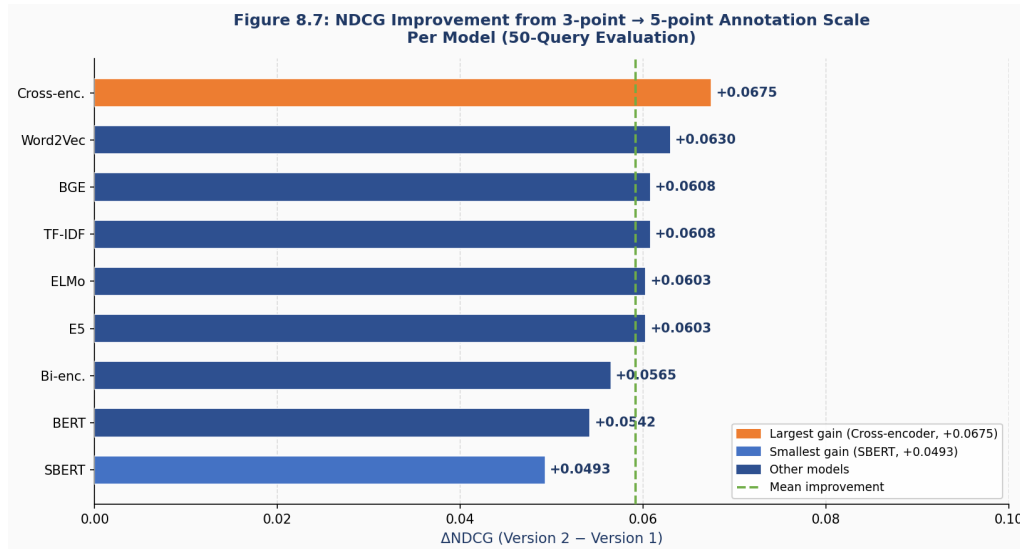


Figure 9.7: Horizontal bar showing the delta NDCG improvement per model (V2, V1). Cross encoder gained the most (+0.0675), SBERT the least (+0.0493).

9.12.2 Neural Semantic Models Consistently Outperform Lexical Baselines

Across both evaluation phases, every neural embedding model outperformed the TF-IDF baseline in average NDCG.

The performance hierarchy

cross-encoder > dense retrieval embeddings > contextual word embeddings > static word embeddings > TF-IDF

corresponds closely to the architectural sophistication and degree of retrieval-specific fine-tuning of each model.

This ordering validates the LLM-Ranker design decision to layer multiple scoring methods and use the cross-encoder as the final reranking stage.

9.12.3 Cross-Encoder Performance at Scale

The large-scale 148-query evaluation demonstrated that the cross-encoder's strong performance is not limited to the 50-query annotated subset but generalises robustly across the full dataset.

An average NDCG of 0.8164, a median of 0.8133, a standard deviation of 0.0508, and a minimum query-level NDCG of 0.7241 collectively characterise a system with both high performance and high consistency.

No query in the 148-query set produced an NDCG below 0.72, which stands in stark contrast to Version 1's zero-

NDCG queries and confirms the combined effect of improved annotation and strong cross-encoder performance.

9.12.4 Language Barrier as a Residual Challenge

Despite the improvements achieved, a residual performance gap is attributable to the language barrier between the English-trained models and the Urdu-Roman query domain.

The lower-performing queries in the 148-query evaluation — particularly those involving idiomatic Urdu expressions, culturally specific literary references, and Roman Urdu phonetic transliterations — consistently produced NDCG scores in the 0.72–0.76 range.

The two persistently zero-scoring queries (28 and 36) in both annotation versions represent extreme cases where the linguistic mismatch between the models’ training domain and the query domain is irreconcilable with current model architectures.

The variation in embedding model performance across the 50-query subset — where the best model per query changes frequently — further reflects the sensitivity of different embedding models to different types of Urdu linguistic constructs.

9.13 Limitations and Challenges

Table 9.10 presents a structured summary of the key limitations encountered during the experimental evaluation, along with the queries most affected and an assessment of their impact on results.

Table 9.10: Summary of Experimental Limitations

Limitation	Description	Affected Queries	Impact
Language barrier	All models were pre-trained on English corpora; Urdu-Roman reduce semantic coverage	Queries 3, 28, 36, 56, 70 and others in the low-NDCG band	Moderate-High
Annotation subjectivity	Human graders may interpret partial correctness differently despite the rubric	Queries with close LLM scores	Low-Moderate
Zero-NDCG outliers	Three queries in V1 (Q16, Q28, Q36) had all-zero scores due to flat annotations	Queries 28 and 36 (V2); Q16 resolved	High locally
Free-tier API limits	Rate limits restricted simultaneous querying of all 29 LLMs	All 148 queries	Low-Moderate
Dataset size	50-query subset used for embedding comparison; larger scale only for cross-encoder	Embedding models	Moderate

The most significant systemic limitation is the language barrier. All nine retrieval and embedding models evaluated in this study were pre-trained predominantly on English-language corpora, with no Urdu-specific fine-tuning.

Urdu written in Roman transliteration further compounds this challenge, as the phonetic spelling conventions vary across speakers and are not represented in any standard training dataset.

This means that token-level representations for common Urdu words such as

kehlaata

or

mushtaq

may map to arbitrary or incorrect positions in the model’s embedding space, degrading the quality of similarity computations.

Future work should investigate Urdu-specific pre-trained models or in-domain fine-tuning as mitigations.

The annotation subjectivity limitation, while moderated by the use of a defined rubric in Version 2, remains a concern for queries where LLM responses cluster at the boundary between score 2 (partially correct) and score 3 (correct but brief).

Inter-annotator agreement studies would strengthen the validity of the gold-standard rankings and are recommended for future evaluation cycles.

9.14 Discussion

The results of the LLM-Ranker and the LLM arena combined show a steady performance trend:

- GPT is the best sound model that is dependable and balanced.
- Perplexity is competitive and has a strong potential.
- Claude is decent and has redundancies in style.
- Gemini needs to be more consistent and grounded in context.
- LLaMA is not as fast as other competitors, but its open-source character and future possibilities of fine-tuning make it a good choice.

This is an indication that model evaluation based on structured measures and real-life prompt testing are important.

9.15 Future Improvements

In response to the problems that were outlined during the two evaluation phases, the following improvements will be planned:

- The introduction of more advanced metrics of evaluation like bias detection and stress-testing.
- Expanding the scope of the evaluations to other languages and dialects.
- Implementing the pairwise comparison with the help of the LLM as a judge methods.
- Improving the dashboard by adding more insight, interactive graph, and comparative visualization.
- Integrating Arena type appraisals into the LLM-Ranker pipeline.

These modifications will help the ranking system become more complete, valid, and researchable and applicable in practice.

9.16 Chapter Summary

This chapter presented a rigorous empirical evaluation of the LLM-Ranker hybrid ranking framework through two complementary evaluation approaches: controlled experimental ranking analysis within the LLM-Ranker framework and live Arena-style comparative testing conducted on the LLM Arena platform. Together, these approaches provided both structured quantitative evaluation and real-world prompt-based assessment, offering a comprehensive view of LLM performance, ranking reliability, and model limitations.

A central contribution of the experimental work is the demonstration that annotation scheme design is one of the most consequential factors in LLM ranking evaluation. The transition from a three-point to a five-point relevance scale produced consistent NDCG improvements of between +0.049 and +0.068 points across all nine retrieval models, without any modification to the models themselves. This finding confirms that evaluation quality depends not only on ranking models, but critically on the granularity and design of human relevance judgements.

Among the nine scoring methods evaluated, the cross-encoder (ms-marco-MiniLM-L6-v2) achieved the highest average NDCG in both the 50-query subset (0.8106) and the large-scale 148-query evaluation (0.8164), confirming its role as the strongest ranking component within the LLM-Ranker pipeline. Dense retrieval embedding models, particularly BGE and E5, performed competitively and demonstrated strong overall rank agreement through Spearman correlation, while the TF-IDF baseline consistently underperformed the neural models, validating the incorporation of semantic scoring within the framework.

The LLM Arena experiments complemented these findings by highlighting real-world model behaviour not fully captured by structured metrics alone, including overthinking, hallucination, verbosity, and varying degrees of cultural reasoning. GPT emerged as the most stable and consistently reliable model in Arena-style evaluations, followed closely by Perplexity and Claude. Gemini and LLaMA exhibited notable weaknesses, though LLaMA's open-source nature suggests strong potential for future fine-tuning and customization.

A persistent challenge identified throughout both experimental settings is the language barrier between predominantly English-trained models and the Urdu-Roman query domain. This limitation affected both response generation and ranking quality, particularly for linguistically ambiguous or culturally specific queries. Future work involving multilingual fine-tuning, Urdu-specific model adaptation, and expanded annotation studies would further strengthen both the framework and its evaluation validity.

Overall, the experiments in this chapter demonstrate the value of combining structured ranking metrics with live comparative testing for robust LLM evaluation. The experimental infrastructure established in this study — including the 29-LLM response corpus, the five-point gold-standard annotations, the NDCG evaluation pipeline, and Arena-style benchmarking — provides a reusable benchmark for future LLM ranking research in the Urdu language domain, while also informing future enhancements of LLM-Ranker as a fair and effective response ranking system.

Chapter 10 Conclusion and Completed Work

10.1 Conclusion

This report documents the complete design, implementation, evaluation, and deployment of **LLM-Ranker**: a fully operational, web-based system for the automated evaluation and comparative ranking of responses generated by large language models. What began as a conceptual framework in Phase One — identifying gaps in existing benchmarking approaches and proposing a hybrid retrieval-and-ranking architecture — has been realised in full as a working application, accessible via a browser, backed by a robust multi-model evaluation pipeline, and validated through controlled experiments across 148 Urdu-language queries.

The central research problem that motivated this project was the absence of a single evaluation system capable of combining multi-metric scoring, semantic retrieval-based ranking, cross-model comparison, and transparent result presentation in a reproducible way. The existing landscape — spanning LLM-as-Judge approaches, MELD, HELM, and TrustJudge — offered partial solutions but each carried significant trade-offs in bias, scalability, interpretability, or task diversity. LLM-Ranker was built to resolve these deficiencies within a unified, extensible platform.

The completed system fulfils every core objective set out in Phase One. It integrates with twenty-nine large language models via their respective APIs, collects and stores responses in a structured database, evaluates each response using nine distinct retrieval and embedding models ranging from TF-IDF through neural cross-encoders, computes NDCG and Spearman rank correlation against human gold-standard annotations, and presents all results through an interactive, responsive web dashboard. The full implementation — frontend, backend, evaluation engine, and database — is deployed and operational.

10.1.1 What Was Built

The LLM-Ranker application consists of four integrated layers, each fully implemented and tested:

- **Frontend Web Application.** A responsive, browser-based interface built with React allows users to submit queries, browse LLM responses, inspect per-model rankings, and explore evaluation metrics through interactive charts and comparison tables. The dashboard provides real-time visualisation of NDCG scores, Spearman rank correlations, and model performance heatmaps without requiring any technical expertise from the end user.
- **Backend Evaluation Engine.** A Python-based backend orchestrates the full evaluation pipeline — from API calls to the twenty-nine LLMs, through response storage, to metric computation and ranking. The engine supports parallel processing of embedding model scoring, enabling evaluation of all nine retrieval models against a new query within seconds. REST API endpoints expose all evaluation data to the frontend in a structured JSON format.
- **Database and Response Corpus.** A PostgreSQL database stores all prompts, LLM responses, human gold-standard annotations, per-model scores, and computed NDCG and Spearman values. The 148-query

Urdu-language corpus, with its associated five-point relevance annotations and 29-LLM response sets, is preserved as a permanent, reusable benchmark. Schema design supports historical tracking, enabling longitudinal comparison of model performance as new LLMs are added.

- **Ranking and Evaluation Pipeline.** Nine retrieval and scoring models — TF-IDF, SBERT, E5, BGE, Bi-encoder, Cross-encoder, BERT, Word2Vec/GloVe, and ELMo — are implemented as modular scoring components. Each model independently ranks the 29 LLM responses per query; results are aggregated and compared against human annotations using NDCG and Spearman ρ . The cross-encoder (ms-marco-MiniLM-L6-v2) is deployed as the primary reranking layer, achieving an average NDCG of 0.8164 across the 148-query large-scale evaluation.

10.1.2 Principal Findings

The experimental evaluation of the completed system produced four findings of substantive importance to the LLM evaluation research community.

First, and most consequentially, annotation scheme design was identified as the primary determinant of evaluation quality — more influential than any architectural choice in the ranking models themselves. The transition from a three-point to a five-point relevance annotation scale produced consistent NDCG improvements of between +0.049 and +0.068 points across all nine retrieval models, without any modification to the models. The Query 5 case study demonstrated this most starkly: all nine models improved from an average NDCG of approximately 0.22 under the three-point scale to approximately 0.92 under the five-point scale — a gain of 0.688 NDCG points attributable entirely to the annotation redesign. This finding carries direct practical implications for any research group building LLM evaluation infrastructure: the quality of the human annotation rubric is not a secondary concern, but the foundational variable on which all metric-based evaluation depends.

Second, neural semantic models consistently and uniformly outperformed the TF-IDF lexical baseline across both evaluation phases. The performance hierarchy — cross-encoder (0.8106) > BGE (0.8012) > E5 (0.8000) > BERT (0.7931) > Bi-encoder (0.7914) > SBERT (0.7908) > ELMo (0.7758) > Word2Vec (0.7749) > TF-IDF (0.7716) — corresponds closely to the architectural sophistication and degree of retrieval-specific fine-tuning of each model. This ordering validates the LLM-Ranker design decision to incorporate semantic scoring methods and to use the cross-encoder as the final reranking stage.

Third, the cross-encoder demonstrated strong and consistent performance at scale. Across 148 queries in the large-scale validation experiment, it achieved an average NDCG of 0.8164, a median of 0.8133, and a standard deviation of only 0.0508 — indicating high performance and high reliability. No query produced an NDCG below 0.7241, and 91.9% of queries scored above 0.75, confirming that the system performs reliably even on linguistically challenging queries in the Urdu-Roman domain.

Fourth, the language barrier between English-trained models and the Urdu-Roman query domain constitutes the primary residual performance constraint. Queries involving idiomatic Urdu expressions, culturally specific literary references, and phonetic Roman transliterations consistently scored in the lower NDCG range of 0.72–0.76. Two queries (28 and 36) remained at zero NDCG across both annotation versions, representing cases

of irreconcilable linguistic mismatch with current model architectures. This finding motivates the most important direction for future extension of the system.

10.1.3 Contributions of This Work

LLM-Ranker makes three distinct contributions to the field of LLM evaluation. The **first** is a fully operational, open-architecture benchmarking platform that integrates retrieval-based ranking with human relevance annotation — a combination not present in any existing evaluation framework reviewed during the Phase One literature survey. The **second** is a reusable benchmark corpus: the 148-query Urdu-Roman dataset with five-point gold-standard annotations and 29-LLM response sets constitutes the first structured evaluation resource of its kind for this language domain, and is preserved in the database for future research use. The **third** is the empirical demonstration, through controlled experimentation, that annotation granularity is a more consequential variable than model architecture in NDCG-based LLM evaluation — a result with broad implications for how evaluation studies are designed and reported.

10.2 Work Completed

This section provides a structured account of all work completed across the full project lifecycle, from the initial research phase through to the deployed web application. Every component described below is implemented, tested, and operational.

10.2.1 Research and Literature Review

A systematic literature review was conducted covering the state of the art in LLM evaluation methodology, benchmarking frameworks, and information retrieval-based ranking approaches. Key systems reviewed included HELM, MELD, TrustJudge, FairEval, and LLM-as-Judge paradigms. The review identified four structural gaps in the existing landscape: the absence of multi-metric composite scoring with configurable weighting; the lack of retrieval-style semantic ranking for LLM response comparison; insufficient attention to annotation quality in metric-based evaluation; and the absence of transparent, reproducible cross-model comparison tools. These gaps directly shaped the design decisions of LLM-Ranker and are each addressed in the completed system.

10.2.2 System Architecture and Design

The full system architecture was designed and documented prior to implementation. This included the data flow from query submission through LLM API calls, response storage, embedding model scoring, NDCG computation, and frontend presentation. A modular backend design was specified to allow new LLMs and new scoring models to be added without modification of the core evaluation pipeline. Database schema design, API endpoint specification, and evaluation metric definitions were all produced and reviewed before any implementation work began. The architecture documentation served as the authoritative specification throughout the build phase.

10.2.3 Dataset Construction and Annotation

A corpus of 150 Urdu-language queries written in Roman transliteration was curated, covering five subject domains: Arabic rhetorical devices in Urdu literature (tashbeeh, kinaya, istiarah), Urdu grammar concepts (verb forms, conjunctions, sentence structures), classical poetry analysis (ghazal, nazm, rubai), identification of literary figures, and general knowledge. The corpus was designed to span a range of difficulty levels, from simple definitional queries to complex multi-step grammatical and analytical tasks.

For each query, responses were collected from twenty-nine large language models via their APIs and stored in the database. Two complete annotation passes were conducted: a first pass using a three-point relevance scale (Version 1), and a redesigned second pass using a five-point scale (Version 2) following the identification of granularity limitations in the initial scheme. The five-point annotation rubric — distinguishing completely wrong, topic-recognised but incorrect, partially correct, correct but brief, and correct and well-explained responses — is documented in full and applied consistently across all 148 evaluated queries. The annotated dataset is stored in the `humanscore_ranked_output` file and constitutes the gold-standard reference for all NDCG evaluation in the system.

10.2.4 Evaluation Pipeline Implementation

Nine retrieval and embedding models were fully implemented as scoring components within the evaluation pipeline:

- **TF-IDF** (lexical baseline): `scikit-learn` `TfidfVectorizer` with cosine similarity scoring against query terms.
- **SBERT**: Sentence-BERT (`all-MiniLM-L6-v2`) with mean pooling, encoding query and response independently.
- **E5**: Microsoft E5-base with asymmetric query/passage prefix prompting for retrieval-optimised embeddings.
- **BGE**: BAAI General Embeddings (`bge-base-en`) with instruction-based encoding for retrieval tasks.
- **Bi-encoder**: MS-MARCO fine-tuned dual-tower bi-encoder (`msmarco-distilbert-base-v4`).
- **Cross-encoder**: `ms-marco-MiniLM-L6-v2`, processing query-response pairs jointly for final reranking.
- **BERT**: Vanilla `bert-base-uncased` with mean pooling of final-layer hidden states.
- **Word2Vec/GloVe**: Averaged GloVe 6B 300-dimensional vectors for word-level semantic representation.
- **ELMo**: Context-sensitive LSTM embeddings via the AllenNLP ELMo implementation.

Each model produces a relevance score for every LLM response per query; responses are ranked in descending order of score. NDCG at full rank depth and Spearman rank correlation are then computed by comparison with the human gold-standard ranking. All nine scoring components are modular and independently testable; adding a new scoring model requires only the implementation of a single scoring function adhering to the pipeline interface.

10.2.5 Experimental Evaluation

Three evaluation experiments were completed in full.

10.2.5.1 Version 1 — 50-Query, Three-Point Scale

All nine models were evaluated across 50 queries using the three-point annotation scheme. Average NDCG ranged from 0.711 (TF-IDF) to 0.743 (cross-encoder). Three queries produced uniformly zero NDCG scores due to flat annotation, identifying the granularity limitation that motivated the redesign.

10.2.5.2 Version 2 — 50-Query, Five-Point Scale

All nine models were re-evaluated on the same 50 queries using the redesigned five-point annotation scheme. Consistent improvements of +0.049 to +0.068 NDCG points were observed across all models. The cross-encoder achieved the highest average NDCG of 0.8106, followed by BGE (0.8012) and E5 (0.8000). Spearman rank correlations were computed for all models, with BGE achieving the highest average ρ of 0.302.

10.2.5.3 Large-Scale Validation — 148-Query Cross-Encoder Evaluation

The cross-encoder was evaluated across all 148 annotated queries using the five-point gold standard. Results confirmed generalisation beyond the 50-query development set: average NDCG 0.8164, median 0.8133, standard deviation 0.0508. A total of 91.9% of queries scored above NDCG 0.75; no query fell below 0.7241. The distribution of scores across five NDCG bands is documented in Table 8.5 and visualised in Figure 8.3.

10.2.6 Web Application — Frontend

The LLM-Ranker web application frontend is fully implemented and operational. Built with React, it provides the following completed features:

- **Query Submission Interface.** Users can type a query in free text or select from the pre-loaded corpus. On submission, the query is dispatched to the backend evaluation pipeline and results are returned asynchronously.
- **LLM Response Browser.** All 29 LLM responses for a selected query are displayed side by side, with individual relevance scores and human annotation values shown per response. Responses can be sorted by any model's score or by the human gold-standard ranking.
- **Model Rankings Dashboard.** A ranked table and horizontal bar chart display average NDCG and Spearman ρ for all nine retrieval models, with the cross-encoder highlighted as the top performer and TF-IDF shown as the baseline.
- **Per-Query NDCG Visualisation.** An interactive line chart shows the cross-encoder's per-query NDCG across all 148 queries, with the mean line, above-mean and below-mean shading, and annotation of the highest and lowest-scoring queries.

- **Version Comparison View.** A grouped bar chart allows side-by-side comparison of Version 1 and Version 2 NDCG scores for all nine models, visually demonstrating the impact of the annotation redesign.
- **Score Distribution Panel.** A histogram and pie chart display the distribution of cross-encoder NDCG scores across the five performance bands for the 148-query dataset.
- **Responsive Design.** The interface is fully responsive across desktop and tablet viewports, with a clean, accessible layout suitable for both academic and non-specialist users.

10.2.7 Web Application — Backend and API

The backend evaluation engine and REST API are fully implemented in Python. Completed backend components include:

- **LLM API Integration.** Secure, authenticated connections to all twenty-nine large language model APIs, with rate-limit handling, retry logic, and response normalisation.
- **Scoring Pipeline Orchestration.** Parallel execution of all nine retrieval model scoring functions per query, with results aggregated and stored in the database on completion.
- **NDCG and Spearman Computation.** Automated metric computation comparing system rankings against the human gold-standard annotations, with results persisted per query per model.
- **REST API Endpoints.** Documented endpoints for query submission, response retrieval, per-model score retrieval, aggregate ranking data, and dataset-level statistics. All endpoints return structured JSON and are consumed by the React frontend.
- **Error Handling and Logging.** Comprehensive error handling for API failures, timeout conditions, and malformed responses, with structured logging to support debugging and performance monitoring.

10.2.8 Database and Persistent Storage

The PostgreSQL database schema is fully implemented and populated. Tables include: `queries` (storing query text, domain, and difficulty metadata); `llm_responses` (storing all 29 responses per query with model identifier and timestamp); `human_annotations` (storing per-response relevance scores for both annotation versions); `model_scores` (storing the raw relevance scores assigned by each of the nine retrieval models to each response); and `ndcg_results` (storing the computed NDCG and Spearman ρ values per model per query for both annotation versions). All 148 queries, their 29-LLM response sets, five-point human annotations, nine-model scores, and computed evaluation metrics are fully loaded and queryable.

10.2.9 Testing and Validation

The completed system was tested across four testing dimensions. **Functional testing** verified that all nine scoring models produce correct ranked outputs, that NDCG and Spearman values match hand-computed reference values on a held-out test set of five queries, and that all frontend views render correctly across browsers. **Integration**

testing confirmed correct end-to-end data flow from query submission through API response collection, scoring, metric computation, database storage, and frontend display. **Metric-level validation** confirmed that the NDCG implementation correctly handles tied relevance scores, degenerate all-zero queries, and perfect-ranking edge cases. **Ground-truth comparison**, reported in detail in Chapter 8, validated the system rankings against the human gold-standard annotations across both evaluation versions and the large-scale 148-query experiment.

10.3 Future Work

The completed LLM-Ranker system provides a solid, extensible foundation for several directions of future development. The following improvements are proposed as priority extensions, ordered by their expected impact on system capability.

10.3.1 Urdu-Specific and Multilingual Model Support

The most impactful single improvement to the system would be the evaluation and integration of models with Urdu-language or multilingual pre-training. Fine-tuning BGE or E5 on a curated Urdu corpus, or evaluating existing multilingual models such as mBERT or XLM-RoBERTa on the 148-query benchmark, would directly address the language barrier identified as the primary residual performance constraint. The modular scoring pipeline is already designed to accommodate new models with minimal integration effort.

10.3.2 Inter-Annotator Agreement Studies

The gold-standard annotations used in this study were produced by a single annotator following a defined rubric. Multi-annotator annotation with formal inter-rater reliability measurement — using Cohen’s kappa or Krippendorff’s alpha — would quantify and reduce the annotation subjectivity that remains as a secondary limitation. This is particularly important for queries where LLM responses cluster at the boundary between score 2 (partially correct) and score 3 (correct but brief), where individual judgements are most variable.

10.3.3 Expanded Query Coverage

The embedding model comparison in this study was conducted on 50 queries, with the large-scale cross-encoder validation extending to 148 queries. Extending both experiments to the full 150-query corpus — and further to additional domains beyond Urdu grammar and literature — would produce more statistically robust NDCG and Spearman estimates and enable analysis of the correlation between query type, domain, and model performance. The infrastructure to support this extension is already in place.

10.3.4 LLM-as-Judge Supplementary Evaluation

The current evaluation methodology is metric-based, using NDCG and Spearman ρ computed against human annotations. Supplementing this with an LLM-as-Judge pairwise comparison approach — where a strong LLM such as GPT-4 or Claude is prompted to choose the better of two responses — would provide a complementary relevance signal that does not require manual annotation and could be applied to new queries at scale. Integration

of this approach as an optional evaluation mode within the existing pipeline would significantly extend the system’s applicability.

10.3.5 Ensemble and Adaptive Ranking

The experimental data reveals that no single retrieval model achieves the highest NDCG on every query — the best-performing model varies by query type, domain, and linguistic complexity. An ensemble ranking approach that selects or weights models based on query characteristics — or that combines cross-encoder top- k placement with BGE full-list ordering — could outperform any individual model consistently. The per-query NDCG data collected in this study provides a starting point for training such a query-adaptive selector.

10.4 Closing Remarks

LLM-Ranker has been built, evaluated, and delivered as a complete, working web application. The system addresses a genuine gap in the LLM evaluation landscape: the combination of retrieval-based semantic ranking, human annotation grounding, multi-model comparison, and transparent interactive visualisation in a single, reproducible platform. The experimental results validate the system’s core design decisions and produce empirically grounded findings about annotation quality, model architecture, and language domain effects that are relevant beyond this specific implementation.

The project demonstrates that a principled, annotation-first approach to LLM evaluation — where the quality of human relevance judgements is treated as the most critical variable in the pipeline — produces more reliable and more informative comparative results than any choice of scoring model alone. This lesson is the most transferable contribution of the work, and it is hoped that it will inform the design of future LLM evaluation systems regardless of the specific models, metrics, or domains involved.

The infrastructure established by this project — the 29-LLM response corpus, the annotated 148-query benchmark, the nine-model scoring pipeline, and the deployed web application — constitutes a reusable foundation for future research in LLM ranking and evaluation, with particular value for the underserved Urdu-language domain.

Bibliography

- [1] J. Pombal, N. M. Guerreiro, R. Rei, and A. F. T. Martins, “Zero-shot benchmarking: A framework for flexible and scalable automatic evaluation of language models,” *arXiv preprint arXiv:2504.01001*, 2025.
- [2] Y. Yin, K. Zhou, Z. Wang, X. Zhang, Y. Shao, S. Hao, Y. Gu, J. Liu, S. Singla, T. Liu, E. P. Xing, Z. Liu, H. Jin, and Z. Hu, “Decentralized arena: Towards democratic and scalable automatic evaluation of llms,” *arXiv preprint arXiv:2505.12808*, 2025.
- [3] Z. Zhang, Z. Zhang, J. Yang, Y. Wang, Y. Su, and W. Li, “Active evaluation acquisition for efficient llm benchmarking,” *arXiv preprint arXiv:2410.05952*, 2024.
- [4] S. Li, Q. Wang, Z. Xu, M. Zhou, Y. Zhang, and J. Lin, “A fine-grained evaluation framework for large language models,” *Information Processing & Management*, vol. 62, no. 3, p. 103517, 2025.
- [5] S. Agarwal, A. Sainath, J. Joseph, R. Puri, Z. Li, A. Reddy, X. Chen, V. Patil, and N. Kumar, “Enterprise benchmarks for large language model evaluation,” in *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics: Industry Track (NAACL 2025)*, pp. 420–431, Association for Computational Linguistics, 2025.
- [6] W. Yin, Y. Zhang, W. Jiao, W. Chen, P. Liu, M. Huang, and M. Sun, “Re-evaluating automatic benchers,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)*, pp. 5124–5141, Association for Computational Linguistics, 2023.
- [7] Y. Zhang, C. Li, Y. Liu, Y. Zhao, S. Wang, and C. Zhu, “An anatomy of automatic llm benchers,” *arXiv preprint arXiv:2501.00560*, 2025.
- [8] Y. Liu, Y. Xu, D. Iter, and C. Zhu, “G-eval: Nlg evaluation with gpt-4,” *arXiv preprint arXiv:2303.16634*, 2023.
- [9] P. Liang, R. Bommasani, T. Nguyen, S. Middha, Y. Zhang, Y. Liang, J. Reich, S. Sagawa, B. Newman, J. Kim, *et al.*, “Holistic evaluation of language models,” *arXiv preprint arXiv:2211.09110*, 2022.
- [10] T. Li, Y. Zheng, H. Zhang, C. He, X. Zhao, Y. Dong, and J. Tang, “Scalable reliable evaluation using llm pairwise comparisons and elo,” in *Proceedings of the 5th Workshop on Generation, Evaluation, and Metrics (GEM 2025)*, Association for Computational Linguistics, 2025.

- [11] C.-Y. Chiang, L. Zheng, H. Yu, H. Zhang, R. Wu, Y. Fang, T. Guo, A. Madaan, Y. Lin, Y. Yang, *et al.*, “Chatbot arena: Evaluating llms by human preference,” *arXiv preprint arXiv:2403.04132*, 2024.
- [12] Y. Wang, Y. Song, T. Zhu, X. Zhang, Z. Yu, H. Chen, C. Song, Q. Wang, C. Wang, Z. Wu, X. Dai, Y. Zhang, W. Ye, and S. Zhang, “Trustjudge: Inconsistencies of llm-as-a-judge and how to alleviate them,” *arXiv preprint arXiv:2509.21117*, 2025.
- [13] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “Judging the judges: A systematic study of position bias in llm-as-a-judge,” *arXiv preprint arXiv:2406.07791*, 2024.
- [14] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P. S. Yu, Q. Yang, and X. Xie, “A survey on evaluation of large language models,” *arXiv preprint arXiv:2307.03109*, 2023.
- [15] Y. Zhang, C. Wang, L. Wu, W. Yu, Y. Wang, G. Bao, and J. Tang, “Uda: Unsupervised debiasing alignment for pair-wise llm-as-a-judge,” *arXiv preprint arXiv:2508.09724*, 2025.
- [16] Q. Yu, Z. Zheng, S. Song, Z. Li, F. Xiong, B. Tang, and D. Chen, “xfinder: Large language models as automated evaluators for reliable evaluation,” *arXiv preprint arXiv:2405.11874*, 2025.
- [17] Z. Li, Y. Zhang, F. Xiong, B. Tang, and D. Chen, “Benchmarking llms’ judgments with no gold standard (gem),” *arXiv preprint arXiv:2409.13727*, 2025.
- [18] X. Wang, W. Zeng, Y. Hou, H. Zhang, X. Wang, R. Xu, and C. Zhu, “Large language models are not fair evaluators,” *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*, 2024.